CKA
CERTIFIED
KUBERNETES
ADMINISTRATOR

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.

EXAM FOCUSED
RESULTS DRIVEN
SUCCESS BUILT

## CERTIFIED KUBERNETES ADMINISTRATOR

# (CKA) MOCK EXAM HANDBOOK

REAL EXAM SIMULATIONS AND PERFORMANCE ASSESSMENT

**REAL EXAM SIMULATIONS**
6 Full-Length Mock Exams with Solutions

**PRACTICE UNDER PRESSURE**
Timed Exams to Build Speed and Confidence

**DETAILED SOLUTIONS**
Step-by-Step Explanations for Every Task

**PERFORMANCE ASSESSMENT**
Track Progress, Identify Weaknesses, and Improve

**SENIOR ADMIN PRINCIPLES**
Think like a Production-Ready Kubernetes Administrator

**6 FULL-LENGTH MOCK EXAMS**
Real Exam Experience

**EXAM-LIKE ENVIRONMENT**
Time. Pressure. Focus.

**BOOST ACCURACY & CONFIDENCE**
Practice. Analyze. Improve.

**MASTER EVERY CKA DOMAIN**
All Objectives Covered

**BUILT FOR SUCCESS**
Your Certification Goal

SCAN TO LEARN MORE

PRACTICE SMART. SOLVE CONFIDENTLY. BECOME CERTIFIED.

www.veriqta.com

veriqta youtube · veriqta github · veriqta linkedin · veriqta x · veriqta instagram · veriqta telegram

LEARN • PRACTICE • VERIQTA • ANALYZE • SUCCEED

# 1. INTRODUCTION TO THE MOCK EXAM HANDBOOK

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

Welcome to your structured path to CKA success.

This handbook contains realistic mock exams designed to simulate the real Certified Kubernetes Administrator exam experience and measure your readiness.

## 1. PURPOSE OF THE HANDBOOK

- Provide real exam-like practice.
- Build confidence under time pressure.
- Identify strengths and weaknesses.
- Improve troubleshooting and problem-solving.
- Prepare you to think like a Kubernetes Administrator.
- Increase accuracy, speed, and consistency.

## REMEMBER

- ✓ Practice like the real exam.
- ✓ Think before you act.
- ✓ Verify everything.
- ✓ Stay calm and focused.
- ✓ Every second counts!

## 2. HOW TO USE THE MOCK EXAMS

1. Read the overview of each mock exam.
2. Start the exam in a clean environment.
3. Complete all tasks within the time limit.
4. Do not skip verification steps.
5. Finish all tasks even if unsure—attempt everything.
6. Review the solutions only after completing the exam.
7. Analyze your performance and learn from mistakes.
8. Re-attempt weak areas until you improve.

## 3. EXAM MINDSET

- Think like an Administrator, not a beginner.
- Understand the problem before solving.
- Use the right tool for the job.
- Favor simple, reliable solutions.
- Validate your work at every step.
- Stay calm—if stuck, move and return.
- Accuracy > Speed, but both are important.

## 4. TIME MANAGEMENT

- Total exam time: 2 Hours (120 Minutes)
- Read all tasks first (5–10 minutes).
- Estimate time per task (avg 5–6 minutes).
- Spend more time on high-confidence tasks.
- If stuck > 2–3 minutes, move on and return.
- Keep 10–15 minutes at the end for review.
- Always verify before moving to the next task.

PLAN YOUR TIME, PROTECT YOUR TIME, USE YOUR TIME.

## 5. SCORING METHODOLOGY

| CRITERIA | DESCRIPTION | WEIGHT |
|---|---|---|
| Task Completed | The task meets all requirements. | 60% |
| Correctness | Solution is correct and works as expected. | 25% |
| Best Practices | Follows Kubernetes best practices. | 10% |
| Efficiency | Completed within a reasonable time. | 5% |

PASSING TARGET: Aim for 80% or higher overall performance.

## YOUR GOAL

By the end of this handbook, you will be:

- ✓ Confident in managing Kubernetes clusters.
- ✓ Faster at solving real-world problems.
- ✓ Ready for the pressure of the real exam.
- ✓ Able to troubleshoot any scenario.
- ✓ Consistent, accurate, and exam-ready.
- ✓ A Certified Kubernetes Administrator!

# 2. REAL CKA EXAM RULES

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

Understand and follow the official rules. Small mistakes can cost you the exam.

## 1. ENVIRONMENT SETUP

- You will get a secure, pre-configured environment
- Access via your exam portal (remote browser)
- One Kubernetes cluster with multiple nodes
- Time limit: 2 Hours (120 Minutes)
- Do not change the environment outside the tasks
- Do not install any additional tools
- Do not reset or restart the cluster
- Do not create extra clusters or namespaces
- Work only on the provided environment
- All tasks must be completed within the environment

Your environment is monitored. Suspicious activity may result in disqualification.

## 2. ALLOWED RESOURCES

- Official Kubernetes Documentation (kubernetes.io/docs)
- kubectl command-line tool (pre-installed)
- Built-in Linux tools (bash, grep, awk, sed, vim, cat, etc.)
- Browser-based exam terminal
- Nothing else is allowed

No external websites, tutorials, blogs, chatbots, or AI tools are allowed.

## 3. BROWSER USAGE

- You will use a proctored browser environment
- Do not open new tabs
- Do not copy, paste to external apps
- Do not share your screen
- Do not switch windows or minimize the exam
- Do not attempt to access unauthorized sites
- Violation may lead to immediate termination

Stay in the exam window at all times.
Focus. Distractions can cost your certification.

## 4. KUBECTL SHORTCUTS (USE SMARTLY)

| | |
|---|---|
| Get resources: | `kubectl get all -A` |
| Describe resource: | `kubectl describe <resource> <name>` |
| Logs: | `kubectl logs <pod> -n <ns> -f` |
| Exec into pod: | `kubectl exec -it <pod> -n <ns> -- /bin/bash` |
| Delete resource: | `kubectl delete <resource> <name> -n <ns>` |
| Edit resource: | `kubectl edit <resource> <name> -n <ns>` |
| Apply manifest: | `kubectl apply -f <file.yaml>` |
| View events: | `kubectl get events -A --sort-by=.lastTimestamp` |

Speed comes from accuracy + repetition.
Practice these until they become second nature.

## 5. COMMON MISTAKES

| READING TASKS TOO FAST | SKIPPING VERIFICATION | WRONG NAMESPACE | INCORRECT RESOURCE TYPE | POOR TIME MANAGEMENT | SYNTAX ERRORS | NOT CHECKING EVENTS / LOGS |
|---|---|---|---|---|---|---|
| • Missing key requirements<br>• Apply wrong solution | • Not checking resource status<br>• Assuming without validating | • Applying in default instead of correct ns<br>• Double-check namespace | • Using Deployment instead of DaemonSet (or vice versa)<br>• Know the difference well | • Spending too long on one task<br>• Leave time for review | • YAML indentation mistakes<br>• Typos in fields or names | • Missing clues in events<br>• Logs help you find root cause |

Slow down to go fast. Accuracy beats speed.

VERIQTA

## 6. REMEMBER

VERIQTA

- Understand the problem
- Plan your approach
- Execute the solution
- Verify the results
- Solve with confidence

VERIQTA

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

# 3. MOCK EXAM STRATEGY

A smart strategy will help you save time, avoid mistakes, and finish the exam with confidence.

## 1. READING TASKS

1. Read the entire task carefully—twice if needed.
2. Identify the goal: what you must achieve.
3. Note the constraints and requirements.
4. Look for keywords: "all", "only", "exactly", "except", "minimum", "must not".
5. Identify the object: pod, deployment, service, namespace, node, etc.
6. Visualize the expected result in your mind.

★ Understand first. Then act.

## 2. PRIORITIZATION

P1 — High Impact

↓

P2 — Medium Impact

↓

P3 — Low Impact

Tackle tasks based on impact and confidence.

- P1: Tasks that block other components or cause outages.
- P2: Important but independent tasks.
- P3: Minor tasks you can do quickly.
- Answer the easy wins fast.
- Leave complex tasks for the second round.

High impact first. Easy wins early.

## 3. VERIFICATION WORKFLOW

1. Apply the changes
2. Check for errors
3. Verify the resource exists
4. Validate the configuration
5. Check related resources
6. Confirm the end result

**USE THIS FLOW EVERY TIME**

1. kubectl apply -f <file> or edit resource
2. kubectl get <resource> -n <ns>
3. kubectl describe <resource> -n <ns>
4. kubectl logs <pod> -n <ns> (if applicable)
5. kubectl get all -n <ns> or related checks
6. Final validation based on task requirements

★ Never assume. Always verify.

## 4. TIME ALLOCATION

Total Time: 120 Minutes

- 20% (24 min) Read all tasks + plan
- 50% (60 min) Execute main tasks
- 20% (24 min) Review + fix
- 10% (12 min) Buffer for tough tasks

Plan early. Execute with focus. Review everything.

## 5. RECOVERY STRATEGY

**STUCK ON A TASK?**

- Skip it.
- Mark it mentally.
- Move to the next task.
- Return later with a fresh mind.

**SOMETHING BROKE?**

- Stop and breathe.
- Understand the current state.
- Revert if needed.
- Start again step by step.

**RUNNING OUT OF TIME?**

- Focus only on critical requirements.
- Ensure the cluster is in a working state.
- Do not chase perfection.

**UNSURE IF IT'S CORRECT?**

- Follow the verification workflow.
- Check multiple times.
- Validate the actual outcome, not assumptions.

**STAY CALM**

- Panic leads to mistakes.
- Control your breathing.
- Trust your preparation.
- You've got this!

EXAMS TEST YOUR SKILLS AND YOUR COMPOSURE. STAY CALM, THINK CLEAR, EXECUTE SMART. VERIQTA

★ STRATEGY + FOCUS + VERIFICATION = CKA SUCCESS ★

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

# 4. MOCK EXAM 1 OVERVIEW

Mock Exam 1 is designed to simulate the real CKA exam experience. It covers all exam domains with a balanced mix of task types and real-world Kubernetes scenarios.

**TOTAL TASKS 20**

## 1. EXAM BLUEPRINT

This mock exam follows the official CKA exam blueprint and weight distribution.

| DOMAIN | WEIGHT | TASK DISTRIBUTION |
|---|---|---|
| Cluster Architecture, Installation & Configuration | 25% | 5 Tasks |
| Workloads & Scheduling | 25% | 5 Tasks |
| Services & Networking | 20% | 4 Tasks |
| Storage | 10% | 2 Tasks |
| Troubleshooting | 20% | 4 Tasks |
| **TOTAL** | **100%** | **20 TASKS** |

## 2. DOMAINS COVERED

This exam covers all five CKA domains with practical, production-like scenarios.

**Cluster Architecture, Installation & Configuration**
- Cluster setup, kubeadm, components, configuration, upgrades

**Workloads & Scheduling**
- Pods, Deployments, DaemonSets, Jobs, CronJobs, Scheduling, Affinity, Taints

**Services & Networking**
- Services, DNS, Ingress, NetworkPolicies, CoreDNS, CNI

**Storage**
- PV, PVC, StorageClasses, ConfigMaps, Secrets, Volume types

**Troubleshooting**
- Debugging, Logs, Events, Resource issues, Network problems, Application failures

## 3. DIFFICULTY

Mock Exam 1 is designed to be moderately challenging, similar to the real CKA exam.

MODERATE (REAL EXAM SIMILARITY)

## 4. TIME ALLOCATION

Total exam time: 120 Minutes

| 0 – 20 min | 20 – 90 min | 90 – 110 min | 110 – 120 min |
|---|---|---|---|
| Read & Understand All Tasks | Execute & Solve Main Tasks | Review & Fix Issues | Final Verification |
| Read every task carefully. Plan your approach. | Focus on high-impact tasks first. Work systematically. | Re-check solutions. Fix errors and optimize. | Verify all tasks. Ensure nothing is left incomplete. |

## 5. PASSING CRITERIA

To pass Mock Exam 1 and be on track for the real exam, aim to meet the following criteria.

- **SCORE** — 80% or Higher (16/20 tasks or more)
- **COMPLETION** — 100% of tasks attempted
- **TIME** — Complete within 120 minutes
- **ACCURACY** — Solutions must meet all requirements of each task.

Consistent practice + smart strategy + focus = CKA Success!

VERIQTA

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.



# MOCK EXAM 1 (TASKS 1 TO 5)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 1 TASK 1: CLUSTER ARCHITECTURE AND INSTALLATION

- Configure kubectl context
- Verify cluster health

**REQUIREMENTS**

- Use the provided kubeconfig
- Set the context to 'exam-cluster'
- Ensure all control plane and worker nodes are Ready

**EXAMPLE COMMANDS**

```
kubectl config use-context exam-cluster
kubectl config current-context
kubectl get nodes
kubectl cluster-info
kubectl get componentstatuses
```

**VERIFY**

- Context set
- Nodes Ready
- Cluster healthy

## 2 TASK 2: WORKLOADS

- Create a Deployment
- Scale replicas
- Verify rollout

**REQUIREMENTS**

- Create deployment named 'webapp' using image nginx:1.25
- Set replicas to 3
- Ensure rollout is successful

**EXAMPLE COMMANDS**

```
kubectl create deployment webapp --image=nginx:1.25
kubectl get deployments
kubectl scale deployment webapp --replicas=3
kubectl rollout status deployment/webapp
kubectl get pods -l app=webapp
```

**VERIFY**

- Deployment created
- 3 replicas running
- Rollout successful

## 3 TASK 3: SERVICES AND NETWORKING

- Expose the application using a ClusterIP Service
- Verify connectivity

**REQUIREMENTS**

- Create Service 'webapp-svc' of type ClusterIP
- Select pods with label app=webapp
- Verify service is reachable from another pod

**EXAMPLE COMMANDS**

```
kubectl expose deployment webapp --name=webapp-svc
--type=ClusterIP --port=80 --target-port=80
kubectl get svc
kubectl run test-pod --rm -it --image=busybox
--restart=Never -- /bin/sh
# Inside test-pod
wget -qO- http://webapp-svc
```

**VERIFY**

- Service created
- ClusterIP assigned
- Connectivity successful

## 4 TASK 4: CONFIGURATION

- Create a ConfigMap
- Mount it into an existing Pod

**REQUIREMENTS**

- Create ConfigMap named app-config with key app.properties
- Mount it to existing Pod 'webapp' at /etc/config

**EXAMPLE COMMANDS**

```
kubectl create configmap app-config --from-literal=
app.properties=mode=production"
kubectl get configmaps
kubectl set volume deployment/webapp \
  --add --name=config-vol --configmap=name=app-config
kubectl set volume deployment/webapp \
  --add --mount-path=/etc/config --name=config-vol
kubectl rollout status deployment/webapp
```

**VERIFY**

- ConfigMap created
- Volume mounted
- Pod running with config

## 5 TASK 5: SCHEDULING

- Apply node labels
- Schedule a Pod to a specific node using nodeSelector

**REQUIREMENTS**

- Label node 'worker-1' with disktype=ssd
- Create pod 'db-pod' using nodeSelector to run only on nodes with disktype=ssd

**EXAMPLE COMMANDS**

```
kubectl label node worker-1 disktype=ssd
kubectl get nodes --show-labels
# pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
   name: db-pod
spec:
   containers:
    - name: db
      image: nginx
   nodeSelector:
     disktype: ssd
kubectl apply -f pod-definition.yaml
kubectl get pods -o wide
```

**VERIFY**

- Node labeled
- Pod scheduled on worker-1
- Pod status Running

### EXAM TIP

Read each task carefully. Follow the requirements exactly. Always verify your work before moving to the next task.

### DOMAINS COVERED ON THIS PAGE

- Cluster Architecture
- Workloads
- Services & Networking
- Configuration
- Scheduling

### TIME REMINDER

You have 120 minutes for the entire exam. Manage your time wisely!

VERIQTA
LEARN. PRACTICE. ANALYZE. SUCCEED.



# MOCK EXAM 1 (TASKS 6 TO 10)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 6 TASK 6: STORAGE

- Create a PersistentVolume
- Create a PersistentVolumeClaim
- Bind successfully

**REQUIREMENTS**

- Create PV with hostPath storage
- Create PVC requesting the storage
- PVC must be Bound successfully

**EXAMPLE COMMANDS**

```
kubectl create -f pv-storage.yaml
kubectl create -f pvc-storage.yaml
kubectl get pv
kubectl get pvc
kubectl describe pvc app-pvc
```

**VERIFY**

- PV created
- PVC created
- PVC status Bound
- Pod able to use the volume

## 7 TASK 7: SECURITY

- Create a ServiceAccount
- Assign appropriate RBAC permissions

**REQUIREMENTS**

- Create ServiceAccount for application
- Create Role/ClusterRole with required permissions
- Bind the role using RoleBinding/ClusterRoleBinding

**EXAMPLE COMMANDS**

```
kubectl create serviceaccount app-sa
kubectl create role app-role --verb=get,list,watch \
--resource=pods,services
kubectl create rolebinding app-rb --role=app-role \
--serviceaccount=default:app-sa
kubectl auth can-i get pods --as=system:serviceaccount:\
default:app-sa
```

**VERIFY**

- ServiceAccount created
- Role/Binding created
- Permissions working

## 8 TASK 8: NAMESPACES

- Create a namespace
- Move application resources into it

**REQUIREMENTS**

- Create namespace named 'app-space'
- Move Deployment, Service and other resources into the namespace
- Verify all resources exist in the namespace

**EXAMPLE COMMANDS**

```
kubectl create namespace app-space
kubectl get ns
kubectl get all -n default
kubectl get all -n app-space
kubectl delete deployment webapp -n default
kubectl delete service webapp-svc -n default
kubectl apply -f deployment.yaml -n app-space
kubectl apply -f service.yaml -n app-space
```

**VERIFY**

- Namespace created
- Resources moved
- All resources visible in app-space

## 9 TASK 9: INGRESS

- Configure an Ingress resource
- Validate routing

**REQUIREMENTS**

- Create Ingress to expose webapp service
- Host: webapp.local
- Path: / → service
- Verify routing works

**EXAMPLE COMMANDS**

```
kubectl apply -f ingress.yaml
kubectl get ingress
kubectl describe ingress webapp-ing
kubectl get svc webapp-svc
# Test routing (if /etc/hosts configured)
curl -H "Host: webapp.local" http://<ingress-ip>/
```

**VERIFY**

- Ingress created
- Rule configured
- Routing successful (HTTP 200 OK)

## 10 TASK 10: JOBS

- Create a CronJob
- Verify execution schedule

**REQUIREMENTS**

- Create CronJob to run every 5 minutes
- Command should write timestamp to a file
- Verify jobs are created on schedule

**EXAMPLE COMMANDS**

```
kubectl apply -f cronjob.yaml
kubectl get cronjob
kubectl get jobs
kubectl logs -l job-name=<job-name>
kubectl describe cronjob app-cron
```

**VERIFY**

- CronJob created
- Jobs running on schedule
- Logs show expected output

**EXAM TIP**

Plan before you act.
Read the requirements carefully, follow the steps, and verify everything.

**DOMAINS COVERED ON THIS PAGE**

Storage · Security · Namespaces · Networking (Ingress) · Workloads (Jobs)

**TIME REMINDER**

Keep track of time.
Aim to complete 10 tasks in 60 minutes to stay on schedule.

# MOCK EXAM 1 (TASKS 11 TO 15)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 11 TASK 11: DAEMONSETS

- Deploy a DaemonSet to every worker node
- Ensure the Pod runs on all nodes

### REQUIREMENTS

- Create DaemonSet named node-monitor using image busybox
- DaemonSet should run on all worker nodes
- Verify one Pod per node

### EXAMPLE COMMANDS

```
kubectl create daemonset node-monitor \
--image=busybox:1.36 \
--restart=Always \
-- /bin/sh -c "while true; do sleep 3600; done"
kubectl get daemonset node-monitor
kubectl get pods -o wide | grep node-monitor
```

### VERIFY

- DaemonSet created
- Pods running on all nodes
- One Pod per node

## 12 TASK 12: NETWORK POLICIES

- Restrict Pod communication using NetworkPolicy
- Verify only allowed traffic is permitted

### REQUIREMENTS

- Deny all ingress to app pods by default
- Allow only traffic from pods with label role=frontend
- Verify policy enforcement

### EXAMPLE COMMANDS

```
kubectl apply -f network-policy.yaml
kubectl get networkpolicies
kubectl describe networkpolicy allow-frontend
kubectl run test-pod --image=busybox -it --rm --
/bin/sh
# test connectivity using wget or nc
```

### VERIFY

- Policy created
- Only allowed traffic is permitted
- Denied traffic is blocked

## 13 TASK 13: SECRETS

- Create a Secret containing credentials
- Inject it into an application securely

### REQUIREMENTS

- Create Secret named db-credentials
- Use it as environment variables in a Deployment
- Ensure the Secret is not exposed in plaintext

### EXAMPLE COMMANDS

```
kubectl create secret generic db-credentials \
--from-literal=username=admin \
--from-literal=password=SecurePass123
kubectl create deployment app --image=nginx \
--env-from=secret/db-credentials
kubectl describe pod -l app=app | grep -A3 Env:
```

### VERIFY

- Secret created
- Env vars loaded from Secret
- No plaintext in pod spec

## 14 TASK 14: TROUBLESHOOTING

- Resolve a CrashLoopBackOff Pod
- Restore application health

### REQUIREMENTS

- Identify the root cause
- Fix configuration or code issue
- Ensure the Pod is running successfully

### EXAMPLE COMMANDS

```
kubectl get podes
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
kubectl edit deployment <deployment-name>
kubectl rollout status deployment/<deployment-name>
```

### VERIFY

- Pod no longer in CrashLoopBackOff
- Pod status is Running
- Application healthy

## 15 TASK 15: NODE ADMINISTRATION

- Drain a node
- Perform maintenance
- Uncordon the node after maintenance

### REQUIREMENTS

- Drain node named worker-1
- Ensure workloads move to other nodes
- Uncordon the node after maintenance

### EXAMPLE COMMANDS

```
kubectl get nodes
kubectl drain worker-1 --ignore-daemonsets \
  --delete-emptydir-data
kubectl get pods -o wide
# perform maintenance on node
kubectl uncordon worker-1
kubectl get nodes
```

### VERIFY

- Node drained
- Pods evicted
- Node uncordoned
- Node Ready

### EXAM TIP

Use a systematic approach: Understand → Plan → Execute → Verify. Always validate each step before moving on.

### DOMAINS COVERED ON THIS PAGE

- Workloads
- Services & Networking (Network Policies)
- Security (Secrets)
- Troubleshooting
- Cluster Architecture, Installation & Configuration
- Cluster Maintenance (Node Administration)

### TIME REMINDER

You are halfway through! Stay focused and pace yourself. Aim to complete tasks 11–15 in 30 minutes.

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.



# MOCK EXAM 1 (TASKS 16 TO 20)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 16 TASK 16: CLUSTER UPGRADE

- Upgrade the control plane
- Verify cluster version

**REQUIREMENTS**

- Upgrade control plane using kubeadm
- Maintain cluster availability
- Verify all components are healthy after upgrade

**EXAMPLE COMMANDS**

```
kubectl drain control-plane --ignore-daemonsets
kubeadm upgrade plan
kubeadm upgrade apply v1.30.0
kubectl uncordon control-plane
kubectl get nodes
kubectl get componentstatuses
kubectl version --short
```

**VERIFY**

- Control plane upgraded
- Nodes Ready
- All components healthy
- Version updated

## 17 TASK 17: TROUBLESHOOTING

- Fix a Service that has no available Endpoints

**REQUIREMENTS**

- Identify why Endpoints are missing
- Fix the root cause
- Restore Service functionality

**EXAMPLE COMMANDS**

```
kubectl get svc <svc-name>
kubectl get endpoints <svc-name>
kubectl describe svc <svc-name>
kubectl get pods -l app=<app-label>
kubectl describe pods <pod-name>
kubectl logs <pod-name>
kubectl edit svc <svc-name>
```

**VERIFY**

- Endpoints show addresses
- Service reachable
- Application working

## 18 TASK 18: APPLICATION SCALING

- Perform a rolling update
- Roll back after detecting a failure

**REQUIREMENTS**

- Update image to a new version
- Verify rollout success
- Simulate failure or use bad image
- Roll back to previous version
- Verify rollback success

**EXAMPLE COMMANDS**

```
kubectl set image deployment/webapp webapp=nginx:1.26
kubectl rollout status deployment/webapp
kubectl rollout undo deployment/webapp
kubectl rollout history deployment/webapp
kubectl rollout history deployment/webapp --revision=2
kubectl rollout undo deployment/webapp --to-revision=2
kubectl get pods -l app=webapp
```

**VERIFY**

- New version deployed
- Rollback completed
- Pods running old stable version
- Application healthy

## 19 TASK 19: STORAGE TROUBLESHOOTING

- Diagnose a PVC stuck in Pending
- Restore normal operation

**REQUIREMENTS**

- Identify reason PVC is Pending
- Fix storage class, PV, permissions or quota issues
- Ensure PVC binds successfully
- Pod using PVC runs successfully

**EXAMPLE COMMANDS**

```
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get storageclass
kubectl describe storageclass <sc-name>
kubectl get pv
kubectl describe pv <pv-name>
kubectl get events --sort-by=.metadata.creationTimestamp
```

**VERIFY**

- PVC Bound
- Pod Running
- Application accessible

## 20 TASK 20: PRODUCTION SCENARIO

- Deploy a complete multi-tier application
- Configure networking
- Configure persistent storage
- Secure access
- Verify application functionality
- Validate all resources before submission

**REQUIREMENTS**

- Deploy frontend, backend and database tiers using Deployments
- Expose frontend via Ingress
- Use Services for internal communication
- Use PersistentVolumes/PVCs for database
- Apply NetworkPolicy to restrict traffic
- Use Secrets for sensitive configuration
- Ensure high availability and health
- Verify end-to-end functionality
- Validate all resources are in desired state

**EXAMPLE COMMANDS (SAMPLE)**

```
# Deployments & Services
kubectl apply -f deployments/
kubectl apply -f services/
# Ingress
kubectl apply -f ingress/
# Storage
kubectl apply -f storage/
# Secrets & Configs
kubectl apply -f configs/
# Network Policies
kubectl apply -f network-policies/
# Verify
kubectl get all -A
kubectl get pvc,pv,ingress,networkpolicy,secret
```

**VERIFY**

- All components running
- Ingress reachable
- Data persists after restart
- Network policies working
- Secrets mounted
- Application fully functional
- All resources validated

**EXAM TIP**

Simulate real exam pressure. Stay calm, read carefully, verify everything, and don't skip verification.

**DOMAINS COVERED ON THIS PAGE**

- Cluster Maintenance (Upgrade)
- Troubleshooting
- Workloads & Scaling
- Storage
- Networking (Ingress & Policies)
- Security (Secrets)

**TIME REMINDER**

You are halfway through! Aim to complete 15 tasks in 45 minutes to leave enough time for the rest.

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.



# MOCK EXAM 1 SOLUTIONS

CKA CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 1 CLUSTER ARCHITECTURE AND INSTALLATION

**Key Points**

- Configure kubectl context
- Verify cluster health

**Example Commands**

```
kubectl config use-context exam-cluster
kubectl get nodes
kubectl get componentstatuses
```

## 2 WORKLOADS

**Key Points**

- Create Deployment
- Scale replicas
- Verify rollout

**Example Commands**

```
kubectl create deploy webapp \
--image=nginx:1.25
kubectl scale deploy webapp --replicas=3
kubectl rollout status deploy/webapp
```

## 3 SERVICES & NETWORKING

**Key Points**

- Expose using ClusterIP Service
- Verify connectivity

**Example Commands**

```
kubectl expose deploy webapp \
 --name=webapp-svc --type=ClusterIP \
 --port=80 --target-port=80
kubectl get svc
kubectl run test-pod --rm -it \
 --image=busybox --restart=Never --
 wget -qO- http://webapp-svc
```

## 4 CONFIGURATION

**Key Points**

- Create ConfigMap
- Mount into existing Pod

**Example Manifest (ConfigMap)**

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  app.properties: |
    app.name=webapp
    app.env=production
```

## 5 SCHEDULING

**Key Points**

- Apply node labels
- Schedule Pod using nodeSelector

**Example Commands / Manifest**

```
kubectl label node worker-1 disktype=ssd
---
apiVersion: v1
kind: Pod
metadata:
  name: db-pod
spec:
  nodeSelector:
    disktype: ssd
  containers:
  - name: db
    image: nginx
```

## 6 STORAGE

**Key Points**

- Create PV and PVC
- Bind successfully

**Example Commands / Manifests**

```
# PV
kubectl apply -f pv.yaml
# PVC
kubectl apply -f pvc.yaml
kubectl get pv,pvc
```

## 7 SECURITY

**Key Points**

- Create ServiceAccount
- Assign RBAC permissions

**Example Commands / Manifests**

```
kubectl apply -f sa.yaml
kubectl apply -f role.yaml
kubectl apply -f rolebinding.yaml
kubectl get sa,role,rolebinding
```

## 8 NAMESPACES

**Key Points**

- Create namespace
- Move resources into it

**Example Commands**

```
kubectl create ns app-space
kubectl get ns
kubectl get all -n default
kubectl get all -n app-space
```

## 9 INGRESS

**Key Points**

- Configure Ingress
- Validate routing

**Example Manifest (Ingress)**

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webapp-ing
spec:
  rules:
  - host: webapp.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: webapp-svc
            port:
              number: 80
```

## 10 JOBS

**Key Points**

- Create CronJob
- Verify schedule

**Example Manifest (CronJob)**

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: job-name
spec:
  schedule: "*/5 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: job
            image: busybox
            command: ["/bin/sh","-c","date"]
          restartPolicy: OnFailure
```

## 11 DAEMONSETS

**Key Points**

- Deploy DaemonSet on all worker nodes

**Example Manifest (DaemonSet)**

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-monitor
spec:
  selector:
    matchLabels:
      app: node-monitor
  template:
    metadata:
      labels:
        app: node-monitor
    spec:
      containers:
      - name: monitor
        image: busybox
        command: ["/bin/sh","-c","sleep 3600"]
```

## 12 NETWORK POLICIES

**Key Points**

- Restrict communication
- Verify allowed traffic

**Example Manifest (NetworkPolicy)**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
spec:
  podSelector:
    matchLabels:
      role: frontend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: backend
  - - podSelector: - portttpp.Pod(ao)
```

## 13 SECRETS

**Key Points**

- Create Secret
- Inject into application

**Example Commands**

```
kubectl create secret generic \
db-credentials --from-literal=username=admin \
--from-literal=password=SecretPass123

# Use in Deployment
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: username
```

## 14 TROUBLESHOOTING

**Key Points**

- Resolve CrashLoopBackOff
- Restore application health

**Example Commands**

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
kubectl edit deploy <deploy-name>
kubectl rollout status deploy/<deploy-name>
```

## 15 NODE ADMINISTRATION

**Key Points**

- Drain node
- Uncordon after maintenance

**Example Commands**

```
kubectl drain worker-1
--ignore-daemonsets \
--delete-emptydir-data
# perform maintenance
kubectl uncordon worker-1
kubectl get nodes
```

## 16 CLUSTER UPGRADE

**Key Points**

- Upgrade control plane
- Verify cluster version

**Example Commands**

```
kubectl drain control-plane \
 --ignore-daemonsets --delete-emptydir-data
kubeadm upgrade plan
kubeadm upgrade apply v1.30.0
kubectl uncordon control-plane
kubectl get nodes
kubectl version --short
```

## 17 TROUBLESHOOTING

**Key Points**

- Fix Service with no available Endpoints

**Example Commands**

```
kubectl get svc <svc-name>
kubectl get endpoints <svc-name>
kubectl describe svc <svc-name>
kubectl get pods -l app=<app-label>
kubectl describe pods <pod-name>
# Fix selector/deployment if needed
```

## 18 APPLICATION SCALING

**Key Points**

- Rolling update
- Rollback on failure

**Example Commands**

```
kubectl set image deploy/webapp \
  webapp=nginx:1.26
kubectl rollout status deploy/webapp
kubectl rollout undo deploy/webapp
kubectl rollout history deploy/webapp
kubectl rollout undo deploy/webapp --to-revision=2
```

## 19 STORAGE TROUBLESHOOTING

**Key Points**

- PVC stuck in Pending
- Restore normal operation

**Example Commands**

```
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get storageclass
kubectl describe sc <sc-name>
kubectl get pv
kubectl describe pv <pv-name>
# Fix storageclass, permissions or quota
# Recreate PVC if necessary
```

## 20 PRODUCTION SCENARIO

**Key Points**

- ☆ Dpploy multi-tier application
- ☆ Configure networking
- ☆ Configure persistent storage
- ☆ Secure access
- ☆ Verify functionality
- ☆ Validate all resources

**Checklist**

- ☑ All Deployments / Pods Running
- ☑ Services reachable
- ☑ Ingress working (HTTP 200 OK)
- ☑ PVCs Bound and mounted
- ☑ Secrets & NetworkPolicies active
- ☑ Resources in desired state

## SOLUTION TIPS

1. Always read the task carefully.
2. Validate after every step.
3. Use kubectl explain for help.
4. Think, Plan, Execute, Verify.

## DOMAINS COVERED IN EXAM 1

- Cluster Architecture
- Workloads
- Services & Networking
- Configuration
- Scheduling
- Storage
- Security
- Namespaces
- Ingress
- Jobs
- DaemonSets
- Network Policies
- Troubleshooting
- Node Management
- Upgrades & Maintenance

## SUCCESS PRINCIPLES

- ☆ Accuracy over speed
- ☆ Verify before moving on
- ☆ Recover smartly, not blindly
- ☆ Stay calm and systematic



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.



# MOCK EXAM 1 DETAILED EXPLANATIONS

CKA CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 1 CLUSTER ARCHITECTURE AND INSTALLATION

Configure kubectl to use the correct context and verify the cluster is healthy. Ensure all control plane components and nodes are Ready.

**Key Takeaway:** Correct context and healthy cluster are the foundation for all tasks.

## 2 WORKLOADS

Create a Deployment for the application, scale replicas to the required number and verify the rollout is successful.

**Key Takeaway:** Understand Deployments, scaling and how rollouts work.

## 3 SERVICES & NETWORKING

Expose the application using a ClusterIP Service and verify that Pods can communicate through the Service.

**Key Takeaway:** Services provide stable networking and service discovery.

## 4 CONFIGURATION

Create a ConfigMap and mount it into an existing Pod as a volume so the application can read configuration data.

**Key Takeaway:** ConfigMaps decouple configuration from container images.

## 5 SCHEDULING

Apply node labels to the target node and use nodeSelector to schedule a Pod to that specific node.

**Key Takeaway:** Labels and nodeSelector control where Pods run.

## 6 STORAGE

Create a PersistentVolume (PV) and a PersistentVolumeClaim (PVC) and ensure they bind successfully.

**Key Takeaway:** PVs are cluster resources; PVCs are requested by users. Binding must succeed.

## 7 SECURITY

Create a ServiceAccount and assign RBAC permissions using Role/ClusterRole and RoleBinding/ClusterRoleBinding.

**Key Takeaway:** RBAC controls what actions a ServiceAccount can perform.

## 8 NAMESPACES

Create a namespace and move existing application resources into it to achieve isolation and better organization.

**Key Takeaway:** Namespaces provide logical isolation within a cluster.

## 9 INGRESS

Configure an Ingress resource to route external traffic to Services and validate that routing works.

**Key Takeaway:** Ingress manages HTTP/HTTPS access to Services.

## 10 JOBS

Create a CronJob to run a task on a schedule and verify that Jobs are created and completed.

**Key Takeaway:** CronJob manages recurring, time-based Jobs.

## 11 DAEMONSETS

Deploy a DaemonSet so that one Pod runs on every worker node. Verify all nodes have the required Pod.

**Key Takeaway:** DaemonSets drive node-level agents and system services.

## 12 NETWORK POLICIES

Create NetworkPolicy to restrict Pod communication and verify only allowed traffic is permitted.

**Key Takeaway:** NetworkPolicies enforce firewall-like rules between Pods.

## 13 SECRETS

Create a Secret and securely inject it into an application (using env variables or volume mounts).

**Key Takeaway:** Secrets store sensitive data safely (base64 encoded, not encrypted).

## 14 TROUBLESHOOTING

Investigate a Pod in CrashLoopBackOff, find the root cause (logs, events) and fix the issue to restore a healthy state.

**Key Takeaway:** Always check logs, describe, and events to find the root cause.

## 15 NODE ADMINISTRATION

Drain a node to safely evict Pods, perform maintenance, then uncordon the node to make it schedulable again.

**Key Takeaway:** Draining protects workloads during maintenance.

## 16 CLUSTER UPGRADE

Upgrade the control plane components using kubeadm and verify the cluster version is updated.

**Key Takeaway:** Control plane upgrades must be done in the correct order and verified.

## 17 TROUBLESHOOTING

Fix a Service that has no available Endpoints by identifying the issue (selector, Pods, readiness) and restoring endpoints.

**Key Takeaway:** Services rely on healthy Pods and correct selectors to have Endpoints.

## 18 APPLICATION SCALING

Perform a rolling update to a new version and roll back to the previous version after a failure is detected.

**Key Takeaway:** Rolling updates minimize downtime; rollbacks restore stability quickly.

## 19 STORAGE TROUBLESHOOTING

Diagnose a PVC stuck in Pending by checking events, StorageClass, PV availability and fix the problem.

**Key Takeaway:** PVC Pending means binding requirements are not satisfied.

## 20 PRODUCTION SCENARIO

Deploy a complete multi-tier application with networking, persistent storage and security. Verify functionality and validate all resources.

**Key Takeaway:** Combines all domains. Think end-to-end and validate before submission.

## DOMAIN COVERAGE IN MOCK EXAM 1

| | |
|---|---|
| Cluster Architecture & Installation | 5% |
| Workloads & Scheduling | 15% |
| Services & Networking | 20% |
| Storage | 10% |
| Configuration & Secrets | 10% |
| Security | 10% |
| Troubleshooting | 10% |
| Cluster Maintenance | 5% |
| Jobs & Automation | 5% |
| **TOTAL** | **100%** |

## HOW TO USE THIS PAGE

- Read each explanation carefully.
- Maintain the key takeaway for quick recall during the exam.
- Recreate each task in a practice environment.
- Understand the "why" behind every solution.
- Review weak areas and retake the exam sections.

## SUCCESS MINDSET

- Plan your time
- Stay calm
- Verify every step
- Double-check before submission
- You've got this!

## EXAM TIP

Practice each task multiple times. Speed and accuracy come with repetition and understanding.

## REMEMBER

The real exam tests your ability to think, troubleshoot and solve in a production environment.

## FOCUS AREAS

Troubleshooting, Networking, Storage and Cluster Maintenance are high-weight areas.

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.



# MOCK EXAM 2 OVERVIEW

CKA CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 1. EXAM BLUEPRINT

| Domain | % Weightage |
|---|---|
| Cluster Architecture, Installation & Configuration | 15% |
| Workloads & Scheduling | 20% |
| Services & Networking | 20% |
| Storage | 10% |
| Security & Access Control | 10% |
| Troubleshooting | 10% |
| Cluster Maintenance | 5% |
| Configuration & Best Practices | 10% |
| TOTAL | 100% |

## 2. DOMAINS COVERED

- Cluster Setup, kubectl, API access, high availability
- Deployments, ReplicaSets, StatefulSets, DaemonSets, Jobs
- Services, DNS, Ingress, Network Policies, CoreDNS
- PersistentVolumes, PersistentVolumeClaims, StorageClasses
- RBAC, ServiceAccounts, Secrets, Security Contexts
- Logs, Events, Debugging, Resource issues, Application failures
- Upgrades, Backups, Node maintenance, etcd
- Helm basics, Resource quotas, LimitRanges, Best practices

## 3. DIFFICULTY LEVEL

This mock exam is designed to be slightly harder than Mock Exam 1 to challenge your production readiness.

MODERATE | **HARD** | EXPERT

70% Hard • 30% Expert

## 4. TIME ALLOCATION

Total Time:

**180 MINUTES (3 HOURS)**

Recommended Allocation:

- Tasks 1 – 5 : 45 minutes
- Tasks 6 – 10 : 45 minutes
- Tasks 11 – 15 : 45 minutes
- Tasks 16 – 20 : 45 minutes

You may move freely between tasks. Manage your time wisely.

## 5. PASSING CRITERIA

Minimum Score Required to Pass:

**70%**

You should be able to:

- Complete all critical tasks
- Ensure resources are in desired state
- Follow best practices
- Maintain cluster stability
- Troubleshoot effectively
- Validate all resources before submission

## 6. EXAM FORMAT

- Performance-based exam
- No multiple choice questions
- Real Kubernetes cluster (environment will reset per task)
- Use kubectl and other tools
- Internet access is NOT available
- Tasks may require troubleshooting and logs analysis

**FOCUS AREAS**

Production Thinking, Troubleshooting, Efficiency, Accuracy, Best Practices.

## 7. KEY GUIDANCE FOR MOCK EXAM 2

**THINK LIKE AN ADMIN**
Make decisions as a Kubernetes administrator in a real production environment.

**READ CAREFULLY**
Understand requirements before executing commands.

**VALIDATE ALWAYS**
Verify resource status after each task. Do not assume.

**MAINTAIN STABILITY**
Avoid actions that break existing workloads.

**BEST PRACTICES**
Follow Kubernetes best practices in all tasks.

**REMEMBER:** Your goal is not just to complete tasks, but to do them the right way like a senior Kubernetes administrator.

# MOCK EXAM 2 (TASKS 21 TO 25)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 21 TASK 21: DEPLOYMENT STRATEGY

- Create a Deployment
- Configure RollingUpdate strategy
- Set maxUnavailable and maxSurge
- Verify zero-downtime deployment

### KEY POINTS

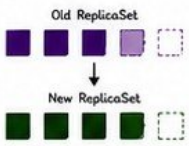
- Use RollingUpdate strategy
- Control maxUnavailable and maxSurge
- Ensure old and new Pods run together
- Validate zero downtime

### EXAMPLE COMMANDS

```
kubectl create deployment webapp
--image=nginx:1.25 --replicas=4
kubectl patch deployment webapp -p '
spec:
 strategy:
   type: RollingUpdate
   rollingUpdate:
     maxUnavailable: 1
     maxSurge: 1
'
kubectl rollout status deployment/webapp
```

### VERIFY

- No downtime during update




## 22 TASK 22: STATEFULSETS

- Deploy a StatefulSet
- Create a Headless Service
- Verify stable Pod identity

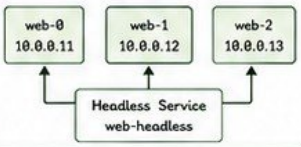
### KEY POINTS

- Use StatefulSet for stateful applications
- Headless Service for stable network identity
- Pod names are stable and ordered

### EXAMPLE COMMANDS

```
kubectl create -f statefulset.yaml
kubectl create -f headless-service.yaml
kubectl get pods -o wide
kubectl get svc
kubectl describe pod web-0
```

### VERIFY STABLE IDENTITY




## 23 TASK 23: HORIZONTAL POD AUTOSCALER

- Deploy Metrics Server
- Configure HPA
- Verify automatic scaling

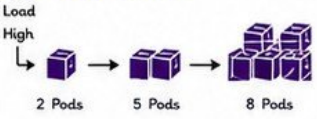
### KEY POINTS

- Metrics Server provides metrics
- Configure CPU or Memory based scaling
- Verify Pods scale automatically under load

### EXAMPLE COMMANDS

```
kubectl apply -f https://github.com/kubernetes-
sigs/metrics-server/releases/latest/download/
components.yaml
kubectl autoscale deployment webapp --cpu-
percent=50 --min=2 --max=10
kubectl get hpa -w
```

### VERIFY SCALING




## 24 TASK 24: MULTI-CONTAINER PODS

- Deploy a Pod with sidecar container
- Share storage using emptyDir
- Verify communication between containers

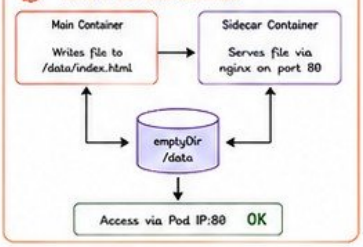
### KEY POINTS

- Use emptyDir for shared storage
- Sidecar assists main container
- Containers communicate via localhost

### EXAMPLE MANIFEST (SNIPPET)

```
containers:
- name: main
  image: busybox
  command: ['sh', '-c', 'while true;
  do echo hello > /data/index.html;
  sleep 2; done']
  volumeMounts:
   - name: shared
     mountPath: /data
- name: sidecar
  image: nginx:alpine
  ports:
  - containerPort: 80
  volumeMounts:
   - name: shared
     mountPath: /usr/share/nginx/html
volumes:
  - name: shared
    emptyDir: {}
```

### VERIFY COMMUNICATION




## 25 TASK 25: WORKLOAD TROUBLESHOOTING

- Diagnose a Deployment stuck in Progressing
- Identify rollout failure
- Restore application availability

### KEY POINTS

- Check rollout status and events
- Identify failing Pods
- Inspect logs and describe resources
- Fix issue and resume rollout

### TROUBLESHOOTING COMMANDS

```
kubectl rollout status deployment/webapp
kubectl describe deployment webapp
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get pods -l app=webapp
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl rollout undo deployment/webapp
```

### RESTORE AVAILABILITY

- Detect Issue
- Identify Root Cause
- Fix Configuration/Image
- Resume/Rollback
- Verify Application

### EXAM TIP

Deployment strategies and stateful workloads are critical in production. Always verify health after every change.

### DOMAINS COVERED IN THESE TASKS

- Workloads & Scheduling
- Configuration
- Configuration
- Serster Architecturing
- Storage
- Troubleshooting
- Cluster Maintenance

### FOCUS AREAS

- Rolling updates
- Stateful applications
- Autoscaling
- Multi-container patterns
- Troubleshooting rollouts

# MOCK EXAM 2 (TASKS 26 TO 30)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 26 TASK 26: ADVANCED SERVICES

- Configure a ClusterIP Service
- Create a NodePort Service
- Verify application access

### KEY POINTS

- ClusterIP for internal access
- NodePort for external access via node IP and port
- Ensure selectors match Pods
- Test connectivity from inside and outside the cluster

### EXAMPLE COMMANDS

```
# ClusterIP Service
kubectl expose deployment webapp \
 --name webapp-clusterip --port=80 --target-port=8080

# NodePort Service
kubectl expose deployment webapp \
 --name webapp-nodeport --type=NodePort \
 --port=80 --target-port=8080 --node-port=30080

kubectl get svc
```

### VERIFY ACCESS

ClusterIP (inside cluster)

Pod (inside) → webapp-clusterip 10.96.123.45:80

NodePort (outside cluster)

Browser / curl → NodeIP:30080 (e.g. 192.168.1.10:30080)

## 27 TASK 27: INGRESS

- Configure multiple host rules
- Configure path-based routing
- Validate application routing

### KEY POINTS

- Use Ingress for HTTP/HTTPS routing
- Host rules for domain-based routing
- Path rules for URL-based routing
- Ensure Ingress controller is running

### EXAMPLE MANIFEST (SNIPPET)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: app-ingress
spec:
rules:
- host: app1.example.com
  http:
   paths:
    - path: /
      pathType: Prefix
      backend:
       service:
        name: app1-service
        port:
         number: 80
- host: app2.example.com
  http:
   paths:
    - path: /api
      pathType: Prefix
      backend:
       service:
        name: app2-api
        port:
         number: 80
    - path: /
      pathType: Prefix
      backend:
       service:
        name: app2-web
        port:
         number: 80
```

### VALIDATE ROUTING

Ingress Controller (Load Balancer)

app1.example.com → / → app1-service (port 80)

app2.example.com → /api → app2-api (port 80); / → app2-web (port 80)

```
curl -H "Host: app1.example.com" http://<LB_IP>/
curl -H "Host: app2.example.com" http://<LB_IP>/api
```

## 28 TASK 28: NETWORK POLICIES

- Create default deny policy
- Allow only required traffic
- Verify communication restrictions

### KEY POINTS

- Default deny all Ingress and egress
- Allow only required Pods and ports
- Label Pods and namespaces correctly
- Test allowed and blocked traffic

### EXAMPLE MANIFEST (SNIPPET)

```
# Default deny all ingress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: default-deny-ingress
spec:
 podSelector: {}
 policyTypes:
 - Ingress
```

```
# Allow only from frontend to backend on port 80
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: allow-frontend-to-backend
spec:
 podSelector:
   matchLabels:
     app: backend
 policyTypes:
 - Ingress
 ingress:
 - from:
   - podSelector:
       matchLabels:
         app: frontend
   ports:
```

### VERIFY RESTRICTIONS

Frontend Pod (app: frontend) → Allow TCP 80 → Backend Pod (app: backend)

Other Pod (app: other) --- Deny (No traffic)

Test using: kubectl exec -it <pod> -- curl http://backend:80

→ Allow  - - - → Deny

## 29 TASK 29: DNS TROUBLESHOOTING

- Diagnose failed Service discovery
- Resolve CoreDNS issues
- Validate DNS resolution

### KEY POINTS

- Check CoreDNS Pod status
- Check Service and Endpoints
- Check CoreDNS ConfigMap
- Test DNS from inside Pods

### TROUBLESHOOTING COMMANDS

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
kubectl describe svc kube-dns -n kube-system
kubectl get svc <service-name>
kubectl get endpoints <service-name>
kubectl exec -it <pod> -- nslookup <service-name>
kubectl get cm -n kube-system coredns -o yaml
```

### VALIDATE DNS

Pod → nslookup my-service.default.svc.cluster.local

Should return ClusterIP (e.g. 10.96.x.x)

## 30 TASK 30: NETWORKING TROUBLESHOOTING

- Fix broken Service selectors
- Restore missing Endpoints
- Verify application connectivity

### KEY POINTS

- Service selectors must match Pod labels
- Endpoints must be populated
- Check Pod labels and Service selectors
- Verify connectivity after fix

### TROUBLESHOOTING STEPS

```
1 kubectl get svc <service>
  kubectl describe svc <service>
  # Check selector
2 kubectl get pods --show-labels
  # Verify Pod labels
3 Fix selector or Pod labels
4 kubectl get endpoints <service>
  # Endpoints should list Pod IPs
5 Test connectivity from Pod
```

### VERIFY CONNECTIVITY

Client Pod → Service (ClusterIP) → Backend Pods (Endpoints)

Test: kubectl exec -it <client-pod> -- curl http://service:80

Response OK = Connectivity Restored

### EXAM TIP

Always verify at each step. Check selectors, endpoints, and connectivity before moving forward.

### DOMAINS COVERED ON THIS PAGE

- Workloads & Scheduling
- Networking & Policies
- Clusteshooting
- Services & Networking
- Troubleshooting
- CoreDNS & DNS

### SUCCESS PRINCIPLE

Identify the root cause. Apply the correct fix. Validate and confirm.

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.

# MOCK EXAM 2 (TASKS 31 TO 35)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 31 TASK 31: PERSISTENT STORAGE

- Create PersistentVolume
- Create PersistentVolumeClaim
- Mount storage successfully

### KEY POINTS

- PV is cluster resource
- PVC is user request
- Binding connects PV and PVC
- Pod mounts PVC
- Data persists across Pod restarts

### EXAMPLE MANIFESTS (SNIPPET)

```yaml
# PersistentVolume
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-data
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /mnt/data
```

```yaml
# PersistentVolumeClaim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

### VERIFY SUCCESS

- PVC status is Bound
- Pod is Running
- Data available after Pod restart

Pod /data → PVC (Bound) → PV (pv-data)

## 32 TASK 32: STORAGE CLASSES

- Configure dynamic provisioning
- Create StorageClass
- Validate automatic volume creation

### KEY POINTS

- StorageClass enables dynamic provisioning
- PVC requests storage using StorageClass
- PV is created automatically
- No manual PV required

### EXAMPLE STORAGECLASS (SNIPPET)

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-sc
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

### VALIDATE DYNAMIC PROVISIONING

PVC created (storageClass: fast-sc) → PV automatically provisioned → PVC status: Bound

## 33 TASK 33: PERSISTENT VOLUME EXPANSION

- Expand an existing PVC
- Verify filesystem resize
- Validate application access

### KEY POINTS

- StorageClass must allow expansion
- PVC size can be increased
- Filesystem must be resized
- Application should see new space

### EXPANSION WORKFLOW (SNIPPET)

```
# Expand PVC
kubectl patch pvc pvc-data -p '{"spec":{"resources":
{"requests":{"storage":"10Gi"}}}}'

# Resize filesystem in Pod
kubectl exec -it <pod-name> -- bash
df -h /data
# or
kubectl exec -it <pod-name> -- resize2fs /dev/<device>
```

### VALIDATE EXPANSION

- PVC size updated
- Filesystem resized
- Application reads/writes beyond old capacity

## 34 TASK 34: STORAGE TROUBLESHOOTING

- Diagnose PVC Pending state
- Resolve binding issues
- Restore storage functionality

### COMMON CAUSES

- No matching PV
- StorageClass misconfiguration
- Insufficient capacity
- AccessModes mismatch
- Node affinity issues

### TROUBLESHOOTING STEPS

1. kubectl get pvc
2. kubectl describe pvc <pvc-name>
3. kubectl get pv
4. Check Events for errors
5. Verify StorageClass and provisioning
6. Fix root cause
7. Verify PVC is Bound

### EXPECTED RESULT

PVC status becomes Bound

PVC (Pending) → PVC (Bound) → Application can use storage

## 35 TASK 35: VOLUME RECOVERY

- Replace a failed volume
- Reattach storage
- Verify data availability

### KEY POINTS

- Detect volume failure
- Recreate or replace PV
- Restore or reattach data
- Verify application integrity

### RECOVERY WORKFLOW

1. Identify failed PV / Disk
2. Detach / Delete failed volume
3. Create new PV (or let dynamic provisioning)
4. Ensure PVC binds to new PV
5. Reattach volume to Pod
6. Validate data is intact

### VALIDATE RECOVERY

- Pod Running
- PVC Bound
- Data intact and accessible
- Application works normally

### EXAM TIP

Storage problems are common in production. Always understand the PV–PVC binding lifecycle.

### DOMAINS COVERED IN THESE TASKS

Storage · Storage Classes · Persistence · Troubleshooting · Recovery

### SUCCESS PRINCIPLE

Master storage operations: provision, expand, troubleshoot, and recover like a production Kubernetes administrator.

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.



CKA
CERTIFIED KUBERNETES ADMINISTRATOR

# MOCK EXAM 2 (TASKS 36 TO 40)

20 REALISTIC TASKS. MIXED DOMAINS. PRODUCTION SCENARIOS.

## 36 TASK 36: PRODUCTION SCENARIO

- Deploy a three-tier application
- Configure networking
- Configure persistent storage
- Configure Ingress

### KEY REQUIREMENTS

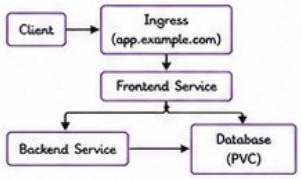
- Frontend (Nginx) served via Ingress
- Backend (API) exposed via ClusterIP
- Database (MySQL) with PVC
- Secure and highly available setup
- All components running and communicating

### EXAMPLE COMMANDS / YAML (SNIPPET)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

### VERIFY DEPLOYMENT

- All Pods Running
- Services have Endpoints
- PVC Bound and Mounted
- Ingress returns 200 OK




## 37 TASK 37: SECURITY INTEGRATION

- Create ServiceAccounts
- Configure RBAC
- Secure Secrets
- Verify least-privilege access

### KEY REQUIREMENTS

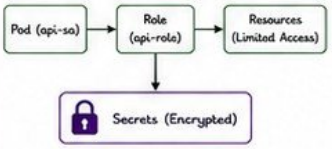
- ServiceAccount per application
- Role with minimal permissions
- RoleBinding to ServiceAccount
- Secrets stored securely
- Access is restricted and audited

### EXAMPLE YAML (SNIPPET)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: api-role
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["get", "list", "watch"]
---
kind: RoleBinding
metadata:
  name: api-rolebinding
subjects:
- kind: ServiceAccount
  name: api-sa
roleRef:
  kind: Role
  name: api-role
  apiGroup: rbac.authorization.k8s.io
```

### VERIFY SECURITY

- ServiceAccount exists
- RBAC limits permissions
- Secrets are not exposed
- Unauthorized actions are denied




## 38 TASK 38: CLUSTER VALIDATION

- Verify all workloads
- Validate Services
- Validate storage
- Validate networking

### VALIDATION CHECKLIST

- [x] All Nodes Ready
- [x] All Pods Running
- [x] Deployments have desired replicas
- [x] Services have Endpoints
- [x] PVCs Bound and Mounted
- [x] Ingress rules working
- [x] DNS resolution working
- [x] NetworkPolicies applied

### USEFUL COMMANDS

```
kubectl get nodes
kubectl get pods -A
kubectl get deploy -A
kubectl get endpoints -A
kubectl get pvc -A
kubectl get ingress -A
kubectl get networkpolicy -A
kubectl get events --sort-by=.lastTimestamp
```

### VALIDATION RESULT

- All resources are healthy
- No pending or error states
- Application is fully accessible
- Cluster is production-ready

CLUSTER STATUS: ✓ HEALTHY

## 39 TASK 39: PRODUCTION TROUBLESHOOTING

- Resolve multiple cluster issues
- Restore application health
- Verify complete functionality

### POSSIBLE ISSUES

- Pod CrashLoopBackOff
- Service has no Endpoints
- PVC in Pending state
- Ingress 502 / 503 errors
- Network connectivity blocked

### TROUBLESHOOTING WORKFLOW

1. Identify the issue from symptoms
2. Check Pods and Logs
3. Check Events and describe resources
4. Identify root cause
5. Fix configuration or resource
6. Verify resolution
7. Validate full application functionality

### EXPECTED OUTCOME

- All issues resolved
- Application is healthy
- Users can access without errors
- SLA and performance restored

APPLICATION HEALTH: ✓ RESTORED

## 40 TASK 40: END-TO-END PRODUCTION ASSESSMENT

- Review the entire cluster
- Validate every objective
- Ensure all resources meet requirements
- Confirm readiness before submission

### ASSESSMENT CHECKLIST

- [x] All workloads deployed correctly
- [x] Networking and Ingress validated
- [x] Storage configured and working
- [x] Security implemented correctly
- [x] Monitoring and logs clean
- [x] Best practices followed
- [x] No critical or warning issues

### FINAL VERIFICATION COMMANDS

```
kubectl get all -A
kubectl get pvc -A
kubectl get ingress -A
kubectl get networkpolicy -A
kubectl get events -A --sort-by=.lastTimestamp
kubectl top nodes
kubectl top pods -A
kubectl auth can-i --list --as=system:serviceaccount:default:api-sa
```

### FINAL RESULT

- All tasks completed successfully
- All objectives validated
- Cluster is secure and stable
- Ready for production use

EXAM STATUS: ✓ PASS

### EXAM TIP

Read every requirement carefully. Plan your approach and validate before moving to the next task.

### DOMAINS COVERED IN THESE TASKS

Workloads · Networking · Storage · Security · Troubleshooting

### SUCCESS PRINCIPLE

Attention to detail, methodical troubleshooting, and thorough validation lead to production-ready Kubernetes clusters.



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# MOCK EXAM 2 SOLUTIONS

CKA CERTIFIED KUBERNETES ADMINISTRATOR

TASKS 21 TO 40 – COMPLETE SOLUTIONS

## 21–25 WORKLOADS & SCALING SOLUTIONS

### 21 DEPLOYMENT STRATEGY

- ✓ Created Deployment
- ✓ Set RollingUpdate strategy
- ✓ maxUnavailable: 1
- ✓ maxSurge: 1
- ✓ Zero-downtime verified

```
# Create Deployment
kubectl apply -f deployment-webapp.yaml
# Verify rollout
kubectl rollout status deployment/webapp
# Check strategy
kubectl get deploy webapp -o yaml |
grep -A5 -i strategy
```

### 22 STATEFULSETS

- ✓ Deployed StatefulSet
- ✓ Created Headless Service
- ✓ Stable Pod identity verified

```
kubectl apply -f headless-service.yaml
kubectl apply -f statefulset.yaml
kubectl get pods -l app=db -o wide
kubectl exec -it db-0 -- hostname
kubectl exec -it db-1 -- hostname
```

### 23 HORIZONTAL POD AUTOSCALER

- ✓ Metrics Server installed
- ✓ HPA configured
- ✓ Automatic scaling verified

```
kubectl apply -f metrics-server.yaml
kubectl apply -f hpa.yaml
kubectl get hpa
kubectl get pods -w
kubectl top pods
```

### 24 MULTI-CONTAINER PODS

- ✓ Pod with sidecar deployed
- ✓ emptyDir volume shared
- ✓ Containers communicate

```
kubectl apply -f multi-container-pod.yaml
kubectl exec -it multi-pod -c main --
cat /shared/index.html
kubectl exec -it multi-pod -c sidecar --
curl http://localhost:8080
```

### 25 WORKLOAD TROUBLESHOOTING

- ✓ Rollout failure identified
- ✓ Root cause fixed
- ✓ Application restored

```
kubectl rollout status deployment/webapp
kubectl describe deployment webapp
kubectl get events --sort-by=.lastTimestamp
kubectl rollout undo deployment/webapp
kubectl rollout status deployment/webapp
```

## 26–30 NETWORKING SOLUTIONS

### 26 ADVANCED SERVICES

- ✓ ClusterIP configured
- ✓ NodePort created
- ✓ Access verified

```
kubectl apply -f svc-clusterip.yaml
kubectl apply -f svc-nodeport.yaml
kubectl get svc
curl http://<node-ip>:<nodeport>
```

### 27 INGRESS

- ✓ Multiple host rules
- ✓ Path-based routing
- ✓ Routing validated

```
kubectl apply -f ingress.yaml
kubectl get ingress
```

Test:

```
curl -H "Host: app1.example.com" http://<ip>/
curl -H "Host: app2.example.com/api" http://<ip>/api
```

### 28 NETWORK POLICIES

- ✓ Default deny applied
- ✓ Required traffic allowed
- ✓ Restrictions verified

```
kubectl apply -f deny-all.yaml
kubectl apply -f allow-rules.yaml
kubectl get networkpolicies
```

Test:

```
kubectl exec -it pod-a -- curl http://pod-b:80   (allowed)
kubectl exec -it pod-a -- curl http://pod-c:80   (denied)
```

### 29 DNS TROUBLESHOOTING

- ✓ CoreDNS issue found
- ✓ CoreDNS restored
- ✓ DNS resolution validated

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system deploy/coredns
kubectl rollout restart deployment coredns -n kube-system
kubectl exec -it test-pod -- nslookup my-service
```

### 30 NETWORKING TROUBLESHOOTING

- ✓ Fixed Service selectors
- ✓ Endpoints restored
- ✓ Connectivity verified

```
kubectl describe svc my-service
kubectl get endpoints my-service
kubectl apply -f svc-fixed.yaml
kubectl get endpoints my-service
```

Test: `curl http://my-service:80`

## 31–35 STORAGE SOLUTIONS

### 31 PERSISTENT STORAGE

- ✓ PersistentVolume created
- ✓ PersistentVolumeClaim bound
- ✓ Storage mounted

```
kubectl apply -f pv.yaml
kubectl apply -f pvc.yaml
kubectl get pv,pvc
kubectl exec -it pod -- df -h /data
```

### 32 STORAGE CLASSES

- ✓ StorageClass created
- ✓ Dynamic provisioning enabled
- ✓ PVC auto provisioned

```
kubectl apply -f storageclass.yaml
kubectl apply -f pvc-dynamic.yaml
kubectl get pvc
kubectl get pv
```

### 33 PERSISTENT VOLUME EXPANSION

- ✓ PVC expanded
- ✓ Filesystem resized
- ✓ Application access OK

```
kubectl patch pvc data-pvc -p '{"spec":
{"resources":{"requests":{"storage":"10Gi"}}}}'
kubectl get pvc
kubectl exec -it pod -- resize2fs /dev/volume
kubectl exec -it pod -- df -h /data
```

### 34 STORAGE TROUBLESHOOTING

- ✓ PVC Pending diagnosed
- ✓ Binding issue resolved
- ✓ Storage functional

```
kubectl describe pvc data-pvc
kubectl get events --sort-by=.lastTimestamp
kubectl apply -f pv-fixed.yaml
kubectl delete -it data-pvc
kubectl apply -f pvc.yaml
```

### 35 VOLUME RECOVERY

- ✓ Failed volume replaced
- ✓ PV reattached
- ✓ Data availability verified

```
kubectl get pv
kubectl edit pv pv-data (update node affinity/path)
kubectl delete pod appd
kubectl exec -it app-pod -- ls /data
```

## 36–40 PRODUCTION & VALIDATION SOLUTIONS

### 36 PRODUCTION SCENARIO

- ✓ Three-tier app deployed
- ✓ Networking configured
- ✓ Persistent storage ready
- ✓ Ingress configured

```
kubectl apply -f 01-namespace.yaml
kubectl apply -f 02-database.yaml
kubectl apply -f 03-backend.yaml
kubectl apply -f 04-frontend.yaml
kubectl apply -f 05-services.yaml
kubectl apply -f 06-ingress.yaml
kubectl get all -n app
```

### 37 SECURITY INTEGRATION

- ✓ ServiceAccounts created
- ✓ RBAC configured
- ✓ Secrets secured
- ✓ Least-privilege verified

```
kubectl apply -f serviceaccounts.yaml
kubectl apply -f rbac.yaml
kubectl apply -f secrets.yaml
kubectl auth can-i get pods --as=system:serviceaccount:app:api-sa
kubectl auth can-i delete pods --as=system:serviceaccount:app:api-sa
                                                          (no)
```

### 38 CLUSTER VALIDATION

- ✓ Workloads healthy
- ✓ Services validated
- ✓ Storage healthy
- ✓ Networking verified

```
kubectl get pods -A
kubectl get svc -A
kubectl get pv,pvc -A
kubectl get ingress -A
kubectl get networkpolicies -A
```

### 39 PRODUCTION TROUBLESHOOTING

- ✓ Issues identified
- ✓ Fixes applied
- ✓ Application healthy

```
kubectl get events --sort-by=.lastTimestamp
kubectl describe <resource>
kubectl logs <pod>
kubectl rollout restart deployment <name>
kubectl get all -A
curl -I http://app.example.com
```

### 40 END-TO-END ASSESSMENT

- ✓ All objectives met
- ✓ Resources validated
- ✓ Production-ready
- ✓ Ready for submission

```
kubectl get all -A
kubectl get pv,pvc -A
kubectl get ingress -A
kubectl get networkpolicies -A
kubectl get nodes
kubectl top pods
```

### SOLUTION CHECKLIST

- ☑ All tasks completed successfully
- ☑ All resources in desired state
- ☑ No errors or pending resources
- ☑ Application fully functional
- ☑ Security and best practices followed

### COMMAND QUICK REFERENCE

| | |
|---|---|
| `kubectl get all -A` | View all resources |
| `kubectl describe <resource>` | Detailed resource info |
| `kubectl logs <pod>` | View pod logs |
| `kubectl get events -A` | View recent events |
| `kubectl top nodes / pods` | Resource usage |

### SUCCESS CRITERIA

- ✓ All 40 tasks completed
- ✓ Every objective achieved
- ✓ Cluster is stable and secure
- ✓ Application is fully functional
- ✓ Exam submission ready

EXAM STATUS: ✓ PASS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

# MOCK EXAM 2 REVIEW

TASKS 21 TO 40 – KEY CONCEPTS, TAKEAWAYS & EXAM READINESS

## 1 KEY TAKEAWAYS

- **Workloads & Scaling:** Master Deployments, StatefulSets, HPA, and multi-container patterns for resilient applications.
- **Networking:** Understand Services, Ingress, Network Policies, DNS, and troubleshooting for reliable connectivity.
- **Storage:** Know how to provision, expand, recover, and troubleshoot persistent storage effectively.
- **Security:** Apply RBAC, ServiceAccounts, and Secrets to enforce least-privilege and secure workloads.
- **Production Readiness:** Validate the cluster, monitor health, resolve issues, and ensure end-to-end functionality.

## 2 COMMON PITFALLS TO AVOID

- ✖ Incorrect Deployment strategies (wrong maxSurge / maxUnavailable)
- ✖ StatefulSets without Headless Service (unstable network identity)
- ✖ HPA without Metrics Server or wrong metrics configuration
- ✖ Services with wrong selectors (no Endpoints created)
- ✖ Ingress host/path mismatches
- ✖ Overly permissive or missing Network Policies
- ✖ PVCs stuck in Pending due to StorageClass or access mode mismatch
- ✖ Forgetting to update filesystem after PVC expansion
- ✖ Not verifying DNS or CoreDNS issues before troubleshooting apps
- ✖ Leaving Secrets unencrypted or accessible to all ServiceAccounts

## 3 REVIEW BY DOMAIN

| DOMAIN | TASKS | FOCUS AREAS | IMPORTANT CONCEPTS |
|---|---|---|---|
| Workloads & Scaling | 21 – 25 | Deployments, StatefulSets, HPA, Multi-Container Pods, Troubleshooting | RollingUpdate, HPA, emptyDir, Rollout failure, Pod identity |
| Networking | 26 – 30 | Services, Ingress, Network Policies, DNS, Networking Troubleshooting | ClusterIP, NodePort, Ingress rules, Policy isolation, CoreDNS, Endpoints |
| Storage | 31 – 35 | PV, PVC, StorageClass, Expansion, Recovery, Troubleshooting | Dynamic provisioning, PVC expansion, binding issues, Volume recovery |
| Production & Validation | 36 – 40 | Production setup, Security, Validation, Troubleshooting, Assessment | RBAC, Secrets, Cluster health, End-to-end validation, Readiness for production |

## 4 EXAM READINESS CHECKLIST

| CHECKPOINT | YES | NO |
|---|---|---|
| I can create and manage Deployments with strategies | ☐ | ☐ |
| I can deploy and manage StatefulSets with stable identity | ☐ | ☐ |
| I can configure HPA and understand auto-scaling | ☐ | ☐ |
| I can work with Services, Ingress, and DNS | ☐ | ☐ |
| I can create and apply Network Policies correctly | ☐ | ☐ |
| I can provision, mount, expand, and troubleshoot storage | ☐ | ☐ |
| I can secure workloads using RBAC and Secrets | ☐ | ☐ |
| I can troubleshoot workloads, networking, and storage | ☐ | ☐ |
| I can validate the entire cluster for production | ☐ | ☐ |
| I can deploy and validate a multi-tier application | ☐ | ☐ |

## 5 HIGH-VALUE COMMANDS CHEAT SHEET

| | |
|---|---|
| Workloads & Scaling | `kubectl rollout status deployment/<name>`<br>`kubectl get statefulset -o wide`<br>`kubectl autoscale deployment <name> --cpu-percent=70 --min=2 --max=5` |
| Services & Networking | `kubectl get svc,ep,ing -o wide`<br>`kubectl describe svc <name>`<br>`kubectl get networkpolicy -A` |
| Storage | `kubectl get pv,pvc,sc -o wide`<br>`kubectl describe pvc <name>`<br>`kubectl exec -it <pod> -- df -h` |
| Troubleshooting | `kubectl get events --sort-by=.metadata.creationTimestamp`<br>`kubectl describe pod <name>`<br>`kubectl logs <pod> -c <container> --tail=100` |
| Cluster Validation | `kubectl get all -A`<br>`kubectl get nodes -o wide`<br>`kubectl top nodes && kubectl top pods -A` |

## 6 MOCK EXAM 2 SUCCESS CRITERIA

| | |
|---|---|
| **Task Completion**<br>All 40 tasks completed successfully | ✓ PASS |
| **Functionality**<br>All workloads, services, storage, and networking working as expected | ✓ PASS |
| **Security**<br>RBAC, ServiceAccounts, Secrets, and Policies applied correctly | ✓ PASS |
| **Stability**<br>No errors, no pending resources, applications are healthy | ✓ PASS |
| **Validation**<br>Cluster validated end-to-end and ready for production | ✓ PASS |

GOAL: ZERO ERRORS. 100% PRODUCTION READY.

### EXAM TIP

Think like a production engineer: Validate every change, check logs, confirm connectivity, and always verify the end result.

### DOMAINS COVERED IN MOCK EXAM 2

Workloads & Scheduling | Services & Networking | Storage & Persistence | Security & RBAC | Troubleshooting & Recovery

### SUCCESS PRINCIPLE

Preparation + Practice + Review = Confidence.

Review your mistakes. Strengthen your weak areas. Pass with confidence!

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

# MOCK EXAM 3 OVERVIEW

40 CHALLENGING TASKS. ADVANCED SCENARIOS. PRODUCTION-GRADE COMPLEXITY.

Mock Exam 3 is the most advanced practice exam in this handbook. It simulates real-world production environments with complex workloads, networking, storage, security, automation, and troubleshooting scenarios.

This exam validates your ability to design, configure, secure, scale, and operate Kubernetes clusters at a production level.

## 1 EXAM SUMMARY

| | |
|---|---|
| Total Tasks | 40 |
| Time Limit | 8 Hours |
| Difficulty Level | Advanced |
| Passing Criteria | All tasks completed successfully |
| Environment | Live Kubernetes Cluster |
| Focus | Production Engineering, Reliability & Operations |

### EXAM FOCUS AREAS

- Advanced Workloads & Scaling
- Networking & Traffic Management
- Storage & Data Protection
- Security & Access Control
- Cluster Reliability & Performance
- Monitoring, Logging & Observability
- Automation & Operations
- Troubleshooting Complex Failures

## 2 WHAT'S NEW IN MOCK EXAM 3

- Multi-cluster concepts and advanced networking
- Advanced Ingress and traffic control
- Persistent storage with snapshots and backups
- Pod Security Standards and advanced RBAC
- High availability and Disaster Recovery scenarios
- Monitoring, logging, and alerting integration
- Helm charts and GitOps workflows
- Complex troubleshooting and root cause analysis

## 3 TASK BREAKDOWN BY DOMAIN

| DOMAIN | TASKS | KEY TOPICS COVERED | EXAMPLE SCENARIOS |
|---|---|---|---|
| Advanced Workloads & Scaling | 01 – 10 | Deployments, StatefulSets, DaemonSets, HPA, VPA, Resource Management | Rolling updates, Pod disruption budgets, Auto-scaling, Multi-container patterns |
| Networking & Traffic Management | 11 – 20 | Services, Ingress, Network Policies, DNS, Service Mesh Concepts | Advanced Ingress routing, NetworkPolicies, Traffic troubleshooting |
| Storage & Data Protection | 21 – 30 | Storage Classes, PV/PVC, Snapshots, Backups, Volume Expansion | Dynamic provisioning, Backup & restore, Disaster recovery |
| Security & Access Control | 31 – 35 | RBAC, ServiceAccounts, Pod Security Standards, Secrets Management | Least privilege, Security contexts, Image pull secrets |
| Operations, Reliability & Monitoring | 36 – 40 | Monitoring, Logging, Alerts, Backup, Troubleshooting, Cluster Validation | Prometheus/Grafana, Centralized logging, Full cluster health check |

## 4 EXAM ENVIRONMENT

- Kubernetes Version: v1.29+
- Nodes: Multi-node cluster (Control Plane + Workers)
- Container Runtime: containerd
- CNI Plugin: Calico / Flannel
- Storage: Dynamic provisioning enabled
- Tools Available: kubectl, vim, curl, jq, helm
- Internet Access: Enabled (if required)

## 5 PREPARATION STRATEGY

- Review all concepts deeply – understand why, not just how.
- Practice troubleshooting methodically – check logs, events, and states.
- Validate every resource after creation.
- Think like a production engineer – reliability, security, and performance first.
- Manage your time – average 12 minutes per task.
- Test your solutions before moving to the next task.

### EXAM TIP

Read each task carefully.
Plan your approach before making changes.
Small mistakes can break the entire flow.

### EXAM OBJECTIVE

Prove you can operate Kubernetes like a Senior Engineer in a real production environment.

### SUCCESS PRINCIPLE

Preparation + Practice + Analysis
=
Production Ready Kubernetes Administrator

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

# MOCK EXAM 3 (TASKS 41 TO 45)

ADVANCED ADMINISTRATION. CLUSTER OPERATIONS. PRODUCTION READINESS.

## 41 TASK 41: CLUSTER ADMINISTRATION

- Configure multiple kubeconfig contexts
- Switch between clusters
- Verify cluster health
- Validate node readiness

### KEY ACTIONS

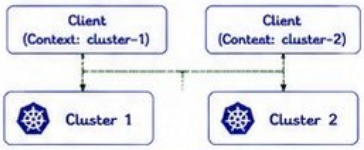
1. Add clusters to kubeconfig
2. Create separate contexts
3. Switch between contexts
4. Verify API Server access
5. Check nodes and system health

### USEFUL COMMANDS

```
# View contexts
kubectl config get-contexts
# Switch context
kubectl config use-context cluster-2
# Current context
kubectl config current-context
# Cluster info
kubectl cluster-info
# Nodes status
kubectl get nodes -o wide
# Component health
kubectl get --raw='/readyz?verbose'
```

### VERIFY SUCCESS

- Contexts configured
- Able to switch between clusters
- API Server reachable
- All nodes show Ready
- Cluster components healthy




## 42 TASK 42: NODE ADMINISTRATION

- Label worker nodes
- Apply taints
- Configure tolerations
- Schedule workloads correctly

### KEY ACTIONS

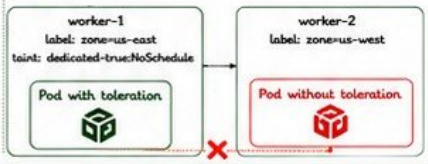
1. Label nodes for identification
3. Apply taints to nodes
3. Add tolerations to Pods
4. Deploy workloads
5. Verify scheduling behavior

### USEFUL COMMANDS

```
# Label node
kubectl label node worker-1 zone=us-east
# Taint node
kubectl taint nodes worker-1
   key=dedicated:NoSchedule
# Describe node
kubectl describe node worker-1 | grep -i taint
# Pod with toleration (example)
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
   tolerations:
   - key: "dedicated"
     operator: "Equal"
     value: "true"
     effect: "NoSchedule"
```

### VERIFY SUCCESS

- Labels applied
- Taints applied
- Tolerations configured
- Pod scheduled on correct node
- No scheduling violations




## 43 TASK 43: CONTROL PLANE MAINTENANCE

- Inspect control plane components
- Verify static Pods
- Validate etcd status
- Restore failed control plane services

### KEY ACTIONS

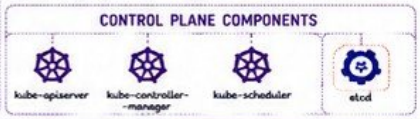
1. Inspect control plane Pods
2. Verify static Pod status
4. Check etcd health
4. Restart failed components
5. Confirm control plane recovery

### USEFUL COMMANDS

```
# Static Pods in control plane
kubectl get pods -n kube-system
  -o wide | grep -E 'apiserver|controller|scheduler|etcd'
# etcd health
echo | etcdctl --endpoints=https://
127.0.0.1:2379 --cacert=/etc/kubernetes/pki/
etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/
health check
# etcd members
etcdctl member list
# Restart static Pod (example)
systemctl restart kube-apiserver
```

### VERIFY SUCCESS

- All control plane Pods Running
- etcd member healthy
- API Server responding
- Scheduler and Controller Manager healthy
- Cluster fully operational




## 44 TASK 44: CLUSTER UPGRADE PLANNING

- Review upgrade path
- Check version compatibility
- Validate version skew
- Prepare rollback strategy

### KEY ACTIONS

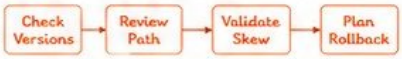
1. Check current versions
2. Review supported upgrade path
3. Validate version skew rules
4. Identify compatibility issues
5. Plan rollback if upgrade fails

### USEFUL COMMANDS

```
# Current version
kubectl version --short
# Upgrade plan (example using kubedam)
kubedam upgrade plan
# Validate version skew
kubedam upgrade version
# Check kubelet version on nodes
kubectl get nodes -o jsonpath='{.items[*].
status.nodeInfo.kubeletVersion}'
```

### VERIFY SUCCESS

- Upgrade path is valid
- All versions are compatible
- Version skew is within limits
- Rollback plan documented
- Cluster ready for upgrade




**VERSION SKEW RULES (KUBERNETES)**

| Component | Rule |
|---|---|
| kube-apiserver | N |
| kube-controller-manager / kube-scheduler | N or N-1 |
| kubelet | N, N-1 or N-2 |
| kubectl | N or N-1 |

**ROLLBACK STRATEGY CHECKLIST**

- [x] Take etcd snapshot
- [x] Document current configuration
- [x] Keep previous binaries
- [x] Plan downtime if required
- [x] Test rollback procedure

**SUPPORTED UPGRADE PATH (EXAMPLE)**

1.27 → 1.28 → 1.29 → 1.30

## 45 TASK 45: PRODUCTION CLUSTER VALIDATION

- Verify cluster-wide resources
- Validate namespaces
- Vnspect workloads
- Confirm cluster stability

### KEY ACTIONS

1. Verify system namespaces
2. Check cluster resources
3. Inspect all workloads
4. Verify Services and Endpoints
5. Confirm cluster stability

### USEFUL COMMANDS

```
kubectl get namespaces
kubectl get nodes
kubectl get podas -A
kubectl get deployments -A
kubectl get services -A
kubectl get pvc -A
kubectl get events -A --sort-by=.lastTimestamp
```

### VERIFY SUCCESS

- All namespaces healthy
- No unexpected resource errors
- All workloads Running
- Services and Endpoints healthy
- No critical events
- Cluster production-ready

**EXAM TIP**

Always validate after every change. Document your steps and verify the result before moving to the next task.

**FOCUS AREAS IN TASKS 41–45**

- Cluster Administration
- Node Management
- Control Plane Maintenance
- Upgrade Planning
- Production Validation

**SUCCESS PRINCIPLE**

Strong administration leads to a stable cluster. Plan carefully, validate continuously, and document everything.

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.



# MOCK EXAM 3 (TASKS 46 TO 50)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

ADVANCED ADMINISTRATION. CLUSTER OPERATIONS. PRODUCTION READINESS.

## 46 TASK 46: CONTROL PLANE UPGRADE

- Upgrade the Kubernetes control plane
- Verify API Server
- Validate Scheduler
- Confirm Controller Manager health

### KEY ACTIONS

1. Check current control plane version
2. Upgrade API Server
3. Upgrade Scheduler
4. Upgrade Controller Manager
5. Verify control plane components

### USEFUL COMMANDS

```
# Check version
kubectl version --short
# Upgrade control plane components
  (example using kubeadm)
kubeadm upgrade plan
kubeadm upgrade apply v1.29.x
# Verify API Server
kubectl get componentstatuses
# Check control plane Pods
kubectl get pods -n kube-system -l tier=control-plane
# Check services
kubectl get pods,svc -n kube-system
```

### VERIFY SUCCESS

- API Server Healthy (200 OK)
- Scheduler Running
- Controller Manager Healthy
- All control plane Pods Running
- No error events in kube-system

#### CONTROL PLANE COMPONENTS

API Server | Scheduler | Controller Manager

## 47 TASK 47: WORKER NODE UPGRADE

- Drain worker node
- Upgrade kubelet and kubectl
- Uncordon node
- Verify workload recovery

### KEY ACTIONS

1. Drain the worker node
2. Upgrade kubelet and kubectl
3. Uncordon the node
4. Verify workloads are running

### USEFUL COMMANDS

```
# Drain node
kubectl drain worker-1 --ignore-daemonsets \
  --deelete-emptydir-data
# Upgrade kubelet (example with apt)
ssh worker-1 "sudo apt-get update && \
  sudo apt-get install -y kubelet=1.29.x-00 \
  && sudo systemctl restart kubelet"
# Upgrade kubectl
scp /etc/kubernetes/admin.conf .
# or install new kubectl on admin machine
# Uncordon node
kubectl uncordon worker-1
# Check node and pods
kubectl get nodes
kubectl get pods -A -o wide
```

### VERIFY SUCCESS

- Node Ready and schedulable
- Kubelet running new version
- All workloads scheduled
- No pending Pods
- No critical events

#### WORKER NODE UPGRADE FLOW

Drain Node → Upgrade Kubelet & Kubectl → Uncordon Node

## 48 TASK 48: CERTIFICATE MANAGEMENT

- Inspect Kubernetes certificates
- Renew expired certificates
- Restart affected services
- Validate cluster communication

### KEY ACTIONS

1. Inspect certs and check expiration
2. Renew expired certificates
3. Restart necessary services
4. Validate cluster communication

### USEFUL COMMANDS

```
# List certs expiration
kubeadm certs check-expiration
# Renew all certs
sudo kubeadm certs renew all
# Renew specific cert
sudo kubeadm certs renew apiserver
# Restart control plane static pods
sudo systemctl restart kubelet
# Verify cluster communication
kubectl get nodes
kubectl get componentstatuses
```

### VERIFY SUCCESS

- All certificates valid
- Control Plane Pods Running
- API Server reachable
- kubectl commands succeed
- No TLS errors in logs

#### CERTIFICATE LOCATIONS

```
/etc/kubernetes/pki/
/var/lib/kubelet/pki/
/etc/kubernetes/manifests/
```

## 49 TASK 49: BACKUP AND RECOVERY

- Create an etcd backup
- Restore etcd
- Validate recovered cluster

### KEY ACTIONS

1. Create etcd snapshot backup
2. Restore etcd from backup
3. Validate cluster after recovery

### USEFUL COMMANDS

```
# Create etcd backup
ETCDCTL_API=3 etcdctl snapshot save \
  /backup/etcd-$(date +%F-%H%M%S).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
  --key=/etc/kubernetes/pki/etcd/healthcheck-client.key
# Restore etcd (on control plane node)
systemctl stop kubelet
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-xxx.db \
  --data-dir /var/lib/etcd-from-backup
# Update manifest if required, then:
systemctl start kubelet
```

### VERIFY SUCCESS

- etcd member healthy
- Cluster API Server accessible
- Nodes Ready
- All workloads running

#### BACKUP BEST PRACTICES

- Schedule regular snapshots
- Store backups off-node securely
- Test restore process periodically

## 50 TASK 50: UPGRADE TROUBLESHOOTING

- Diagnose failed upgrade
- Resolve version mismatch
- Restore cluster functionality

### KEY ACTIONS

1. Identify failure point
2. Resolve version mismatch or dependency issues
3. Roll back or complete upgrade
4. Validate cluster functionality

### USEFUL COMMANDS

```
# Check node and component versions
kubectl get nodes -o wide
kubectl get componentstatuses
# Check logs for control plane pods
kubectl -n kube-system logs <pod-name>
# Reset failed upgrade (if needed)
kubeadm reset phase <phase> (if required)
# Rejoin node / reinitialize as needed
# Verify cluster
kubectl get nodes
kubectl get pods -A
```

### VERIFY SUCCESS

- All components healthy
- Version consistency
- Cluster stable
- No critical errors
- Applications functional

#### COMMON ISSUES

- Version skew beyond supported range
- Kubelet / kubectl version mismatch
- Certificate or CNI plugin issues
- etcd or API Server failures

### EXAM TIP

Always verify after every change. Check logs, events, and resource status. Small validation steps prevent big problems.

### FOCUS AREAS IN TASKS 46–50

- Control Plane Upgrades
- Worker Node Management
- Certificate Management
- Backup & Recovery
- Upgrade Troubleshooting
- Cluster Stability

### SUCCESS PRINCIPLE

Careful planning, validation, and recovery readiness make a cluster production-ready and resilient.

# MOCK EXAM 3 (TASKS 51 TO 55)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

ADVANCED ADMINISTRATION. CLUSTER OPERATIONS. PRODUCTION READINESS.

## 51 TASK 51: ADVANCED SCHEDULING

- Configure node affinity
- Configure Pod affinity
- Configure anti-affinity
- Validate scheduling behavior

### KEY ACTIONS

1. Define node affinity rules
2. Define Pod affinity rules
3. Define Pod anti-affinity rules
4. Deploy workloads
5. Validate scheduling results

### USEFUL COMMANDS

```
# View node labels
kubectl get nodes --show-labels
# Describe Pod scheduling
kubectl describe pod <pod-name>
# View events
kubectl get events --sort-by=.lastTimestamp
# View Pod placement
kubectl get pods -o wide
# Check node affinity in Pod spec
kubectl get pod <pod-name> -o yaml
```

### VERIFY SUCCESS

- Pods scheduled on correct nodes
- Affinity rules respected
- Anti-affinity rules respected
- Pods distributed as expected
- No unexpected Pending Pods

**AFFINITY VISUAL EXAMPLE**

Allowed Placement  Disallowed Placement

## 52 TASK 52: PRIORITY AND PREEMPTION

- Create PriorityClass
- Configure workload priorities
- Validate preemption decisions

### KEY ACTIONS

1. Create PriorityClass objects
2. Assign priorities to workloads
3. Deploy multiple workloads
4. Trigger preemption
5. Validate preemption behavior

### USEFUL COMMANDS

```
# Create PriorityClass
kubectl apply -f priorityclass.yaml
# View PriorityClasses
kubectl get priorityclass
# View Pods by priority
kubectl get pods --sort-by=.spec.priority
# Describe Pod for preemption
kubectl describe pod <pod-name>
# View events
kubectl get events --sort-by=.lastTimestamp
```

### VERIFY SUCCESS

- PriorityClass created
- Pods have correct priority
- Lower priority Pods preempted
- High priority Pods running
- Cluster remains stable

**PREEMPTION FLOW**

## 53 TASK 53: RESOURCE GOVERNANCE

- Configure ResourceQuota
- Configure LimitRange
- Validate namespace resource limits

### KEY ACTIONS

1. Create ResourceQuota
2. Create LimitRange
3. Deploy workloads
4. Test resource limits
5. Validate enforcement

### USEFUL COMMANDS

```
# Create ResourceQuota
kubectl apply -f resourcequota.yaml
# Create LimitRange
kubectl apply -f limitrange.yaml
# View ResourceQuota
kubectl get resourcequota
# View LimitRange
kubectl get limitrange
# Describe namespace
kubectl describe namespace <ns>
```

### VERIFY SUCCESS

- ResourceQuota enforced
- LimitRange enforced
- Pods blocked when limits exceeded
- Warnings shown for LimitRange defaulting
- Namespace resources governed

**RESOURCE GOVERNANCE EXAMPLE**

| Resource | Requested | Limit | Status |
|---|---|---|---|
| CPU | 2 | 4 | Allowed |
| Memory | 4Gi | 8Gi | Allowed |
| Pods | 10 | 10 | Denied (max 10) |

## 54 TASK 54: PRODUCTION SCHEDULING TROUBLESHOOTING

- Diagnose Pending Pods
- Resolve scheduling conflicts
- Verify successful scheduling

### KEY ACTIONS

1. Identify Pending Pods
2. Inspect events and reasons
3. Identify constraints/conflicts
4. Apply corrective actions
5. Verify Pods running

### USEFUL COMMANDS

```
# Get Pending Pods
kubectl get pods --field-selector=status.phase=Pending
# Describe Pending Pod
kubectl describe pod <pod-name>
# View events for namespace
kubectl get events -n <ns> --sort-by=.lastTimestamp
# Check node resources
kubectl describe nodes
# Check taints
kubectl describe nodes | grep -A3 Taints:
```

### VERIFY SUCCESS

- No Pending Pods
- Scheduling conflicts resolved
- Pods running on expected nodes
- Events show successful scheduling
- Cluster stable

**COMMON PENDING REASONS**

- Insufficient resources
- NodeSelector mismatch
- Taints not tolerated
- Affinity/Anti-affinity rules
- PVC not bound

## 55 TASK 55: HIGH AVAILABILITY SCHEDULING

- Distribute workloads across nodes
- Avoid single points of failure
- Validate resilience

### KEY ACTIONS

1. Spread workloads across zones/nodes
2. Use topology-aware rules
2. Test node failure scenarios
4. Verify workload recovery
5. Validate high availability

### USEFUL COMMANDS

```
# View Pod distribution
kubectl get pods -o wide
# View topology spread constraints
kubectl get pod <pod> -o yaml | grep -A5"
  "topologySpread"
# Simulate node drain
kubectl drain <node> --ignore-daemonsets
  --delete-emptydir-data
# Check Pod recovery
kubectl get pods -w
```

### VERIFY SUCCESS

- Pods distributed across nodes/az
- No single node hosts all replicas
- Workloads recover after node failure
- No downtime observed
- Cluster highly available

**HIGH AVAILABILITY DISTRIBUTION**

### EXAM TIP

Always validate after every change. Small validation steps prevent big production issues.

### FOCUS AREAS IN TASKS 51–55

- Advanced Scheduling
- Priority & Preemption
- Resource Governance
- Troubleshooting
- High Availability

### SUCCESS PRINCIPLE

A resilient cluster is built with the right scheduling policies, proper governance, and continuous validation.

# MOCK EXAM 3 (TASKS 56 TO 60)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

ADVANCED ADMINISTRATION. CLUSTER OPERATIONS. PRODUCTION READINESS.

## 56 TASK 56: RBAC ADMINISTRATION

- Create Roles
- Create ClusterRoles
- Configure RoleBindings
- Configure ClusterRoleBindings

### KEY ACTIONS

1. Create Role for namespace access
2. Create ClusterRole for cluster-wide access
3. Bind Role to users or ServiceAccounts
4. Bind ClusterRole to users or groups
5. Validate access for subjects

### USEFUL COMMANDS

```
# Create Role
kubectl create role dev-role --verb=get,list,watch \
  --resource=pods -n dev
# Create ClusterRole
kubectl create clusterrole view-cluster --verb=get,
list,watch --resources=nodes,pods,deployments
# RoleBinding
kubectl create rolebinding dev-role-binding \
  --role=dev-role --user=dev-user -n dev
# ClusterRoleBinding
kubectl create clusterrolebinding view-binding \
  --clusterrole=view-cluster --user=dev-user
# Check permissions
kubectl auth can-i list pods -n dev --as=dev-user
```

### VERIFY SUCCESS

- Roles created successfully
- ClusterRoles created successfully
- RoleBindings configured
- ClusterRoleBindings configured
- Access validated for users/subjects

RBAC FLOW

User / SA → Role / ClusterRole → Binding → Access Granted

## 57 TASK 57: SERVICE ACCOUNT SECURITY

- Create ServiceAccounts
- Assign least-privilege permissions
- Validate access

### KEY ACTIONS

1. Create ServiceAccount
2. Create Role with limited permissions
3. Bind Role to ServiceAccount
4. Use ServiceAccount in workload
5. Validate access using impersonation

### USEFUL COMMANDS

```
# Create ServiceAccount
kubectl create serviceaccount app-sa -n app-ns
# Create Role
kubectl create role app-role --verb=get,list \
  --resource=configmaps -n app-ns
# Bind Role to ServiceAccount
kubectl create rolebinding app-sa-binding \
  --role=app-role --serviceaccount=app-ns:app-sa
# Impersonate ServiceAccount
kubectl auth can-i get configmaps -n app-ns \
  --as=system:serviceaccount:app-ns:app-sa
```

### VERIFY SUCCESS

- ServiceAccount created
- Least-privilege permissions applied
- Workload uses ServiceAccount
- Access validated successfully
- No excessive permissions granted

SERVICE ACCOUNT USAGE

ServiceAccount SA) → Pod / Deployment → Authorized Access

## 58 TASK 58: AUTHORIZATION TROUBLESHOOTING

- Diagnose RBAC permission failures
- Rosalve access denied errors
- Verify authorization

### KEY ACTIONS

1. Reproduce the permission error
2. Review Roles and Bindings
3. Identify missing permissions or wrong namespace
4. Apply correct RBAC rules
5. Re-test and verify access

### USEFUL COMMANDS

```
# Check what user can do
kubectl auth can-i <verb> <resource> [-n <ns>]
# Describe Role / ClusterRole
kubectl describe role <role> -n <ns>
kubectl describe clusterrole <role>
# Describe RoleBinding / ClusterRoleBinding
kubectl describe rolebinding <rb> -n <ns>
kubectl describe clusterrolebinding <crb>
# Who am I (current user)
kubectl auth whoami
kubectl auth can-i --list
```

### VERIFY SUCCESS

- Permission issue identified
- Correct RBAC rule applied
- Access works as expected
- No unnecessary privileges
- Authorization verified

TROUBLESHOOT RBAC FLOW

Access Denied → Investigate RBAC → Fix Permissions → Access Granted

## 59 TASK 59: PRODUCTION SECURITY VALIDATION

- Audit cluster permissions
- Review Secrets usage
- Validate Security Contexts
- Confirm least-privilege implementation

### KEY ACTIONS

1. Audit ClusterRoles and ClusterRoleBindings
2. Review ServiceAccount permissions
3. Inspect Secrets and their usage
4. Validate Pod Security Contexts
5. Ensure least-privilege across the cluster

### USEFUL COMMANDS

```
# List ClusterRoles and bindings
kubectl get clusterrole,clusterrolebinding
# List ServiceAccounts
kubectl get sa -A
# List Secrets
kubectl get secrets -A
# Check Pod Security Contexts
kubectl get pods -A -o jsonpath="{..securityContext}"
# Audit permissions
kubectl auth can-i --list --all-namespaces
```

### VERIFY SUCCESS

- No excessive cluster permissions
- ServiceAccounts follow least privilege
- Secrets are properly managed
- Security Contexts are enforced
- Cluster security posture validated

SECURITY CHECKLIST

- RBAC Least Privilege
- Secrets Encrypted / Minimal Access
- Security Contexts Enforced
- No Privilege Escalation Paths
- Audit Complete

## 60 TASK 60: ENTERPRISE ADMINISTRATION SCENARIO

- Perform full cluster administration
- Upgrade components where required
- Configure scheduling policies
- Implement secure RBAC
- Validate complete cluster functionality
- Confirm production readiness before submission

### KEY ACTIONS

1. Review cluster state and requirements
2. Upgrade control plane or nodes if needed
3. Configure advanced scheduling policies
4. Implement RBAC and security policies
5. Validate workloads and services
6. Confirm cluster health and production readiness

### USEFUL COMMANDS

```
# Cluster overview
kubectl get nodes
kubectl get pods -A
kubectl get cs
# Upgrade (example for control plane)
kubeadm upgrade plan
kubeadm upgrade apply v1.xx.x
# Scheduling checks
kubectl get pods -o wide
kubectl get events --sort-by=.lastTimestamp
# Final validation
kubectl get all -A
kubectl get nodes -o wide
```

### VERIFY SUCCESS

- All components healthy
- Upgrades completed successfully
- Scheduling policies working
- RBAC secure and validated
- Workloads running as expected
- No critical events or errors
- Cluster production-ready

ENTERPRISE READINESS

Admin → Upgrade → Scheduling → RBAC → Validate

### EXAM TIP

This section tests real-world administration skills. Work methodically, verify every step, and validate the cluster before submission.

### FOCUS AREAS IN TASKS 56–60

- RBAC & Security
- Upgrades & Maintenance
- Scheduling & Availability
- Troubleshooting
- Production Readiness

### SUCCESS PRINCIPLE

A secure, well-administered cluster is reliable, observable, and ready for production at any time.

# MOCK EXAM 3 SOLUTIONS (TASKS 41–60)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

ADVANCED ADMINISTRATION. CLUSTER OPERATIONS. PRODUCTION READINESS.

## TASK 41: CLUSTER ADMINISTRATION

1. **Configure multiple kubeconfig contexts**
```
kubectl config set-context cluster1 \
--cluster=cluster1 --user=user1
kubectl config set-context cluster2 \
--cluster=cluster2 --user=user2
```
2. **Switch between clusters**
```
kubectl config use-context cluster1
kubectl config use-context cluster2
```
3. **Verify cluster health**
```
kubectl get componentstatuses
kubectl get cs
```
4. **Validate node readiness**
```
kubectl get nodes
kubectl get nodes -o wide
```

### CHECK HEALTH

- All components healthy ✓
- Nodes Ready ✓
- No error events ✓
- CoreDNS running ✓
- kube-system stable ✓

## TASK 42: NODE ADMINISTRATION

1. **Label worker nodes**
```
kubectl label node worker1 \
env=prod tier=backend
```
2. **Apply taints**
```
kubectl taint nodes worker2 \
dedicated=frontend:NoSchedule
```
3. **Configure tolerations (in Pod spec)**
```
tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "frontend"
    effect: "NoSchedule"
```
4. **Schedule workloads correctly**
```
kubectl get pods -o wide
kubectl describe pod <pod-name> |
        grep -i tolerations
```

### TAINTS & TOLERATIONS

Taint on node

dedicated=frontend:NoSchedule

↓

Pod with toleration

### VERIFY

- ✓ Pod scheduled on correct node
- ✓ No Pending pods
- ✓ Taints respected

## TASK 43: CONTROL PLANE MAINTENANCE

1. **Inspect control plane components**
```
kubectl get componentstatuses
kubectl get cs
```
2. **Verify static Pods**
```
kubectl get pods -n kube-system -o wide \
-l component=kube-apiserver
kubectl describe pod kube-apiserver-<node> \
-n kube-system
```
3. **Validate etcd status**
```
ETCDCTL_API=3 etcdctl --endpoints=https://
127.0.0.1:2379 endpoint health
ETCDCTL_API=3 etcdctl member list
```
4. **Restore failed control plane services**
```
systemctl restart kube-apiserver
systemctl restart kube-controller-manager
systemctl restart kube-scheduler
```

### CONTROL PLANE COMPONENTS

- kube-apiserver
- kube-scheduler
- kube-controller-manager
- etcd
- CoreDNS

### STATIC POD LOCATION

/etc/kubernetes/manifests/

Static Pods are managed by the kubelet.
Do not delete or edit directly unless necessary.

## TASK 44: CLUSTER UPGRADE PLANNING

1. **Review upgrade path**
```
kubeadm upgrade plan
```
2. **Check version compatibility**
```
kubeadm upgrade plan
kubeadm config images list
```
3. **Validate version skew**
```
kubectl version --short
kubeadm version
```
4. **Prepare rollback strategy**
```
kubeadm upgrade plan --allow-experimental-
upgrades
```

Backup etcd!
Ensure full cluster backup before upgrade.

### VERSION SKEW RULES

- kube-apiserver may be newer by up to 1 minor version than kubelet.
- kubelet may be older by up to 1 minor version than kube-apiserver.
- kubectl may be newer or older by up to 1 minor version.

### ROLLBACK CHECKLIST

- ☑ Backup etcd
- ☑ Document current state
- ☑ Test upgrade in staging
- ☑ Know rollback steps

## TASK 45: PRODUCTION CLUSTER VALIDATION

1. **Verify cluster-wide resources**
```
kubectl get nodes
kubectl get cs
kubectl get storageclass
```
2. **Validate namespaces**
```
kubectl get namespaces
kubectl get ns
```
3. **Inspect workloads**
```
kubectl get deployments -A
kubectl get pods -A -o wide
kubectl get svc -A
```
4. **Confirm cluster stability**
```
kubectl get events --all-namespaces
--sort-by=.lastTimestamp
kubectl top nodes
kubectl top pods -A
```

### VALIDATION CHECKLIST

- ✓ All nodes Ready
- ✓ No critical events
- ✓ Workloads running
- ✓ Services reachable
- ✓ Resources healthy
- ✓ Cluster stable

### HEALTH COMMANDS

```
kubectl get cs
kubectl get nodes
kubectl get pods -A
kubectl get events -A
```

## QUICK REFERENCE COMMANDS

| | |
|---|---|
| Contexts | `kubectl config get-contexts`<br>`kubectl config use-context <name>` |
| Nodes | `kubectl get nodes -o wide`<br>`kubectl describe node <node>` |
| Pods | `kubectl get pods -A -o wide`<br>`kubectl describe pod <pod> -n <ns>` |
| Taints | `kubectl taint nodes <node> key=value:NoSchedule`<br>`kubectl describe node <node> \| grep -i taints` |
| Labels | `kubectl label node <node> key=value`<br>`kubectl get nodes --show-labels` |
| Upgrade | `kubeadm upgrade plan`<br>`kubeadm upgrade apply vX.X.X` |
| Certificates | `kubeadm certs check-expiration`<br>`kubeadm certs renew all` |
| etcd | `etcdctl --endpoints=https://127.0.0.1:2379 endpoint health`<br>`etcdctl snapshot save <file.db>` |
| RBAC | `kubectl auth can-i <verb> <resource> [-n <ns>]`<br>`kubectl get roles,clusterroles -A` |

## SUCCESS CRITERIA FOR MOCK EXAM 3

- ✓ Cluster healthy and stable
- ✓ All nodes Ready
- ✓ No unexpected Pending pods
- ✓ Control plane components running
- ✓ etcd healthy and backed up
- ✓ Upgrades planned and executed
- ✓ Version compatibility maintained
- ✓ Workloads scheduled correctly
- ✓ Resources governed and enforced
- ✓ RBAC least-privilege implemented
- ✓ Access authorized correctly
- ✓ Security best practices applied
- ✓ Cluster resilient and highly available
- ✓ Production readiness confirmed
- ✓ All tasks completed successfully

### EXAM TIP

Read tasks carefully.
Plan before you execute.
Verify each step.
Time management is key to success!

### FOCUS AREAS IN MOCK EXAM 3

- Cluster Administration
- Upgrades & Maintenance
- Advanced Scheduling
- RBAC & Security
- Production Validation

### REMEMBER

- Small mistakes cause big failures.
- Verify every change.
- Use dry-run where possible.
- Think like a production engineer.
- Stay calm and methodical.

### SUCCESS PRINCIPLE

A well-administered Kubernetes cluster is secure, resilient, observable, and ready for production workloads.
You've got this!

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.



# MOCK EXAM 3 PERFORMANCE REVIEW

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

ADVANCED ADMINISTRATION. CLUSTER OPERATIONS. PRODUCTION READINESS.

## EXAM OVERVIEW

Total Tasks
**60**

Total Time
**3 HOURS**

Task Format
Multiple choice + Performance

Passing Score
**66% OR HIGHER**

Exam Focus
Enterprise cluster administration, troubleshooting, and production operations

## TASK DISTRIBUTION

| DOMAIN | TASKS | PERCENTAGE |
|---|---|---|
| Cluster Architecture | 1 – 10 | 16.7% |
| Workloads & Scheduling | 11 – 20 | 16.7% |
| Services & Networking | 21 – 30 | 16.7% |
| Storage & Persistent Data | 31 – 40 | 16.7% |
| Cluster Maintenance | 41 – 50 | 16.7% |
| Security & Access Control | 51 – 60 | 16.7% |
| TOTAL | 60 | 100% |

## TIME MANAGEMENT

- Total Exam Time: 180 minutes
- Average per task: 3 minutes
- Complex tasks: 6 – 8 minutes
- Review time: 15 – 20 minutes
- Do not spend too long on one task.

### TIME STRATEGY

- ☑ Read the task carefully
- ☑ Plan before you execute
- ☑ Complete easier tasks first
- ☑ Mark and return to difficult tasks
- ☑ Review all tasks if time permits

## PERFORMANCE SUMMARY BY DOMAIN

| DOMAIN | STRENGTH INDICATORS | COMMON CHALLENGES | FOCUS RECOMMENDATIONS | SELF SCORE (1 – 5) |
|---|---|---|---|---|
| Cluster Architecture | • Multi-master HA setup<br>• Networking and DNS<br>• Cluster components<br>• Health and readiness | • etcd endpoints<br>• Component failures<br>• Cluster upgrades<br>• Version compatibility | • Study HA topologies<br>• Practice kubectl and etcd<br>• Understand upgrade workflow<br>• Review cluster health checks | ☐ ☐ ☐ ☐ ☐ |
| Workloads & Scheduling | • Deployments & ReplicaSets<br>• Jobs & CronJobs<br>• Scheduling policies<br>• Resource management | • Scheduling conflicts<br>• Affinity rules<br>• Resource limits<br>• Priority & preemption | • Practice scheduling scenarios<br>• Understand resource governance<br>• Review priority classes<br>• Analyze pending pods | ☐ ☐ ☐ ☐ ☐ |
| Services & Networking | • Services and Endpoints<br>• Ingress and TLS<br>• Network Policies<br>• CoreDNS and DNS | • Network connectivity<br>• DNS resolution<br>• Policy misconfigurations<br>• Ingress routing | • Practice network troubleshooting<br>• Validate policies<br>• Review service discovery<br>• Test Ingress rules | ☐ ☐ ☐ ☐ ☐ |
| Storage & Persistent Data | • PV, PVC, StorageClass<br>• Dynamic provisioning<br>• Volume lifecycle<br>• Backup and restore | • Storage binding<br>• Permission issues<br>• Backup failures<br>• Data recovery | • Practice storage workflows<br>• Test backup and restore<br>• Review reclaim policies<br>• Understand access modes | ☐ ☐ ☐ ☐ ☐ |
| Cluster Maintenance | • Cluster upgrades<br>• Node maintenance<br>• Certificate management<br>• Backup and recovery | • Upgrade failures<br>• Version mismatches<br>• Certificate expiry<br>• Component downtime | • Practice upgrade process<br>• Review kubeadm workflow<br>• Understand rollback steps<br>• Test maintenance scenarios | ☐ ☐ ☐ ☐ ☐ |
| Security & Access Control | • RBAC and permissions<br>• ServiceAccounts<br>• Security Contexts<br>• Cluster hardening | • RBAC misconfigurations<br>• Access denied errors<br>• Secret management<br>• Privilege escalation | • Practice RBAC scenarios<br>• Enforce least privilege<br>• Audit permissions<br>• Review security best practices | ☐ ☐ ☐ ☐ ☐ |

## SUCCESS INDICATORS

- ☑ Cluster is healthy and stable
- ☑ All workloads running as expected
- ☑ No pending pods
- ☑ Networking fully functional
- ☑ Storage mounted and accessible
- ☑ Security policies enforced
- ☑ RBAC working correctly
- ☑ Upgrades and maintenance successful
- ☑ Backups and recovery validated
- ☑ All tasks completed within time

## COMMON MISTAKES TO AVOID

- ✗ Not reading the full task requirements
- ✗ Skipping validation steps
- ✗ Incorrect context or namespace
- ✗ Forgetting to save configurations
- ✗ Not verifying resource status
- ✗ Ignoring error messages
- ✗ Incorrect permissions or RBAC
- ✗ Not completing all task steps
- ✗ Poor time management
- ✗ Forgetting to review before submission

## FINAL EXAM CHECKLIST

- ☑ All tasks attempted
- ☑ Cluster healthy and stable
- ☑ Workloads running correctly
- ☑ Networking and DNS verified
- ☑ Storage and PVs validated
- ☑ RBAC and security validated
- ☑ Upgrades and maintenance complete
- ☑ Backups successful
- ☑ Time utilized effectively
- ☑ Ready for submission!

YOU HAVE PREPARED. YOU HAVE PRACTICED. YOU ARE READY.
STAY CALM. PLAN. EXECUTE. VERIFY. SUCCEED.

TRUST THE PROCESS
FINISH STRONG!



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# MOCK EXAM 4 OVERVIEW

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

ADVANCED ADMINISTRATION. CLUSTER OPERATIONS. PRODUCTION READINESS.

## EXAM SUMMARY

**Total Tasks**
60

**Total Time**
3 HOURS

**Task Format**
Multiple choice + Performance tasks

**Passing Score**
66% OR HIGHER

**Exam Focus**
Enterprise Kubernetes administration, advanced operations, troubleshooting, and production readiness.

## WHAT MOCK EXAM 4 COVERS

**Advanced Cluster Operations**
Large scale cluster management, node lifecycle, and component upgrades.

**Workloads & Controllers**
Advanced workload management, controllers, and job automation.

**Networking & Ingress**
Advanced networking, multi-cluster connectivity, and network policies.

**Security & Compliance**
Advanced RBAC, Pod security, network security, and compliance controls.

**Storage & Data Protection**
Advanced storage, CSI drivers, backups, snapshots, and disaster recovery.

**Monitoring & Observability**
Cluster monitoring, logging, alerting, and performance optimization.

## EXAM ENVIRONMENT

## KEY EXAM RULES

- Do not share solutions or talk during exam
- Do not leave the exam environment
- Do not use external documentation
- Complete all tasks within the time limit
- Submit before time expires
- Follow instructions carefully

## TASK DISTRIBUTION BY DOMAIN

| DOMAIN | TASK RANGE | NUMBER OF TASKS | PERCENTAGE | FOCUS AREAS |
|---|---|---|---|---|
| Cluster Operations & Maintenance | 1 – 10 | 10 | 16.7% | Cluster scaling, component management, upgrades, node maintenance, and high availability. |
| Workloads & Controllers | 11 – 20 | 10 | 16.7% | Deployments, StatefulSets, DaemonSets, Jobs, CronJobs, and custom controllers. |
| Networking & Ingress | 21 – 30 | 10 | 16.7% | CNI plugins, Ingress controllers, DNS, NetworkPolicies, service mesh basics, and multi-cluster networking. |
| Security & Compliance | 31 – 40 | 10 | 16.7% | RBAC advanced, ServiceAccounts, Pod Security Standards, secrets management, and cluster hardening. |
| Storage & Data Protection | 41 – 50 | 10 | 16.7% | CSI drivers, storage classes, snapshots, backups, restore, and disaster recovery strategies. |
| Monitoring & Observability | 51 – 60 | 10 | 16.7% | Metrics, logging, alerting, tracing, resource monitoring, and performance tuning. |
| TOTAL | 1 – 60 | 60 | 100% | |

## SUCCESS FACTORS

- Strong understanding of Kubernetes architecture and internals.
- Hands-on experience with enterprise workloads and operations.
- Deep knowledge of security and compliance best practices.
- Practice troubleshooting real-world scenarios.
- Time management and task prioritization.

## CHALLENGES TO EXPECT

- Complex multi-step tasks
- Advanced troubleshooting
- Resource constraints
- Time pressure
- Real-world scenarios
- Interdependent configurations
- Production-level requirements

## TIME ALLOCATION GUIDE

Total Time: 180 minutes

Plan your time. Execute efficiently. Review carefully.

## EXAM MINDSET

Think like a production Kubernetes administrator.

- Plan before you execute.
- Verify every step.
- Check logs and events.
- Validate continuously.
- Deliver production-ready solutions.

THIS EXAM TESTS YOUR ABILITY TO ADMINISTER KUBERNETES AT SCALE.
YOU HAVE THE KNOWLEDGE. YOU HAVE THE SKILLS. SUCCEED!

VERIQTA

# MOCK EXAM 4 (TASKS 61 TO 65)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

HEAVY TROUBLESHOOTING. PRODUCTION FAILURES. DEBUGGING.

## 61 TASK 61: PRODUCTION POD FAILURE

### OBJECTIVES

- Diagnose CrashLoopBackOff
- Inspect logs
- Verify Events
- Identify root cause
- Restore application

### KEY ACTIONS

```
# Check pod status
kubectl get pods -n <ns>
# Describe pod
kubectl describe pod <pod> -n <ns>
# View logs
kubectl logs <pod> -n <ns> --previous
# Check events
kubectl get events -n <ns> --sort-by=.lastTimestamp
# Fix the issue and restart pod
kubectl rollout restart deployment/<deploy> -n <ns>
```

### SUCCESS CRITERIA

- Pod is running
- No CrashLoopBackOff
- No new errors in Events
- Application is healthy
- All containers are ready

STATUS CHECK

Failed → Running

## 62 TASK 62: IMAGE PULL FAILURE

### OBJECTIVES

- Diagnose ImagePullBackOff
- Verify image repository
- Fix image tag
- Validate successful deployment

### KEY ACTIONS

```
# Check pod status
kubectl get pods -n <ns>
# Describe pod
kubectl describe pod <pod> -n <ns>
# Check events
kubectl get events -n <ns> --sort-by=.lastTimestamp
# Verify image
kubectl set image deployment/<deploy> <container
>=<image>:<tag> -n <ns>
# Rollout restart
kubectl rollout restart deployment/<deploy> -n <ns>
```

### SUCCESS CRITERIA

- Image pulls successfully
- No ImagePullBackOff
- Pod is running
- Deployment is successful
- Application is healthy

STATUS CHECK

Failed → Running

## 63 TASK 63: PENDING POD INVESTIGATION

### OBJECTIVES

- Diagnose Pending Pods
- Check scheduler events
- Resolve resource constraints
- Validate successful scheduling

### KEY ACTIONS

```
# Check pod status
kubectl get pods -n <ns>
# Describe pod
kubectl describe pod <pod> -n <ns>
# Check node resources
kubectl top nodes
# Check resource limits
kubectl describe nodes
# Fix resources and wait
kubectl get pods -n <ns> -w
```

### SUCCESS CRITERIA

- Pod is scheduled
- No Pending status
- All resource constraints resolved
- Pod is running
- Application is accessible

STATUS CHECK

Pending → Running

## 64 TASK 64: NODE FAILURE RECOVERY

### OBJECTIVES

- Diagnose NotReady node
- Inspect kubelet
- Verify container runtime
- Restore node health

### KEY ACTIONS

```
# Check nodes
kubectl get nodes
# Describe node
kubectl describe node <node>
# Check kubelet status (on node)
systemctl status kubelet
# Check container runtime (on node)
systemctl status containerd  # or docker
# Restart services if needed
systemctl restart kubelet
# Verify node is ready
kubectl get nodes
```

### SUCCESS CRITERIA

- Node status is Ready
- Kubelet is running
- Container runtime is healthy
- No critical errors in Events
- Pods are running on node

STATUS CHECK

NotReady → Ready

## 65 TASK 65: PRODUCTION APPLICATION RECOVERY

### OBJECTIVES

- Restore a failed production workload
- Validate Services
- Confirm application availability
- Verify complete recovery

### KEY ACTIONS

```
# Check deployments
kubectl get deploy -n <ns>
# Check pods
kubectl get pods -n <ns>
# Validate services
kubectl get svc -n <ns>
# Check endpoints
kubectl get endpoints -n <ns>
# Test application
kubectl port-forward svc/<svc> <port>:<port> -n <ns>
```

### SUCCESS CRITERIA

- All pods running
- Services are healthy
- Endpoints are correct
- Application is accessible
- Full recovery verified

STATUS CHECK

Failed → Healthy

### EXAM TIP

Read the problem carefully.
Use describe, logs, and events for deep analysis.
Fix the root cause, not just the symptom.

### FOCUS ON:

- Logs
- Events
- Resource constraints
- Networking
- Configuration
- Knowledge of Kubernetes components

VERIQTA

LEARN. PRACTICE. ANALYZE. SUCCEED.

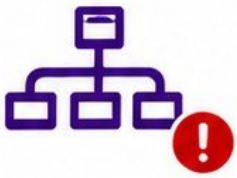


# MOCK EXAM 4 (TASKS 66 TO 70)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

HEAVY TROUBLESHOOTING. PRODUCTION FAILURES. DEBUGGING.

## 66 TASK 66: SERVICE CONNECTIVITY FAILURE



### OBJECTIVES

- Diagnose Service communication issues
- Verify selectors
- Restore Endpoints
- Validate traffic flow

### KEY ACTIONS

```
# Check service status
kubectl get svc -n <ns>
# Verify selectors
kubectl get endpoints <service> -n <ns>
# Check pod labels
kubectl get pods --show-labels -n <ns>
# Restore endpoints if missing
kubectl delete endpoints <service> -n <ns>
# Test traffic flow
curl -s http://<service>.<ns>.svc.cluster.local
```

### SUCCESS CRITERIA

- Service communication restored
- Selectors match pods
- Endpoints available
- Traffic flows successfully

## 67 TASK 67: INGRESS FAILURE

### OBJECTIVES

- Diagnose Ingress routing
- Verify TLS
- Fix host rules
- Restore external access

### KEY ACTIONS

```
# Check ingress status
kubectl get ingress -n <ns>
# Verify TLS and certificates
kubectl get secret -n <ns>
# Inspect ingress rules
kubectl describe ingress <name> -n <ns>
# Fix host rules
kubectl edit ingress <name> -n <ns>
# Test external access
curl -k https://<host>
```

### SUCCESS CRITERIA

- Ingress resources healthy
- TLS configured correctly
- Host rules fixed
- External access restored

## 68 TASK 68: DNS PRODUCTION FAILURE

### OBJECTIVES

- Diagnose CoreDNS issues
- Validate Service discovery
- Restore DNS resolution
- Confirm application communication

### KEY ACTIONS

```
# Check CoreDNS pods
kubectl get pods -n kube-system
# Verify CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns
# Test DNS resolution
kubectl run dns-test --image=busybox --rm --
nslookup <service>.<ns>
# Restart CoreDNS if needed
kubectl rollout restart deployment/coredns -n kube-system
```

### SUCCESS CRITERIA

- CoreDNS healthy
- Service discovery working
- DNS resolution restored
- Applications communicate successfully

## 69 TASK 69: NETWORK POLICY FAILURE

### OBJECTIVES

- Diagnose blocked traffic
- Review NetworkPolicies
- Restore required communication
- Validate isolation rules

### KEY ACTIONS

```
# Check network policies
kubectl get networkpolicy -n <ns>
# Describe active policies
kubectl describe networkpolicy <name> -n <ns>
# Test connectivity between pods
kubectl exec -it <pod> -- curl <target-pod>
# Fix policy rules (if needed)
kubectl edit networkpolicy <name> -n <ns>
```

### SUCCESS CRITERIA

- Blocked traffic restored
- NetworkPolicies reviewed
- Required communication working
- Isolation rules maintained

## 70 TASK 70: END-TO-END NETWORKING RECOVERY

### OBJECTIVES

- Verify Pod networking
- Verify Services
- Verify Ingress
- Validate production traffic

### KEY ACTIONS

```
# Verify pod connectivity
kubectl exec -it <pod> -- ping <pod>
# Verify services
kubectl get svc -n <ns>
# Verify ingress
kubectl get ingress -n <ns>
# Test production traffic
curl -I https://<app>.<ns>.com
```

### SUCCESS CRITERIA

- Pod networking verified
- Services working
- Ingress functioning
- Production traffic validated

### EXAM TIP

Follow the troubleshooting process:

1. Gather information 2. Analyze 3. Fix 4. Verify

Think like a production engineer. Solve root cause, not just the symptom.

### FOCUS ON:

- Logs
- Events
- Resource status
- Networking
- Storage
- Application health

TRUST THE PROCESS FINISH STRONG!



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# MOCK EXAM 4 (TASKS 71 TO 75)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

STORAGE & DATA RECOVERY. PRODUCTION FAILURES. DEBUGGING.

## 71 TASK 71: PERSISTENT VOLUME FAILURE

**OBJECTIVES**
- Diagnose PVC Pending
- Resolve binding issues
- Validate storage access

**KEY ACTIONS**
- Check PVC status: kubectl get pvc
- Describe PVC: kubectl describe pvc
- Inspect StorageClass: kubectl get sc
- Check PV availability: kubectl get pv
- Fix binding issues (StorageClass/PV)
- Validate persistent storage access

**SUCCESS CRITERIA**
- PVC is Bound
- Storage is accessible
- Application starts successfully
- Data is readable and writable

STATUS CHECK: Pending → Running

## 72 TASK 72: STORAGECLASS FAILURE

**OBJECTIVES**
- Diagnose dynamic provisioning
- Repair StorageClass
- Validate automatic provisioning

**KEY ACTIONS**
- Check StorageClass status
- Describe StorageClass
- Identify provisioning errors
- Fix StorageClass configuration
- Validate dynamic volume creation

**SUCCESS CRITERIA**
- StorageClass is healthy
- Dynamic provisioning works
- PVCs are created automatically
- Applications access storage

STATUS CHECK: Failed → Running

## 73 TASK 73: VOLUME MOUNT FAILURE

**OBJECTIVES**
- Diagnose mount errors
- Repair volume configuration
- Verify mounted storage

**KEY ACTIONS**
- Check Pod events
- Inspect mount errors
- Validate volume configuration
- Fix mount issues
- Verify storage mount

**SUCCESS CRITERIA**
- Volume mounts successfully
- Pod starts without errors
- Storage is accessible in pod
- Application runs correctly

STATUS CHECK: Failed → Running

## 74 TASK 74: SECRETS AND CONFIGMAP FAILURE

**OBJECTIVES**
- Diagnose missing configuration
- Restore Secrets
- Restore ConfigMaps
- Validate application startup

**KEY ACTIONS**
- Check for missing Secrets/ConfigMaps
- Describe Secret: kubectl describe secret
- Describe ConfigMap: kubectl describe configmap
- Recreate missing resources
- Restart application pods
- Validate startup

**SUCCESS CRITERIA**
- Secrets and ConfigMaps exist
- Application loads successfully
- Configuration is applied
- Workload is running

STATUS CHECK: Failed → Running

## 75 TASK 75: PRODUCTION DATA RECOVERY

**OBJECTIVES**
- Recover persistent storage
- Validate application data
- Confirm workload recovery

**KEY ACTIONS**
- Restore from backup (if needed)
- Validate data integrity
- Reattach storage volumes
- Restart application
- Confirm full workload recovery

**SUCCESS CRITERIA**
- Persistent storage is recovered
- Application data is intact
- Workloads are running
- Business functionality restored

STATUS CHECK: Not Ready → Ready

**PRODUCTION STORAGE AND DATA RECOVERY:**
KEEP DATA SAFE. RESTORE FAST. VERIFY COMPLETELY.

VERIQTA



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# MOCK EXAM 4 (TASKS 76 TO 80)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

HEAVY TROUBLESHOOTING. PRODUCTION FAILURES. DEBUGGING.

## 76 TASK 76: CONTROL PLANE FAILURE

### OBJECTIVES

- Diagnose API Server issues
- Validate Scheduler
- Restore Controller Manager
- Verify cluster health

### KEY ACTIONS

```
# Check API Server
kubectl get --raw='/healthz'
kubectl logs -n kube-system -l component=kube-apiserver
# Validate Scheduler
kubectl logs -n kube-system -l component=kube-scheduler
# Restore Controller Manager
kubectl logs -n kube-system -l component=kube-controller-manager
# Verify cluster components
kubectl get componentstatuses 2>/dev/null || \
kubectl get --raw='/readyz?verbose'
# Verify overall cluster health
kubectl get nodes
kubectl get pods --all-namespaces
```

### SUCCESS CRITERIA

- API Server is healthy
- Scheduler is running
- Controller Manager is running
- All control plane components are healthy
- Cluster is fully operational

STATUS CHECK: Unhealthy → Healthy

## 77 TASK 77: RBAC FAILURE

### OBJECTIVES

- Diagnose authorization errors
- Repair Roles
- Repair RoleBindings
- Validate user access

### KEY ACTIONS

```
# Check RBAC errors
kubectl auth can-i --list
kubectl get events --all-namespaces | grep -i forbidden
# Repair Roles
kubectl get roles,clusterroles -A
kubectl apply -f role-fix.yaml
# Repair RoleBindings
kubectl get rolebindings,clusterrolebindings -A
kubectl apply -f rolebinding-fix.yaml
# Validate user access
kubectl auth can-i get pods -n default --as=<user>
kubectl auth can-i --list --as=<user>
```

### SUCCESS CRITERIA

- No authorization errors
- Roles are correctly configured
- RoleBindings are correctly configured
- Users have appropriate access
- Access validation is successful

STATUS CHECK: Access Denied → Access Granted

## 78 TASK 78: MULTI-FAILURE PRODUCTION INCIDENT

### OBJECTIVES

- Resolve networking failure
- Resolve storage failure
- Resolve workload failure
- Validate full application recovery

### KEY ACTIONS

```
# Diagnose all symptoms
kubectl get nodes,pods,svc,pvc -A
kubectl get events -A --sort-by='.lastTimestamp'
# Fix networking issue
(Apply NetworkPolicy/Service/Ingress fixes)
# Fix storage issue
(PVC, StorageClass, PV binding or mounts)
# Fix workload issue
kubectl describe pod <pod> -n <ns>
# Validate full recovery
kubectl get pods,svc,ingress,pvc -A
```

### SUCCESS CRITERIA

- Networking is restored
- Storage is working
- Workloads are running
- Services are reachable
- Application is fully recovered

STATUS CHECK: Incident Open → Recovered

## 79 TASK 79: ENTERPRISE DEBUGGING CHALLENGE

### OBJECTIVES

- Analyze logs
- Correlate Events
- Identify root cause
- Implement corrective actions

### KEY ACTIONS

```
# Collect evidence
kubectl get events -A --sort-by='.lastTimestamp' > events.log
kubectl get pods -A -o wide > pods.log
kubectl describe <resource> -n <ns> > describe.log
# Analyze logs
kubectl logs <pod> -n <ns> --all-containers --tail=200
# Correlate and identify root cause
# Document findings
# Apply corrective actions
# Validate resolution
```

### SUCCESS CRITERIA

- Root cause is clearly identified
- Evidence is properly analyzed
- Corrective actions are implemented
- Issue is fully resolved
- Application is stable

STATUS CHECK: Root Cause Unknown → Resolved

## 80 TASK 80: COMPLETE PRODUCTION INCIDENT RESPONSE

### OBJECTIVES

- Perform end-to-end troubleshooting
- Recover cluster services
- Validate workloads
- Verify networking
- Verify storage
- Confirm production readiness before submission

### KEY ACTIONS

```
# 1. Assess the incident
kubectl get nodes,svc,pods,pvc,ingress -A
kubectl get events -A --sort-by='.lastTimestamp'
# 2. Troubleshoot and fix
(Networking > Storage > Workloads > Control Plane)
# 3. Validate recovery
kubectl get pods -A
kubectl get svc,ingress -A
kubectl get pvc -A
kubectl get nodes
# 4. Final verification
kubectl get --raw='/readyz?verbose'
kubectl get componentstatuses
# 5. Confirm production readiness
# Submit when fully recovered
```

### SUCCESS CRITERIA

- All issues are resolved
- Cluster services are healthy
- All workloads are running
- Networking is fully operational
- Storage is accessible
- Control plane is healthy
- Application is available
- Production is ready
- Ready for submission

STATUS CHECK: Not Ready → Ready

### EXAM TIP

Follow a systematic troubleshooting approach:
Observe → Analyze → Isolate → Fix → Verify
Document your steps and verify full recovery before moving to the next task.

### FOCUS ON

- Root Cause Analysis
- Recovery Validation
- Production Readiness
- Deep Troubleshooting
- Log Analysis
- Event Correlation

# MOCK EXAM 4 SOLUTIONS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

HEAVY TROUBLESHOOTING. PRODUCTION FAILURES. DEBUGGING.

These are example solutions and key commands. Actual exam output may vary. Always validate in your environment.

| TASK ID | SCENARIO | ROOT CAUSE (EXAMPLE) | SOLUTION SUMMARY | KEY COMMANDS (EXAMPLES) |
|---|---|---|---|---|
| 61 | PRODUCTION POD FAILURE (CrashLoopBackOff) | • Application error<br>• Misconfiguration (config/secret/env)<br>• Dependency not available<br>• Resource limits | ✓ Inspect logs and events<br>✓ Identify root cause (app/config)<br>✓ Fix configuration or code<br>✓ Redeploy or restart pod<br>✓ Verify application is stable | # Check pod status and events<br>kubectl get pod <pod> -n <ns><br>kubectl describe pod <pod> -n <ns><br># View logs<br>kubectl logs <pod> -n <ns> --previous<br># Check events<br>kubectl get events -n <ns> --sort-by=.lastTimestamp |
| 62 | IMAGE PULL FAILURE (ImagePullBackOff) | • Wrong image name or tag<br>• Image does not exist<br>• Private registry authentication issue<br>• Network/registry unreachable | ✓ Verify image name and tag<br>✓ Check image repository access<br>✓ Fix image name/tag or credentials<br>✓ Update deployment<br>✓ Validate successful deployment | # Describe pod<br>kubectl describe pod <pod> -n <ns><br># Check image repository<br>kubectl get pods -n <ns> -o wide<br># Verify image pull manually<br>crictl pull <image><br># Check registry secret<br>kubectl get secret -n <ns> |
| 63 | PENDING POD INVESTIGATION (Pending) | • Insufficient CPU/Memory<br>• Node selectors/ taints mismatch<br>• PVC not bound<br>• Too many pods (max pods) | ✓ Check scheduler events<br>✓ Identify reason for pending<br>✓ Resolve resource/selector/ storage issues<br>✓ Validate pod is scheduled | # Check pending pods<br>kubectl get pods -n <ns> --field-selector=status.phase=Pending<br># Describe for events<br>kubectl describe pod <pod> -n <ns><br># Check node resources<br>kubectl top nodes<br># Check PVC status if used<br>kubectl get pvc -n <ns> |
| 64 | NODE FAILURE RECOVERY (NotReady) | • Kubelet stopped<br>• Container runtime (containerd/docker) down<br>• Network issue<br>• Disk/Resource pressure | ✓ Inspect node and kubelet<br>✓ Verify container runtime status<br>✓ Fix underlying issue<br>✓ Restart services if needed<br>✓ Validate node is Ready | # Check node status<br>kubectl get nodes<br># Describe node<br>kubectl describe node <node><br># Check kubelet logs (on node)<br>journalctl -u kubelet -xe<br># Check container runtime (containerd)<br>systemctl status containerd |
| 65 | PRODUCTION APPLICATION RECOVERY | • Pod crash or down<br>• Service misconfiguration<br>• Dependency failure<br>• Config/Secret/Storage issue | ✓ Identify failed component(s)<br>✓ Fix root cause<br>✓ Redeploy or restart workload<br>✓ Validate Services and Endpoints<br>✓ Confirm application availability<br>✓ Verify complete recovery | # Check all resources<br>kubectl get all -n <ns><br># Check services and endpoints<br>kubectl get svc,endpoints -n <ns><br># Validate application<br>kubectl get pods -n <ns><br>curl -I http://<service>.<ns>.svc.cluster.local<br># Check logs if needed<br>kubectl logs <pod> -n <ns> |

## GENERAL TROUBLESHOOTING TOOLBOX (USE WHEN NEEDED)

**PODS & EVENTS**
```
kubectl get pods -A
kubectl describe pod <pod>
kubectl get events -A
--sort-by=.lastTimestamp
```

**LOGS**
```
kubectl logs <pod> -n <ns>
--previous
kubectl logs -f <pod> -n <ns>
```

**NODES**
```
kubectl get nodes
kubectl describe node <node>
kubectl top nodes
```

**STORAGE**
```
kubectl get pvc,pv -A
kubectl describe pvc <pvc>
kubectl get sc
```

**NETWORKING**
```
kubectl get svc -A
kubectl get endpoints -A
kubectl get ingress -A
kubectl get netpol -A
```

**DNS**
```
kubectl get pods -n kube-system -l
[k8s-app=kube-dns
kubectl logs -n kube-system
deploy/coredns
```

**CLUSTER HEALTH**
```
kubectl cluster-info
kubectl get componentstatuses
kubectl get cs
```

Remember: In the exam, validate after every fix and ensure the system is fully functional before moving to the next task.

**EXAM TIP**
Stay calm, read carefully, triage the issue, fix methodically, and always validate your solution.

**FOCUS ON**
- ✓ Root Cause Identification
- ✓ Minimal Changes
- ✓ Complete Validation
- ✓ Production Readiness

**SUCCESS MINDSET**
Real production incidents test your troubleshooting skills, not just your Kubernetes knowledge.

**TIME MANAGEMENT**
Allocate time wisely, don't get stuck. Skip and return if needed.

VERIQTA
LEARN. PRACTICE. ANALYZE. SUCCEED.



# ROOT CAUSE ANALYSIS WALKTHROUGH

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

A SYSTEMATIC APPROACH TO FIND, FIX AND PREVENT PRODUCTION ISSUES.

Great engineers don't guess. They observe, isolate, verify, and fix.

| STEP | FOCUS | WHAT TO CHECK | KEY COMMANDS (EXAMPLES) | WHAT YOU ARE LOOKING FOR |
|---|---|---|---|---|
| 1 | **OBSERVE THE SYMPTOM**<br>Understand what is broken and the impact. | ✓ User report / Alert<br>✓ Affected system<br>✓ Time of incident<br>✓ Error messages<br>✓ Scope of impact | `# Check cluster overview`<br>`kubectl get nodes`<br>`kubectl get pods -A`<br>`# Check recent events`<br>`kubectl get events -A --sort-by=.lastTimestamp \| tail -n 20` | ✓ Which component is affected<br>✓ When it started<br>✓ How many users/workloads are impacted<br>✓ Any obvious error or warning |
| 2 | **GATHER EVIDENCE**<br>Collect facts before making any changes. | ✓ Logs<br>✓ Events<br>✓ Resource status<br>✓ Configuration<br>✓ Metrics (if available) | `# Describe the affected resource`<br>`kubectl describe <pod\|svc\|ing\|node\|pvc> <name>`<br>`# Get logs`<br>`kubectl logs <pod> -n <ns> --previous`<br>`# Check related events`<br>`kubectl get events -n <ns> --sort-by=.lastTimestamp` | ✓ Error messages in logs<br>✓ Events showing failures<br>✓ Configuration mismatches<br>✓ Resource or dependency issues |
| 3 | **ISOLATE THE ISSUE**<br>Narrow down the problem to the root cause. | ✓ Component dependency<br>✓ Recent changes<br>✓ Configuration changes<br>✓ Resource usage<br>✓ External dependencies | `# Check pod status and reason`<br>`kubectl get pods -n <ns> -o wide`<br>`# Describe endpoints`<br>`kubectl get endpoints -n <ns> <svc> -o yaml`<br>`# Check resource usage`<br>`kubectl top pods -n <ns>`<br>`kubectl top nodes` | ✓ Which component is failing<br>✓ What changed recently<br>✓ Missing or wrong configuration<br>✓ Resource limits or pressure<br>✓ External service down/unreachable |
| 4 | **IDENTIFY ROOT CAUSE**<br>Find the real reason, not just the error. | ✓ Misconfiguration<br>✓ Resource constraint<br>✓ Network / DNS issue<br>✓ Storage issue<br>✓ Bug / Application issue | `# Check configuration`<br>`kubectl get <resource> -n <ns> -o yaml`<br>`# Check DNS resolution`<br>`kubectl exec -n <ns> <pod> -- nslookup <service>`<br>`# Check network connectivity`<br>`kubectl exec -n <ns> <pod> -- curl -v <url>` | ✓ The primary failure reason<br>✓ Why the component failed<br>✓ How it affects other parts<br>✓ What must be fixed |
| 5 | **IMPLEMENT THE FIX**<br>Apply the right change to resolve the issue. | ✓ Configuration fix<br>✓ Restart / Redeploy<br>✓ Resource adjustment<br>✓ Policy / Permission fix<br>✓ Dependency restoration | `# Apply configuration / fix`<br>`kubectl apply -f <fixed-file.yaml>`<br>`# Restart deployment`<br>`kubectl rollout restart deploy/<name> -n <ns>`<br>`# Delete bad pods (if needed)`<br>`kubectl delete pod <pod> -n <ns>` | ✓ Correct configuration applied<br>✓ Workload restarted or recreated<br>✓ Dependencies restored<br>✓ No new errors after fix |
| 6 | **VERIFY THE FIX**<br>Confirm the issue is completely resolved. | ✓ Resource status<br>✓ Application health<br>✓ Logs and events<br>✓ User access<br>✓ Metrics (if available) | `# Verify resource status`<br>`kubectl get all -n <ns>`<br>`# Check events`<br>`kubectl get events -n <ns> --sort-by=.lastTimestamp`<br>`# Test application`<br>`curl -I http://<service>.<ns>.svc.cluster.local` | ✓ All resources in desired state<br>✓ No errors in logs/events<br>✓ Application accessible<br>✓ Users can confirm it works |
| 7 | **PREVENT RECURRENCE**<br>Make it better so it doesn't happen again. | ✓ Add monitoring<br>✓ Alerts & dashboards<br>✓ Runbooks / Docs<br>✓ Resource limits<br>✓ Resilience design | `# Add alerts / monitors`<br>`kubectl apply -f monitoring/`<br>`# Update documentation`<br>`Runbook, postmortem, diagrams`<br>`# Improve resilience`<br>`HPA, limits, probes, backups, redundancy` | ✓ Monitoring for early detection<br>✓ Documentation updated<br>✓ Alerts for future incidents<br>✓ Better design to avoid repeats |

## TROUBLESHOOTING TOOLKIT

### PODS & CONTAINERS
```
kubectl get pods -A
kubectl describe pod <pod>
kubectl logs <pod> -n <ns>
kubectl exec -it <pod> -n <ns> -- sh
```

### EVENTS
```
kubectl get events -A
kubectl get events -n <ns>
--sort-by=.lastTimestamp
```

### NODES
```
kubectl get nodes
kubectl describe node <node>
kubectl top nodes
```

### NETWORKING
```
kubectl get svc -A
kubectl get ep -A
kubectl get ing -A
kubectl get netpol -A
```

### STORAGE
```
kubectl get pvc -A
kubectl describe pvc <pvc>
kubectl get sc
```

### CONFIGURATION
```
kubectl get cm -A
kubectl get secret -A
kubectl get deploy -A -o yaml
```

### PERMISSIONS (RBAC)
```
kubectl auth can-i --list
kubectl get roles -A
kubectl get rolebindings -A
```

### CLUSTER HEALTH
```
kubectl cluster-info
kubectl get componentstatuses
kubectl get cs
```

**THINK SYSTEMS**
Everything is connected. Follow the flow from request to container.

**DOCUMENT EVERYTHING**
Capture symptoms, findings, cause, fix and lessons learned.

**STAY CALM**
Calm mind, clear steps, accurate fixes. No guessing.

**LEARN & IMPROVE**
Every incident is an opportunity to become better.

# MOCK EXAM 5 OVERVIEW

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

REAL-WORLD SCENARIOS. PRODUCTION INCIDENTS. EXAM-LEVEL COMPLEXITY.

Mock Exam 5 is the final and most advanced practice exam. It covers multi-dimensional troubleshooting, production recovery, and end-to-end Kubernetes administration under real-world pressure.

**EXAM PURPOSE**
Evaluate your ability to troubleshoot, recover and operate a production Kubernetes environment end-to-end.

**TOTAL TASKS**
85 TASKS
(Task 1 to Task 85)

**TIME DURATION**
2 HOURS
Real exam time simulation

**DIFFICULTY LEVEL**
EXTREME
Most difficult mock. Closest to real CKA

**GOAL**
Complete all tasks, recover all systems and restore full cluster health.

## CKA DOMAINS COVERED

| DOMAIN | WEIGHT | FOCUS AREAS |
|---|---|---|
| Cluster Architecture, Installation & Configuration | 25% | • Cluster components, HA, etcd<br>• Kubeadm, kubelet, system services<br>• Networking, DNS, core add-ons |
| Workloads & Scheduling | 15% | • Pods, Deployments, StatefulSets<br>• Scheduling, taints, tolerations<br>• Resource limits, QoS classes |
| Services & Networking | 20% | • Services, Endpoints, kube-proxy<br>• Ingress, DNS, CoreDNS<br>• NetworkPolicies, CNI |
| Storage | 15% | • PV, PVC, StorageClass<br>• Dynamic provisioning<br>• Volume management |
| Security & Access Control | 10% | • RBAC, ServiceAccounts<br>• Secrets, ConfigMaps<br>• Security contexts |
| Troubleshooting (Production) | 15% | • Logs, Events, Metrics<br>• Root cause analysis<br>• Full recovery workflows |

This exam covers all domains with a strong emphasis on Troubleshooting and Production Incidents.

## MOCK EXAM 5 TASKS SUMMARY

| | | |
|---|---|---|
| Task 1 – 20 | : | Production Workload Failures |
| Task 21 – 40 | : | Networking & Ingress Failures |
| Task 41 – 60 | : | Storage & Data Recovery |
| Task 61 – 75 | : | Control Plane & Cluster Failures |
| Task 76 – 85 | : | Multi-Failure Scenarios & Complete Incident Response |

## KEY FOCUS AREAS IN MOCK EXAM 5

- Multi-component failure analysis
- Advanced troubleshooting under time pressure
- End-to-end application recovery
- Networking, storage, and control plane restoration
- RBAC, security, and access recovery
- Production readiness validation
- Real-world incident response mindset

## EXAM ENVIRONMENT

| | | |
|---|---|---|
| Kubernetes Version | : | v1.30+ |
| Cluster Nodes | : | 3+ (Control Plane + Worker Nodes) |
| Platform | : | Ubuntu / RHEL |
| Access | : | kubectl (CLI) |
| Permissions | : | Cluster-admin |
| Internet Access | : | No |
| External Tools | : | Not allowed |
| Resources | : | Pre-installed by exam environment |

## IMPORTANT EXAM RULES

- Read each task carefully and understand the requirements.
- Validate your changes before moving to the next task.
- Always check logs, events, and status before applying fixes.
- Do not assume. Investigate, confirm, and then fix.
- Restore the system to a healthy and working state.
- Ensure final validation passes before moving on.
- Time management is key. Do not get stuck on one task.

**SUCCESS TIP**
Think like a production engineer. Observe, isolate, fix, and validate. Repeat until the system is 100% healthy.

## MOCK EXAM 5 WORKFLOW

1. **Read Task** — Understand the requirements
2. **Investigate** — Check logs, events, and status
3. **Identify Root Cause** — Find the real issue
4. **Implement Fix** — Apply the correct solution
5. **Validate** — Confirm the fix works
6. **Restore & Verify** — Ensure full system health
7. **Complete** — Move to next task and repeat

## CKA OFFICIAL RESOURCES

**Kubernetes Docs**
https://kubernetes.io/docs/

**kubectl Cheat Sheet**
https://kubernetes.io/docs/reference/kubectl/cheatsheet/

**CKA Exam Curriculum**
https://github.com/cncf/curriculum-for-cka

Practice. Analyze. Improve. Repeat.
That's the path to CKA success!

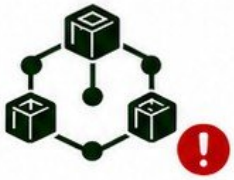
# MOCK EXAM 5 (TASKS 81 TO 85)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

MIXED ENTERPRISE SCENARIOS. MULTI-STEP QUESTIONS. TIME PRESSURE.

These tasks are real-world, multi-domain challenges that test your ability to troubleshoot, and validate a production Kubernetes environment under pressure.

## 81 TASK 81: ENTERPRISE MICROSERVICES RECOVERY



**OBJECTIVES**

- Deploy a multi-tier application
- Diagnose cross-service failures
- Restore inter-service communication
- Validate application health
- Verify production readiness

**KEY ACTIONS**

```
# Deploy application stack
kubectl apply -f app/namespace.yaml
kubectl apply -f app/
# Check pods and services
kubectl get pods -n app
kubectl get svc -n app
kubectl get endpoints -n app
# Diagnose failures
kubectl describe pod <pod> -n app
kubectl logs <pod> -n app --tail=200
# Restore communication
kubectl rollout restart deployment <deploy> -n app
kubectl patch svc <svc> -n app -p '{"spec":{...}}'
# Validate health
curl -k https://app.example.com/health
kubectl get all -n app
```

**SUCCESS CRITERIA**

- All application pods are Running
- All Services have healthy Endpoints
- All microservices communicate
- Application health endpoint returns OK
- Application is fully functional

STATUS CHECK: Unhealthy → Healthy

## 82 TASK 82: PRODUCTION CLUSTER NETWORKING

**OBJECTIVES**

- Diagnose DNS failures
- Repair Ingress routing
- Restore Service connectivity
- Validate NetworkPolicies
- Confirm end-to-end traffic flow

**KEY ACTIONS**

```
# Check DNS
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system logs deployment/coredns --tail=100
# Check Ingress
kubectl get ingress -A
kubectl describe ingress <ingress> -n <ns>
# Check Services
kubectl get svc -A
kubectl get endpoints -A
# Validate NetworkPolicies
kubectl get networkpolicies -A
kubectl describe networkpolicy <np> -n <ns>
# End-to-end test
curl -I https://app.example.com
kubectl exec <pod> -n <ns> -- nslookup <svc>
```

**SUCCESS CRITERIA**

- CoreDNS pods are Running
- DNS resolution works
- Ingress routes traffic correctly
- Services have healthy Endpoints
- NetworkPolicies allow required traffic
- End-to-end traffic flow is successful

STATUS CHECK: Traffic Broken → Traffic Healthly

## 83 TASK 83: STORAGE AND STATEFUL APPLICATIONS

- Recover StatefulSet workloads
- Repair PersistentVolumeClaims
- Restore application data
- Validate StorageClasses
- Verify persistent storage availability

```
# Check StatefulSets
kubectl get sts -A
kubectl describe sts <sts> -n <ns>
# Check PVCs
kubectl get pvc -A
kubectl describe pvc <pvc> -n <ns>
# Check StorageClasses
kubectl get sc
kubectl describe sc <sc>
# Verify volumes
kubectl get pv
kubectl describe pv <pv>
# Restore and validate
kubectl rollout restart sts/<sts> -n <ns>
kubectl exec -it <pod> -n <ns> -- ls /data
kubectl logs <pod> -n <ns> --tail=200
```

**SUCCESS CRITERIA**

- StatefulSet pods are Running
- All PVCs are Bound
- Application data is intact
- StorageClass is functioning
- Persistent volume is accessible

STATUS CHECK: Storage Issue → Storage OK

## 84 TASK 84: SECURITY INCIDENT RESPONSE

**OBJECTIVES**

- Diagnose RBAC failures
- Restore ServiceAccount permissions
- Validate Secrets configuration
- Verify Security Contexts
- Confirm least-privilege access

**KEY ACTIONS**

```
# Check RBAC
kubectl auth can-i --list --as=system:serviceaccount:<ns>:<sa>
kubectl get role,rolebinding,clusterrole,clusterrolebinding -A
# Check ServiceAccounts
kubectl get sa -A
kubectl describe sa <sa> -n <ns>
# Check Secrets
kubectl get secrets -A
kubectl describe secret <secret> -n <ns>
# Check Security Contexts
kubectl get pods -A -o jsonpath='{..securityContext}' | jq
# Validate access
kubectl auth can-i get pods --as=system:serviceaccount:<ns>:<sa> -n <ns>
```

**SUCCESS CRITERIA**

- RBAC permissions are correct
- ServiceAccounts have required access
- Secrets are present and valid
- Security Contexts are enforced
- Least-privilege access is confirmed

STATUS CHECK: Access Broken → Access Granted

## 85 TASK 85: ENTERPRISE VALIDATION

**OBJECTIVES**

- Validate all workloads
- Verify networking
- Verify storage
- Confirm security posture
- Complete final production assessment

**KEY ACTIONS**

```
# Validate workloads
kubectl get all -A
# Validate networking
kubectl get svc,ingress,endpoints,networkpolicies -A
# Validate storage
kubectl get pvc,pv,sc,sts -A
# Validate security
kubectl auth can-i --list
kubectl get sa,role,rolebinding,clusterrole,clusterrolebinding -A
# Final health check
kubectl get --raw='/readyz?verbose'
kubectl get componentstatuses
```

**SUCCESS CRITERIA**

- All workloads are healthy
- Networking is fully functional
- Storage is accessible and healthy
- Security posture is correct
- Cluster passes final assessment

STATUS CHECK: Not Ready → Production Ready

**WORKLOADS** — Deploy, Manage, Recover

**NETWORKING** — DNS, Services, Ingress, Policies

**STORAGE** — PVC, PV, StatefulSets

**SECURITY** — RBAC, Secrets, Security Context

**OPERATIONS** — Monitoring, Health, Validation



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# MOCK EXAM 5 (TASKS 86 TO 90)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

MIXED ENTERPRISE SCENARIOS. MULTI-STEP QUESTIONS. TIME PRESSURE.

These tasks are real-world, multi-domain challenges that test your ability to troubleshoot, recover, and validate a production Kubernetes environment under pressure.

## 86 TASK 86: CONTROL PLANE FAILURE RECOVERY

**OBJECTIVES**

- Restore API Server
- Verify Scheduler
- Recover Controller Manager
- Validate etcd health
- Confirm cluster functionality

**KEY ACTIONS (EXAMPLES)**

```
# Check control plane components
kubectl get --raw='/readyz?verbose'
kubectl get componentstatuses
# Restore API Server
systemctl restart kube-apiserver
# Verify Scheduler
systemctl status kube-scheduler
# Recover Controller Manager
systemctl restart kube-controller-manager
# Check etcd health
ETCDCTL_API=3 etcdctl endpoint health
# Confirm cluster functionality
kubectl get nodes
kubectl get all -A
```

**SUCCESS CRITERIA**

- API Server is Running
- Scheduler is healthy
- Controller Manager is Running
- etcd endpoints are healthy
- Cluster is fully functional

STATUS CHECK: Control Plane Unhealthy → Control Plane Healthy

## 87 TASK 87: NODE FAILURE INVESTIGATION

**OBJECTIVES**

- Recover failed worker nodes
- Repair kubelet issues
- Restore container runtime
- Validate node readiness
- Verify workload recovery

**KEY ACTIONS (EXAMPLES)**

```
# Check node status
kubectl get nodes
# Check kubelet logs
journalctl -u kubelet -xe
# Restart kubelet
systemctl restart kubelet
# Check container runtime
systemctl restart containerd
# Validate node readiness
kubectl get nodes
# Verify workload recovery
kubectl get pods -A -o wide
```

**SUCCESS CRITERIA**

- Node is Ready
- kubelet is Running
- Container runtime is healthy
- All workloads are Running
- No pods are in CrashLoopBackOff

STATUS CHECK: Node Not Ready → Node Ready

## 88 TASK 88: CLUSTER UPGRADE RECOVERY

- Diagnose failed upgrade
- Resolve version conflicts
- Recover cluster services
- Validate component compatibility
- Confirm successful upgrade

```
# Check upgrade status
kubectl get nodes -o wide
kubectl get pods -n kube-system
# Resolve version conflicts
kubeadm upgrade plan
kubeadm upgrade apply v1.xx.x
# Restart components
systemctl restart kubelet
# Validate components
kubectl get componentstatuses
# Confirm successful upgrade
kubectl version --short
```

- All control plane components healthy
- All worker nodes upgraded
- Component versions are compatible
- Services in kube-system are Running
- Upgrade completed successfully

STATUS CHECK: Upgrade Failed → Upgrade Successful

## 89 TASK 89: CERTIFICATE RECOVERY

- Identify expired certificates
- Renew certificates
- Restart affected components
- Validate secure communication
- Verify cluster health

```
# Check certificate expiration
kubeadm certs check-expiration
# Renew certificates
kubeadm certs renew all
# Restart control plane components
systemctl restart kube-apiserver kube-controller-manager \
kube-scheduler etcd
# Validate secure communication
kubectl get --raw='/readyz?verbose'
# Verify cluster health
kubectl get nodes
```

- All certificates are valid
- All components are Running
- TLS communication is secure
- No x509 errors in logs
- Cluster is healthy

STATUS CHECK: Certificates Invalid → Certificates Valid

## 90 TASK 90: DISASTER RECOVERY

- Restore etcd from backup
- Recover workloads
- Validate namespaces
- Verify cluster integrity
- Confirm production recovery

```
# Restore etcd from backup
etcdctl snapshot restore /backup/etcd.db \
--data-dir=/var/lib/etcd-new
# Update etcd static pod manifest
# Restart etcd
systemctl restart etcd
# Validate cluster
kubectl get nodes
kubectl get all -A
# Confirm production recovery
kubectl get pods -A
```

- etcd is restored and healthy
- All workloads are Running
- All namespaces are intact
- No data loss or corruption
- Cluster is fully recovered

STATUS CHECK: Recovery Failed → Recovery Successful

## DOMAIN COVERAGE IN MOCK EXAM 5

- **CLUSTER ARCHITECTURE** — Control Plane, Nodes, HA, Configuration
- **WORKLOADS & SCHEDULING** — Deployments, Pods, Scheduling, Affinity, Taints
- **SERVICES & NETWORKING** — Services, Ingress, DNS, NetworkPolicies, CNI
- **STORAGE** — PVC, PV, StorageClasses, StatefulSets
- **SECURITY** — RBAC, ServiceAccounts, Secrets, Security Contexts
- **OPERATIONS & RECOVERY** — Monitoring, Backups, Upgrades, DR, Troubleshooting

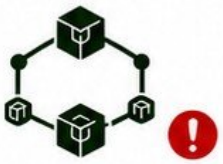
# MOCK EXAM 5 (TASKS 91 TO 95)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

MIXED ENTERPRISE SCENARIOS. MULTI-STEP QUESTIONS. TIME PRESSURE.

These tasks are real-world, multi-domain challenges that test your ability to troubleshoot, and validate a production Kubernetes environment under pressure.

## 91 TASK 91: MULTI-STEP SCHEDULING CHALLENGE



### OBJECTIVES

- Configure taints and tolerations
- Configure node affinity
- Configure Pod anti-affinity
- Validate workload placement
- Optimize availability

### KEY ACTIONS (EXAMPLES)

```
# Add taint
kubectl taint nodes prod-node1
workload=critical:NoSchedule
# Create toleration in pod spec
# Add node affinity
kubectl apply -f pod-affinity.yaml
# Add pod anti- affinity
kubectl apply -f pod-anti-affinity.yaml
# Verify placement
kubectl get pods -o wide --show-labels
# Check topology spread
kubectl get pods -n app -o wide
```

### SUCCESS CRITERIA

- Pods scheduled on correct nodes
- Taints and tolerations work
- Node affinity is respected
- Pod anti-affinity is enforced
- Workloads are highly available

STATUS CHECK

Scheduling Issue → Optimized

## 92 TASK 92: ENTERPRISE RESOURCE GOVERNANCE

### OBJECTIVES

- Configure ResourceQuota
- Configure LimitRange
- Apply PriorityClass
- Validate namespace policies
- Prevent resource exhaustion

### KEY ACTIONS (EXAMPLES)

```
# Create ResourceQuota
kubectl apply -f resourcequota.yaml
# Create LimitRange
kubectl apply -f limitrange.yaml
# Create PriorityClass
kubectl apply -f priorityclass.yaml
# Verify policies
kubectl describe resourcequota
kubectl describe limitrange
# Check priority classes
kubectl get priorityclass
```

### SUCCESS CRITERIA

- ResourceQuota is enforced
- LimitRange is applied
- PriorityClass is working
- Namespace policies are valid
- Resources are protected

STATUS CHECK

Policy Violation → Governed

## 93 TASK 93: ADVANCED WORKLOAD RECOVERY

### OBJECTIVES

- Repair failed Deployments
- Recover StatefulSets
- Restore DaemonSets
- Validate rolling updates
- Confirm application availability

### KEY ACTIONS (EXAMPLES)

```
# Check failed deployments
kubectl get deploy
kubectl rollout status deploy/<name>
# Rollback if needed
kubectl rollout undo deploy/<name>
# Recover StatefulSet
kubectl rollout restart sts/<name>
# Restart DaemonSet
kubectl rollout restart ds/<name>
# Verify availability
kubectl get all -A
```

### SUCCESS CRITERIA

- All Deployments are healthy
- StatefulSets are running
- DameonSets are running
- Rolling updates are successful
- Applications are available

STATUS CHECK

Workload Down → Workload Healthy

## 94 TASK 94: COMPLEX NETWORKING RECOVERY

### OBJECTIVES

- Repair Services
- Recover Ingress
- Restore CoreDNS
- Validate Endpoints
- Confirm external access

### KEY ACTIONS (EXAMPLES)

```
# Check Services
kubectl get svc -A
# Repair Service
kubectl apply -f service-fixed.yaml
# Check Ingress
kubectl get ingress -A
# Repair Ingress
kubectl apply -f ingress-fixed.yaml
# Restart CoreDNS
kubectl rollout restart deploy/coredns -n kube-system
# Validate Endpoints
kubectl get endpoints -A
```

### SUCCESS CRITERIA

- All Services are healthy
- Ingress routes correctly
- CoreDNS is functioning
- Endpoints are correct
- External access is working

STATUS CHECK

Network Broken → Network Healthy

## 95 TASK 95: PRODUCTION PERFORMANCE VALIDATION

### OBJECTIVES

- Review cluster resources
- Validate scaling
- Check application performance
- Verify monitoring
- Confirm production stability

### KEY ACTIONS (EXAMPLES)

```
# Check resource usage
kubectl top nodes
kubectl top pods --all-namespaces
# Check HPA status
kubectl get hpa -A
# Check pod metrics (if metrics-server)
kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods
# Check monitoring
kubectl -n monitoring get pods
# Validate cluster health
kubectl get nodes
kubectl get pods -A
```

### SUCCESS CRITERIA

- Cluster resources are healthy
- Scaling works as expected
- Performance metrics are good
- Monitoring is operational
- Cluster is production stable

STATUS CHECK

Performance Issue → Production Stable

## DOMAIN COVERAGE IN MOCK EXAM 5

- **CLUSTER ARCHITECTURE** — Nodes, Control Plane, Components, Scheduling
- **WORKLOADS & SCHEDULING** — Deployments, STS, DaemonSets, HPA, Affinity, Taints
- **SERVICES & NETWORKING** — Services, Ingress, DNS, Endpoints, NetworkPolicies
- **STORAGE** — PVC, PV, StorageClasses, Volume Health
- **SECURITY** — RBAC, ServiceAccounts, Secrets, Policies, Security Contexts
- **OPERATIONS & RECOVERY** — Upgrades, DR, Monitoring, Troubleshooting, Performance

# MOCK EXAM 5 (TASKS 96 TO 100)

CKA CERTIFIED KUBERNETES ADMINISTRATOR

MIXED ENTERPRISE SCENARIOS. MULTI-STEP QUESTIONS. TIME PRESSURE.

These tasks are real-world, multi-domain challenges that test your ability to troubleshoot, recover, and validate a production Kubernetes environment under pressure.

## 96 TASK 96: ENTERPRISE INCIDENT INVESTIGATION

**OBJECTIVES**

- Correlate logs
- Analyze Events
- Identify root cause
- Prioritize remediation
- Document findings

**KEY ACTIONS (EXAMPLES)**

```
# Check and correlate logs
kubectl logs -n <ns> <pod> -c <container> --tail=200
kubectl logs -l app=<app> --all-containers --tail=100
# Analyze events
kubectl get events -A --sort-by=.lastTimestamp
kubectl describe <resource> -n <ns>
# Identify root cause
kubectl get pods -A
kubectl describe pod <pod> -n <ns>
# Prioritize remediation
kubectl top nodes
kubectl get pods -A --field-selector=status.phase!=Running
# Document findings
kubectl get all -A -o wide > incident-report.txt
```

**SUCCESS CRITERIA**

- Logs are correlated and clear
- Events are analyzed accurately
- Root cause is identified
- Remediation is prioritized
- Findings are documented

STATUS CHECK: Investigation Incomplete → Root Cause Identified

## 97 TASK 97: CROSS-DOMAIN RECOVERY

**OBJECTIVES**

- Resolve workload failures
- Restore networking
- Recover storage
- Validate security
- Confirm application health

**KEY ACTIONS (EXAMPLES)**

```
# Resolve workload failures
kubectl get deploy,sts,ds -A
kubectl rollout restart deploy/<name> -n <ns>
# Restore networking
kubectl get svc,ing,ep,netpol -A
kubectl rollout restart -n kube-system deploy/coredns
# Recover storage
kubectl get pvc,pv,sc -A
kubectl describe pvc <pvc> -n <ns>
# Validate security
kubectl get sa,role,rolebinding,clusterrole,clusterrolebinding -A
kubectl auth can-i --list --as=system:serviceaccount:<ns>:<sa>
# Confirm application health
kubectl get pods -A
kubectl get ingress -A
```

**SUCCESS CRITERIA**

- Workloads are running
- Networking is functional
- Storage is accessible
- Security is validated
- Applications are healthy

STATUS CHECK: Recovery Incomplete → All Domains Healthy

## 98 TASK 98: PRODUCTION READINESS ASSESSMENT

**OBJECTIVES**

- Verify all Kubernetes resources
- Validate cluster configuration
- Confirm policy compliance
- Check operational readiness
- Resolve remaining issues

**KEY ACTIONS (EXAMPLES)**

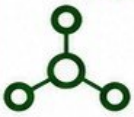
```
# Verify resources
kubectl get all -A
kubectl get crd -A
# Validate configuration
kubectl get cm,secret -A
kubectl describe nodes
# Check policies
kubectl get networkpolicies -A
kubectl get resourcequota,limitrange -A
# Operational readiness
kubectl top nodes
kubectl get pods -A --field-selector=status.phase!=Running
# Resolve issues
kubectl describe <resource> -n <ns>
```

**SUCCESS CRITERIA**

- All resources are validated
- Configuration is correct
- Policies are compliant
- Operational readiness is confirmed
- No critical issues remain

STATUS CHECK: Not Ready → Ready for Production

## 99 TASK 99: END-TO-END CLUSTER VALIDATION



**OBJECTIVES**

- Verify control plane
- Verify worker nodes
- Validate networking
- Validate storage
- Confirm complete cluster health

**KEY ACTIONS (EXAMPLES)**

```
# Control plane
kubectl get componentstatuses
kubectl get --raw /readyz?verbose
# Worker nodes
kubectl get nodes
kubectl describe nodes
# Networking
kubectl get svc,ing,ep -A
kubectl get pods -n kube-system -l k8s-app=kube-dns
# Storage
kubectl get pv,pvc,sc -A
# Final health check
kubectl get pods -A
kubectl get all -A
```

**SUCCESS CRITERIA**

- Control plane is healthy
- All nodes are Ready
- Networking is fully functional
- Storage is accessible
- Cluster is 100% healthy

STATUS CHECK: Issues Found → Cluster Healthy

## 100 TASK 100: FINAL ENTERPRISE CHALLENGE

**OBJECTIVES**

- Troubleshoot multiple production failures
- Recover complete Kubernetes environment
- Validate every exam objective
- Confirm production readiness
- Perform final submission review

**KEY ACTIONS (EXAMPLES)**

```
# Diagnose all issues
kubectl get events -A --sort-by=.lastTimestamp
kubectl get pods -A --field-selector=status.phase!=Running
# Recover environment
(Apply all previous recovery actions as required)
# Validate all objectives
kubectl get all -A
kubectl auth can-i --list --as=system:serviceaccount:<ns>:<sa>
# Final checks
kubectl top nodes
kubectl get pods -A
# Final submission checklist
Review every task. Ensure all criteria are met.
```

**SUCCESS CRITERIA**

- All failures are resolved
- Environment is fully recovered
- All exam objectives validated
- Production readiness confirmed
- Final submission is complete

STATUS CHECK: Submission Incomplete → Exam Completed

## EXAM DOMAINS COVERED

- **CLUSTER ARCHITECTURE** — Control Plane, Nodes, Components, HA
- **WORKLOADS & SCHEDULING** — Deployments, STS, DaemonSets, Affinity, Taints
- **SERVICES & NETWORKING** — Services, Ingress, DNS, NetworkPolicies, Endpoints
- **STORAGE** — PVC, PV, StorageClasses, StatefulSets
- **SECURITY** — RBAC, ServiceAccounts, Secrets, Security Contexts, Policies
- **OPERATIONS** — Monitoring, Logging, Backups, Upgrades, Troubleshooting



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# MOCK EXAM 5 SOLUTIONS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

TASKS 81 TO 100 – HIGH LEVEL SOLUTIONS & KEY COMMANDS.

These are reference solutions and key command patterns. Actual exam steps, names, and details may vary.

## 81 ENTERPRISE MICROSERVICES RECOVERY

- Re-deploy missing components
- Check logs and describe resources
- Fix service discovery and DNS
- Ensure endpoints are healthy
- Validate application end-to-end

**KEY COMMANDS**

```
kubectl get all -n app
kubectl describe pod -n app <pod>
kubectl logs -n app <pod> -c <ctr>
kubectl get svc,ep -n app
kubectl rollout restart deploy -n app
kubectl get all -n app
curl -I https://app.example.com/health
```

## 82 PRODUCTION CLUSTER NETWORKING

- Fix CoreDNS issues
- Repair Ingress configuration
- Restore Service connectivity
- Adjust NetworkPolicies
- Verify end-to-end traffic

**KEY COMMANDS**

```
kubectl -n kube-system get pods
kubectl -n kube-system logs deploy/coredns
kubectl get ingress -A
kubectl describe ingress <ing> -n <ns>
kubectl get svc,endpoints -A
kubectl get networkpolicies -A
curl -I https://app.example.com
```

## 83 STORAGE AND STATEFUL APPLICATIONS

- Recover StatefulSets
- Fix PersistentVolumeClaims
- Restore application data
- Validate StorageClasses
- Verify storage availability

**KEY COMMANDS**

```
kubectl get sts -A
kubectl describe sts <sts> -n <ns>
kubectl get pvc -A
kubectl describe pvc <pvc> -n <ns>
kubectl get sc
kubectl get pv
kubectl get pods -o wide
```

## 84 SECURITY INCIDENT RESPONSE

- Fix RBAC permissions
- Restore ServiceAccounts
- Validate Secrets
- Verify Security Contexts
- Ensure least-privilege access

**KEY COMMANDS**

```
kubectl auth can-i --as=system:serviceaccount:<ns>:<sa> --list
kubectl get role,rolebinding,clusterrole,clusterrolebinding -A
kubectl describe sa <sa> -n <ns>
kubectl get secrets -A
kubectl describe secret <secret> -n <ns>
kubectl get pods -o yaml | grep -A5 securityContext
```

## 85 ENTERPRISE VALIDATION

- Validate all workloads
- Verify networking
- Verify storage
- Confirm security posture
- Complete final assessment

**KEY COMMANDS**

```
kubectl get all -A
kubectl get nodes
kubectl get pv,pvc,sc
kubectl get networkpolicies -A
kubectl auth can-i --list
kubectl get componentstatuses
```

## 86 CONTROL PLANE FAILURE RECOVERY

- Restore API Server
- Verify Scheduler
- Recover Controller Manager
- Validate etcd health
- Confirm cluster functionality

**KEY COMMANDS**

```
kubectl get componentstatuses
kubectl get --raw '/readyz?verbose'
systemctl status kube-apiserver
systemctl status kube-scheduler
systemctl status kube-controller-manager
etcdctl endpoint health
kubectl get nodes
```

## 87 NODE FAILURE INVESTIGATION

- Recover failed nodes
- Fix kubelet issues
- Restore container runtime
- Validate node readiness
- Verify workload recovery

**KEY COMMANDS**

```
kubectl get nodes
journalctl -u kubelet
systemctl restart kubelet
systemctl status containerd
crictl ps -a
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets
kubectl uncordon <node>
```

## 88 CLUSTER UPGRADE RECOVERY

- Diagnose failed upgrade
- Resolve version conflicts
- Recover cluster services
- Validate compatibility
- Confirm successful upgrade

**KEY COMMANDS**

```
kubectl get nodes -o wide
kubectl get pods -n kube-system
kubectl describe node <node>
kubeadm upgrade plan
kubeadm upgrade apply v1.xx.x
systemctl restart kubelet
kubectl get componentstatuses
```

## 89 CERTIFICATE RECOVERY

- Find expired certificates
- Renew certificates
- Restart components
- Validate secure communication
- Verify cluster health

**KEY COMMANDS**

```
kubectl get csr
openssl x509 -in <cert>.crt -noout -dates
kubeadm certs renew all
systemctl restart kube-apiserver
systemctl restart kube-controller-manager
kubectl get componentstatuses
```

## 90 DISASTER RECOVERY

- Restore etcd from backup
- Recover workloads
- Validate namespaces
- Verify cluster integrity
- Confirm production recovery

**KEY COMMANDS**

```
etcdctl snapshot restore /backup/etcd.db \
--data-dir=/var/lib/etcd-new
systemctl restart etcd
kubeadm init phase etcd local
kubectl get all -A
kubectl get nodes
```

### QUICK VERIFICATION

- kubectl get all -A
- kubectl get nodes -o wide
- kubectl get events -A --sort-by=.lastTimestamp
- kubectl get pods -A
- kubectl get componentstatuses

### COMMON LOGS

- kubectl logs <pod> -n <ns> -c <ctr>
- kubectl logs -n kube-system deploy/coredns
- journalctl -u kubelet -f
- journalctl -u containerd -f
- kubectl describe <resource>

### RESOURCE CHECKS

- kubectl top nodes
- kubectl top pods -A
- kubectl get pvc -A
- kubectl get storageclass
- kubectl get quota,limitrange -A

### HEALTH CHECKS

- kubectl get componentstatuses
- kubectl get --raw '/readyz?verbose'
- kubectl get --raw '/healthz'
- etcdctl endpoint health
- kubectl get nodes

**EXAM DOMAINS COVERED**

- **CLUSTER ARCHITECTURE** — Control Plane, Nodes, HA
- **WORKLOADS & SCHEDULING** — Deployments, STS, DaemonSets
- **SERVICES & NETWORKING** — Services, Ingress, DNS, NetworkPolicies
- **STORAGE** — PVC, PV, StorageClasses, StatefulSets
- **SECURITY** — RBAC, Secrets, Security Contexts, Policies
- **OPERATIONS** — Monitoring, Logging, Backup, Recovery



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# MOCK EXAM 5 PERFORMANCE ASSESSMENT

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

TASKS 81 TO 100 – PERFORMANCE, SCORING & EXPECTATIONS.

Mock Exam 5 is designed to simulate real CKA exam pressure. Scoring is based on accuracy, completeness, efficiency and time. Practice with focus, automate your workflow, and build confidence.

## 1 ASSESSMENT OVERVIEW

- **Tasks:** 81 to 100 (20 tasks)
- **Total Points:** 200
- **Passing Score:** 158 (79%)
- **Time Limit:** 4 Hours
- **Average Time per Task:** 12 Minutes
- **Task Types:** Troubleshooting, Recovery, Configuration, Validation, Multi-step

## 2 PERFORMANCE SUMMARY

- **Focus:** Production readiness
- **Scope:** All CKA domains
- **Difficulty:** High
- **Scenario Complexity:** Multi-domain
- **Typical Engineer Level:** Senior / Staff
- **Success Factor:** Accuracy + Speed + Complete Validation

## 3 SCORING BREAKDOWN (200 POINTS)

| DOMAIN | POINTS | TASKS (EXAMPLES) | WEIGHT |
|---|---|---|---|
| CLUSTER ARCHITECTURE, INSTALLATION & CONFIGURATION | 40 | 81, 86, 91 | 20% |
| WORKLOADS & SCHEDULING | 40 | 83, 87, 93, 97 | 20% |
| SERVICES & NETWORKING | 40 | 82, 94, 97, 99 | 20% |
| STORAGE | 30 | 83, 90, 97, 99 | 15% |
| SECURITY & RBAC | 30 | 84, 89, 97, 98 | 15% |
| OPERATIONS & TROUBLESHOOTING | 20 | 85, 95, 96, 100 | 10% |
| TOTAL | 200 | 20 TASKS | 100% |

## PERFORMANCE BENCHMARKS

| SCORE RANGE | LEVEL | INTERPRETATION |
|---|---|---|
| 180 – 200 | EXPERT | Outstanding performance. Production ready at any scale. |
| 158 – 179 | PASS | Meets exam standard. Strong production skills. |
| 120 – 157 | DEVELOPING | Below passing. Needs focused improvement. |
| 0 – 119 | NOT READY | Significant improvement required. |

## 5 TIME MANAGEMENT GUIDE

| PHASE | TASKS | RECOMMENDED TIME |
|---|---|---|
| Phase 1 | 81 – 85 | 60 Minutes |
| Phase 2 | 86 – 90 | 60 Minutes |
| Phase 3 | 91 – 95 | 60 Minutes |
| Phase 4 | 96 – 100 | 60 Minutes |
| TOTAL | 81 – 100 | 240 Minutes |

If you spend more than 15 minutes on a task, move on and return if time allows.

## 6 EVALUATION CRITERIA

- Correct resource configuration
- Full problem resolution
- All validations completed
- No remaining errors or warnings
- Security and best practices applied
- Efficient and accurate command usage
- Clean, working system at the end
- Documentation where required

### WHAT EXAMINERS LOOK FOR

A working solution that is complete, secure, and production ready.

## 7 DOMAIN EXPECTATIONS

### CLUSTER ARCHITECTURE (20%)

- Control plane stability
- Component health
- Cluster configuration
- High availability
- Upgrade readiness

### WORKLOADS & SCHEDULING (20%)

- Workload recovery
- Scheduling accuracy
- Resource governance
- High availability
- Scaling and updates

### SERVICES & NETWORKING (20%)

- Service connectivity
- DNS and Ingress
- Network policies
- Traffic flow
- External access

### STORAGE (15%)

- Persistent storage
- Stateful applications
- Backup & recovery
- Storage integrity
- Data availability

### SECURITY & RBAC (15%)

- RBAC configuration
- Least privilege access
- Secrets management
- Security contexts
- Policy enforcement

### OPERATIONS & TROUBLESHOOTING (10%)

- Incident investigation
- Log and event analysis
- Root cause identification
- System recovery
- Production validation

## 8 FINAL CHECKLIST BEFORE SUBMISSION

- All tasks completed
- No error messages
- All workloads are running
- All Pods are Ready
- All Services are reachable
- Ingress routes are working
- Persistent storage is healthy
- RBAC and security are correct

## 9 SUCCESS MINDSET

- Plan before you act
- Validate after every change
- Stay calm under pressure
- Think like a senior engineer
- You've got this. Finish strong!

**EXAM TIP** Practice under real exam conditions. Use a timer, avoid distractions, and build speed with accuracy.

**REMEMBER** The goal is a production ready, secure, and fully functional Kubernetes environment.

**FINAL GOAL** Pass the CKA. Build confidence. Engineer the future. Lead at scale.

# MOCK EXAM 6 OVERVIEW

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

TASKS 101 TO 120 – ENTERPRISE SCENARIOS. MAXIMUM REALISM. PRODUCTION FOCUS.

Mock Exam 6 is the most advanced set. It combines deep troubleshooting, multi-domain failures, incident response, recovery, and production validation across all CKA objectives.

## 1 EXAM OVERVIEW

- Tasks: 101 to 120 (20 tasks)
- Total Points: 200
- Time Limit: 4 Hours
- Average Time per Task: 12 Minutes
- Difficulty: Very High
- Focus: Production Incidents, Recovery, Optimization, Validation
- Expected Outcome: Production Ready, Secure, Fully Functional Cluster

## 2 EXAM OBJECTIVES

- Troubleshoot complex production failures
- Recover workloads, storage, and networking
- Secure cluster and respond to incidents
- Optimize resources and performance
- Validate complete cluster functionality
- Demonstrate senior-level Kubernetes skills
- Fulpraten Reacter volidtion and assessment

## 5 EXAM DOMAINS & WEIGHTAGE

| DOMAIN | DESCRIPTION | WEIGHT | POINTS | TASK RANGE |
|---|---|---|---|---|
| CLUSTER ARCHITECTURE, INSTALLATION & CONFIGURATION | Cluster setup, components, configuration, and validation | 20% | 40 | 101 – 104 |
| WORKLOADS & SCHEDULING | Deployments, scheduling, scaling, updates, and orchestration | 20% | 40 | 105 – 108 |
| SERVICES & NETWORKING | Services, DNS, Ingress, NetworkPolicies, CNI, and traffic management | 20% | 40 | 109 – 112 |
| STORAGE | Volumes, PV/PVC, StorageClasses, stateful applications, backups | 15% | 30 | 113 – 116 |
| SECURITY & RBAC | RBAC, Security Contexts, Secrets, Policies, and compliance | 15% | 30 | 117 – 118 |
| OPERATIONS & TROUBLESHOOTING | Monitoring, logging, incidents, performance, and recovery | 10% | 20 | 119 – 120 |
| TOTAL | | 100% | 200 | 20 TASKS |

## 3 WHAT'S NEW IN MOCK EXAM 6

- Cross-domain failure scenarios
- Control plane deep recovery
- Advanced networking troubleshooting
- Storage and stateful application recovery
- Security incident response
- Performance tuning and optimization
- Disaster recovery and backup restoration
- Full production validation and assessment

NEW

## 6 TASK DISTRIBUTION SUMMARY

- Architecture (20%)
- Workloads (20%)
- Networking (20%)
- Storage (15%)
- Security (15%)
- Operations (10%)

## 7 TASK TYPES

- Troubleshooting
- Incident Response
- Recovery
- Validation
- Optimization
- Multi-Step Challenges

Each task is multi-step and requires real-world engineering judgment.

## 8 EXAM HIGHLIGHTS

**Cross-Domain Scenarios**
Multiple domains affected in a single scenario.

**Production Realism**
Real incidents, real logs, real failures.

**Recovery Focused**
Restore services, data, and cluster health.

**Performance Matters**
Optimize resources and ensure stability.

**Final Validation**
Everything must be validated end-to-end.

## 9 SUCCESS CRITERIA

- All workloads are running
- All services are reachable
- Networking is functional
- Storage is accessible
- RBAC and security are correct
- No critical errors in events
- Logs are healthy and clean
- Performance is optimized
- Cluster is fully operational
- Production standards met
- Documentation completed
- Final submission successful

## 10 MINDSET FOR MOCK EXAM 6

- Think like a SRE / Platform Engineer
- Stay calm under pressure
- Validate every change
- Automate and repeatable solutions
- Quality over speed
- Deliver production ready results

**TIAM TIP**
Read the scenario carefully.
Identify the root cause.
Plan your steps. Execute with precision.

**REMEMBER**
This is not about doing everything.
It's about doing the right things in the right order.

**FOCUS**
Recover. Validate. Optimize.
Make the cluster production ready.
That's the goal.

**DOMAINS COVERED**

- **CLUSTER ARCHITECTURE** — Control Plane, Nodes, Components
- **WORKLOADS & SCHEDULING** — Deployments, STS, DaemonSets, HPA
- **SERVICES & NETWORKING** — Services, Ingress, DNS, NetworkPolicies
- **STORAGE** — PV, PVC, StorageClasses, Backups
- **SECURITY** — RBAC, Secrets, Policies, Security Contexts
- **OPERATIONS & TROUBLESHOOTING** — Monitoring, Logging, Incidents, Recovery



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

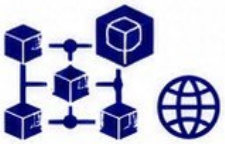
# MOCK EXAM 6 (TASKS 101 TO 105)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

FULL REAL-WORLD SIMULATION. ADVANCED ADMINISTRATION. HIGH-DIFFICULTY SCENARIOS.

Mock Exam 6 focuses on enterprise-grade Kubernetes administration.
Each task is multi-step, realistic, and aligned to CKA exam objectives.

## 101 ENTERPRISE MULTI-CLUSTER ADMINISTRATION



### TASK STEPS

1. Configure multiple kubeconfig contexts
2. Switch between production clusters
3. Validate cluster connectivity
4. Verify cluster health
5. Confirm administrative access

```
# View contexts
kubectl config get-contexts
# Use context
kubectl config use-context prod-cluster-1
# Verify connectivity
kubectl get nodes
# Check cluster info
kubectl cluster-info
# Verify admin access
kubectl auth can-i --list
```

### SUCCESS CRITERIA

- All contexts configured correctly
- Able to switch between clusters
- Cluster connectivity is valid
- All nodes are Ready
- Administrative access is confirmed

## 102 CONTROL PLANE RECOVERY

### TASK STEPS

1. Diagnose API Server failure
2. Recover Scheduler
3. Restore Controller Manager
4. Validate etcd health
5. Verify cluster stability

```
# Check API Server
kubectl get --raw=/healthz
# Check Schaduler
kubectl -n kube-system get pods -l k8s-app=kube-scheduler
# Check Controller Manager
kubectl -n kube-system get pods -l k8s-app=kube-
controller-manager
# Validate etcd
ETCDCTL_API=3 etcdctl endpoint health
# Verify cluster
kubectl get componentstatuses
```

### SUCCESS CRITERIA

- API Server is running
- Scheduler is running
- Controller Manager is running
- etcd is healthy
- Cluster is stable and functional

## 103 ENTERPRISE NODE ADMINISTRATION

### TASK STEPS

1. Drain worker nodes
2. Upgrade kubelet
3. Uncordon nodes
4. Validate workload recovery
5. Confirm node readiness

```
# Drain node
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
# Upgrade kubelet
# (perform upgrade as per environment)
# Uncordon node
kubectl uncordon <node>
# Check node status
kubectl get nodes
# Verify workloads
kubectl get pods -A -o wide
```

### SUCCESS CRITERIA

- Node drained successfully
- kubelet upgraded
- Node uncordoned
- Workloads are running
- Node status is Ready

## 104 CERTIFICATE LIFECYCLE MANAGEMENT

### TASK STEPS

1. Inspect cluster certificates
2. Renew expired certificates
3. Restart affected services
4. Validate secure communication
5. Verify cluster functionality

```
# Check certificates
kubectl get csr
# Approve CSR
kubectl certificate approve <csr-name>
# Restart components
kubectl -n kube-system rollout restart deployment/kube-apiserver
kubectl -n kube-system rollout restart deployment/kube-controller-manager
kubectl -n kube-system rollout restart deployment/kube-scheduler
# Verify
kubectl get componentstatuses
```

### SUCCESS CRITERIA

- All certificates are valid
- Expired certificates renewed
- Services restarted successfully
- Secure communication verified
- Cluster is fully functional

## 105 PRODUCTION CLUSTER VALIDATION

### TASK STEPS

1. Verify namespaces
2. Validate workloads
3. Check control plane
4. Confirm networking
5. Ensure production readiness

```
# Check namespaces
kubectl get ns
# Check workloads
kubectl get pods -A
# Check control plane
kubectl get componentstatuses
# Check networking
kubectl get svc -A
kubectl get ingress -A
# Final check
kubectl get nodes
kubectl cluster-info
```

### SUCCESS CRITERIA

- All namespaces are present
- All workloads are healthy
- Control plane components are healthy
- Networking is functional
- Cluster is production ready

## EXAM 6 KEY AREAS COVERED

- **MULTI-CLUSTER ADMINISTRATION** — Contexts, Access, Health
- **CONTROL PLANE RECOVERY** — API Server, Scheduler, Controller, etcd
- **NODE ADMINISTRATION** — Drain, Upgrade, Kubelet, Readiness
- **CERTIFICATE MANAGEMENT** — CSR, Renewal, Security
- **PRODUCTION VALIDATION** — Namespaces, Workloads, Networking, Readiness

# MOCK EXAM 6 (TASKS 106 TO 110)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

FULL REAL-WORLD SIMULATION. ADVANCED ADMINISTRATION. HIGH-DIFFICULTY SCENARIOS.

Mock Exam 6 continues with advanced scheduling, governance, scaling, recovery, and validation.
Each task is multi-step, realistic, and aligned to CKA exam objectives.

## 106 ADVANCED SCHEDULING

### TASK STEPS

1. Configure taints and tolerations
2. Configure node affinity
3. Configure Pod anti-affinity
4. Validate workload placement
5. Optimize availability

```
# Taint a node
kubectl taint nodes node1 env=prod:NoSchedule

# Toleration in pod
tolerations:
- key: "env"
  operator: "Equal"
  value: "prod"
  effect: "NoSchedule"
# Node affinity
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disktype
          operator: In
          values: ["ssd"]
# Pod anti-affinity
podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
  - labelSelector:
      matchLabels:
        app: web
    topologyKey: kubernetes.io/hostname
```

### SUCCESS CRITERIA

- Taints and tolerations applied
- Node affinity is working
- Pod anti-affinity enforced
- Pods are placed on correct nodes
- Availability is optimized

## 107 RESOURCE GOVERNANCE

### TASK STEPS

1. Configure ResourceQuota
2. Configure LimitRange
3. Apply PriorityClass
4. Validate namespace policies
5. Prevent resource exhaustion

```
# ResourceQuota
apiVersion: v1
kind: ResourceQuota
metadata:
  name: rq
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
# PriorityClass
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
preemptionPolicy: PreemptLowerPriority
```

### SUCCESS CRITERIA

- ResourceQuota applied
- LimitRange configured
- PriorityClass created and used
- Namespace policies validated
- Resources are protected from exhaustion

## 108 PRODUCTION SCALING

### TASK STEPS

1. Configure Horizontal Pod Autoscaler
2. Validate Metrics Server
3. Verify automatic scaling
4. Confirm workload stability
5. Optimize resource utilization

```
# HPA
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
minReplicas: 2
maxReplicas: 10
metrics:
 - type: Resource
   resource:
     name: cpu
     target:
       type: Utilization
       averageUtilization: 70
```

### SUCCESS CRITERIA

- HPA configured correctly
- Metrics Server is working
- Pods scale automatically
- Application remains stable
- Resources are efficiently used

## 109 ADVANCED WORKLOAD RECOVERY

### TASK STEPS

1. Recover Deployments
2. Recover StatefulSets
3. Recover DaemonSets
4. Validate rolling updates
5. Confirm application availability

```
# Recover Deployment
kubectl rollout restart deployment web
# Recover StatefulSet
kubectl rollout restart statefulset db
# Recover DaemonSet
kubectl rollout restart daemonset node-agent
# Check rollout status
kubectl rollout status deployment web
# Check pods
kubectl get pods -A | grep -v Running
```

### SUCCESS CRITERIA

- Deployments recovered
- StatefulSets are healthy
- DaemonSets are running
- Rolling updates successful
- Applications are available

## 110 ENTERPRISE SCHEDULING VALIDATION

### TASK STEPS

1. Verify workload distribution
2. Confirm high availability
3. Validate node utilization
4. Review scheduling policies
5. Confirm production stability

```
# Check pod distribution
kubectl get pods -o wide
# Check node utilization
kubectl top nodes
# Check pod utilization
kubectl top pods --all-namespaces
# Check policies
kubectl get networkpolicies --all-namespaces
# Check availability
kubectl get pods -A | grep -v Running
```

### SUCCESS CRITERIA

- Workloads are evenly distributed
- High availability is confirmed
- Node utilization is balanced
- Scheduling policies are correct
- Cluster is stable and production ready

## EXAM 6 KEY AREAS COVERED

- **ADVANCED SCHEDULING** — Placement, Affinity, Taints, Availability
- **RESOURCE GOVERNANCE** — Quotas, Limits, Priority Classes
- **PRODUCTION SCALING** — HPA, Metrics, Optimization
- **WORKLOAD RECOVERY** — Deployments, StatefulSets, DaemonSets
- **SCHEDULING VALIDATION** — Distribution, Policies, Stability

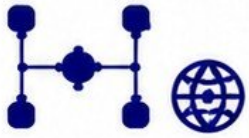
# MOCK EXAM 6 (TASKS 111 TO 115)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

FULL REAL-WORLD SIMULATION. ADVANCED ADMINISTRATION. HIGH-DIFFICULTY SCENARIOS.

Mock Exam 6 continues with networking recovery, security, storage, backup, and validation.
Each task is multi-step, realistic, and aligned to CKA exam objectives.

## 111 ENTERPRISE NETWORKING RECOVERY



### TASK STEPS

1. Restore Services
2. Recover Ingress
3. Repair CoreDNS
4. Validate Endpoints
5. Confirm external connectivity

```
# Check Services
kubectl get svc -A
# Check Ingress
kubectl get ingress -A
# Check CoreDNS
kubectl -n kube-system get pods -l k8s-app=kube-dns
# Validate Endpoints
kubectl get endpoints -A
# Test external connectivity
kubectl run net-test --rm -it --image=busybox:1.36
--restart=Never -- wget -qO- http://<external-ip>
```

### SUCCESS CRITERIA

- All Services are running
- Ingress is accessible
- CoreDNS is healthy
- Endpoints are correct
- External connectivity is confirmed

## 112 ADVANCED NETWORK SECURITY

### TASK STEPS

1. Configure NetworkPolicies
2. Restrict Pod communication
3. Validate traffic rules
4. Confirm secure connectivity
5. Verify policy enforcement

```
# List NetworkPolicies
kubectl get networkpolicies -A
# Apply deny-all policy
kubectl apply -f deny-all.yaml
# Apply allow rule
kubectl apply -f allow-app.yaml
# Test connectivity
kubectl run net-test -rm -it --image=busybox:1.36 --
nc -zv <target-ip> <port>
# Describe policy
kubectl describe networkpolicy <policy-name> -n <ns>
```

### SUCCESS CRITERIA

- NetworkPolicies are applied
- Pod communication is restricted
- Traffic rules are valid
- Secure connectivity works
- Policies are enforced

## 113 PERSISTENT STORAGE RECOVERY

### TASK STEPS

1. Recover PersistentVolumes
2. Recover PersistentVolumeClaims
3. Validate StorageClasses
4. Restore Stateful workloads
5. Verify data integrity

```
# Check PVs
kubectl get pv
# Check PVCs
kubectl get pvc -A
# Check StorageClasses
kubectl get storageclass
# Check StatefulSets
kubectl get statefulsets -A
# Verify data
kubectl exec -it <pod> -- ls /data
kubectl exec -it <pod> -- cat /data/health.txt
```

### SUCCESS CRITERIA

- PVs are available and bound
- PVCs are bound
- StorageClasses are valid
- Stateful workloads are running
- Data integrity is verified

## 114 PRODUCTION BACKUP AND RECOVERY

### TASK STEPS

1. Create etcd backup
2. Restore cluster state
3. Validate workloads
4. Confirm cluster consistency
5. Verify recovery success

```
# Backup etcd
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
# Restore etcd (example)
etcdctl snapshot restore /backup/etcd-YYYY-MM-DD.db \
  --data-dir=/var/lib/etcd-new
# Validate workloads
kubectl get pods -A
# Check cluster health
kubectl get componentstatuses
```

### SUCCESS CRITERIA

- etcd backup is created
- Cluster state is restored
- All workloads are running
- Cluster data is consistent
- Recovery is successful

## 115 ENTERPRISE STORAGE VALIDATION

### TASK STEPS

1. Verify persistent storage
2. Validate dynamic provisioning
3. Confirm volume expansion
4. Review storage health
5. Ensure application availability

```
# Check PVCs
kubectl get pvc -A
# Check StorageClasses
kubectl get storageclass
# Check volume expansion
kubectl get pvc -A -o wide
# Check storage health
kubectl get csidrivers
# Validate applications
kubectl get pods -A
```

### SUCCESS CRITERIA

- Persistent storage is healthy
- Dynamic provisioning works
- Volume expansion is confirmed
- Storage components are healthy
- Applications are available

## EXAM 6 KEY AREAS COVERED

- **NETWORKING** — Services, Ingress, CoreDNS, Endpoints
- **NETWORK SECURITY** — NetworkPolicies, Traffic Control
- **STORAGE** — PVs, PVCs, StorageClasses, Data Integrity
- **BACKUP & RECOVERY** — etcd Backup, Restore, Consistency
- **SCALING & SCHEDULING** — Autoscaling, Policies, Workload Placement

# MOCK EXAM 6 (TASKS 116 TO 120)

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

FULL REAL-WORLD SIMULATION. ADVANCED ADMINISTRATION. HIGH-DIFFICULTY SCENARIOS.

Mock Exam 6 concludes with security recovery, incident investigation, cross-domain recovery, and final production validation. Every task is multi-step, realistic, and aligned to CKA objectives.

## 116 ENTERPRISE SECURITY RECOVERY

### TASK STEPS

1. Diagnose RBAC failures
2. Restore Roles and RoleBindings
3. Validate ServiceAccounts
4. Verify Secrets
5. Confirm least-privilege access

```
# Check RBAC issues
kubectl auth can-i --list
kubectl get rolebindings,clusterrolebindings -A
# Restore Role
kubectl apply -f role.yaml
# Restore RoleBinding
kubectl apply -f rolebinding.yaml
# Validate ServiceAccounts
kubectl get serviceaccounts -A
# Verify Secrets
kubectl get secrets -A
# Test least privilege
kubectl auth can-i get pods --as system:serviceaccount:
default:app-user -n default
```

### SUCCESS CRITERIA

- RBAC issues resolved
- Roles and RoleBindings restored
- ServiceAccounts are valid
- Secrets are accessible
- Least-privilege access confirmed

## 117 PRODUCTION INCIDENT INVESTIGATION

### TASK STEPS

1. Correlate logs
2. Idealyze Events
3. Identify root cause
4. Prioritize remediation
5. Document findings

```
# Check logs
kubectl logs <pod> -n <ns> --all-containers --since=1h
# Analyze events
kubectl get events -A --sort-by=.lastTimestamp
# Describe resources
kubectl describe pod <pod> -n <ns>
kubectl describe node <node>
# Record root cause
echo "Root cause: <cause>" > findings.txt
# Document remediation
echo "Action plan: <steps>" >> findings.txt
```

### SUCCESS CRITERIA

- Logs correlated successfully
- Events analyzed
- Root cause identified
- Remediation prioritized
- Findings documented

## 118 CROSS-DOMAIN RECOVERY

### TASK STEPS

1. Resolve networking failures
2. Recover storage
3. Restore workloads
4. Validate security
5. Confirm cluster health

```
# Fix networking
kubectl get svc,ingress,endpoints -A
kubectl rollout restart deployment coredns -n kube-system
# Recover storage
kubectl get pvc -A
kubectl describe pvc <pvc> -n <ns>
# Restore workloads
kubectl rollout restart deployment -n <ns> --all
# Validate security
kubectl get networkpolicies -A
kubectl auth can-i --list
# Confirm health
kubectl get nodes,pods -A
kubectl get cs
```

### SUCCESS CRITERIA

- Networking restored
- Storage recovered
- Workloads running
- Security validated
- Cluster health confirmed

## 119 COMPLETE PRODUCTION VALIDATION

### TASK STEPS

1. Verify control plane
2. Verify worker nodes
3. Validate networking
4. Validate storage
5. Confirm operational readiness

```
# Control plane
kubectl get cs
kubectl get componentstatuses
# Worker nodes
kubectl get nodes -o wide
# Networking
kubectl get svc,ingress,endpoints -A
kubectl get networkpolicies -A
# Storage
kubectl get pv,pvc,storageclass -A
# Final validation
kubectl get pods -A
kubectl top nodes
```

### SUCCESS CRITERIA

- Control plane is healthy
- All nodes are Ready
- Networking is functional
- Storage is healthy
- Operational readiness confirmed

## 120 FINAL ENTERPRISE ASSESSMENT

### TASK STEPS

1. Troubleshoot multiple failures
2. Recover complete environment
3. Validate every objective
4. Confirm production readiness
5. Perform final submission review

```
# Diagnose all issues
kubectl get events -A --sort-by=.lastTimestamp
# Full recovery
kubectl rollout restart deployment -A
kubectl delete pods --all -A --grace-period=0 --force
# Validate objectives
kubectl get all -A
kubectl get cs
kubectl get nodes
kubectl get pv,pvc,storageclass -A
kubectl get networkpolicies -A
# Final review
echo "All objectives validated" > submission.txt
echo "Cluster ready for production" >> submission.txt
```

### SUCCESS CRITERIA

- All issues resolved
- Complete environment recovered
- All objectives validated
- Production readiness confirmed
- Final submission review complete

## EXAM 6 KEY AREAS COVERED

- **SECURITY** — RBAC, Secrets, ServiceAccounts, Least Privilege
- **INVESTIGATION** — Logs, Events, Root Cause, Remediation
- **CROSS-DOMAIN RECOVERY** — Networking, Storage, Workloads, Security
- **VALIDATION** — Control Plane, Nodes, Networking, Storage, Readiness
- **FINAL ASSESSMENT** — End-to-End Recovery, Objective Validation, Production Ready

**PRO TIP**

- Work methodically and do not skip steps.
- Use kubectl describe and logs to investigate.
- Validate continuously as you recover.

**REMEMBER**

- CKA is about solving real problems.
- Stay calm, verify, and take the right actions.
- Finish strong and submit with confidence.

**REMEMBER**

**FINAL GOAL**

Recover. Validate. Optimize. Deliver a fully functional, production-ready cluster.

# MOCK EXAM 6 SOLUTIONS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

FULL REAL-WORLD SIMULATION. ADVANCED ADMINISTRATION. HIGH-DIFFICULTY SCENARIOS.

Below are reference solutions for Tasks 101 to 120.
Commands are examples only. Actual solutions may vary based on your environment.

## 101 ENTERPRISE MULTI-CLUSTER ADMINISTRATION

- Configure contexts
- Switch clusters
- Validate connectivity
- Verify health
- Confirm access

SOLUTION EXAMPLE

```
# Add contexts
kubectl config get-contexts
kubectl config use-context prod-cluster-1
kubectl config use-context prod-cluster-2
# Validate connection
kubectl cluster-info
kubectl get nodes
kubectl get ns
kubectl auth can-i --list
```

## 102 CONTROL PLANE RECOVERY

- Diagnose API Server
- Recover Scheduler
- Restore Controller Manager
- Validate etcd
- Verify stability

SOLUTION EXAMPLE

```
# Check API Server
kubectl get --raw=/healthz
# Restart components (static pods)
kubectl delete pod -n kube-system \
  -l k8s-app=kube-scheduler
kubectl delete pod -n kube-system \
  -l k8s-app=kube-controller-manager
# Validate etcd
ETCDCTL_API=3 etcdctl endpoint health
--cluster
kubectl get componentstatuses
```

## 103 ENTERPRISE NODE ADMINISTRATION

- Drain worker nodes
- Upgrade kubelet
- Uncordon nodes
- Validate recovery
- Confirm readiness

SOLUTION EXAMPLE

```
kubectl drain node01 --ignore-daemonsets \
  --delete-emptydir-data
# Upgrade kubelet (example)
sudo apt-get update && sudo apt-get install \
  -y kubelet=1.28.0-00 kubeadm=1.28.0-00
sudo systemctl daemon-reload
sudo systemctl restart kubelet

kubectl uncordon node01
kubectl get nodes
```

## 104 CERTIFICATE LIFECYCLE MANAGEMENT

- Inspect certs
- Renew expired certs
- Restart services
- Validate TLS
- Verify functionality

SOLUTION EXAMPLE

```
# Inspect certificates
sudo kubeadm certs check-expiration
# Renew expired certs
sudo kubeadm certs renew all
# Restart control plane
sudo systemctl restart kubelet
# Validate secure communication
kubectl get --raw=/healthz
kubectl get nodes
```

## 105 PRODUCTION CLUSTER VALIDATION

- Verify namespaces
- Validate workloads
- Check control plane
- Confirm networking
- Ensure readiness

SOLUTION EXAMPLE

```
kubectl get ns
kubectl get all -A
kubectl get componentstatuses
kubectl get nodes
kubectl get svc,ing,ep -A
kubectl get pods --all-namespaces
kubectl cluster-info
kubectl get cs
```

## 106 ADVANCED SCHEDULING

- Taints & tolerations
- Node affinity
- Pod anti-affinity
- Validate placement
- Optimize availability

SOLUTION EXAMPLE

```
# Taint
kubectl taint nodes node01 env=prod:NoSchedule
# Toleration (pod spec)
tolerations:
- key: "env"
  operator: "Equal"
  value: "prod"
  effect: "NoSchedule"
# Node affinity (pod spec)
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: disktype
              operator: In
              values: ["ssd"]
# Anti-affinity (pod spec)
podAntiAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
    labelSelector:
      matchLabels:
        app: web
      topologyKey: kubernetes.io/hostname
kubectl get pods -o wide
```

## 107 RESOURCE GOVERNANCE

- ResourceQuota
- LimitRange
- PriorityClass
- Validate policies
- Prevent exhaustion

SOLUTION EXAMPLE

```
# ResourceQuota
kubectl apply -f resourcequota.yaml
# LimitRange
kubectl apply -f limitrange.yaml
# PriorityClass
kubectl apply -f priorityclass.yaml

kubectl get resourcequota
kubectl get limitrange
kubectl get priorityclass
```

## 108 PRODUCTION SCALING

- HPA configuration
- Validate metrics
- Verify scaling
- Confirm stability
- Optimize resources

SOLUTION EXAMPLE

```
kubectl apply -f hpa.yaml
kubectl get hpa
kubectl get pods
kubectl top pods
kubectl describe hpa web-hpa
```

## 109 ADVANCED WORKLOAD RECOVERY

- Recover Deployments
- Recover StatefulSets
- Recover DaemonSets
- Validate updates
- Confirm availability

SOLUTION EXAMPLE

```
kubectl rollout undo deployment/web
kubectl rollout status deployment/web
kubectl rollout undo statefulset/db
kubectl rollout status statefulset/db
kubectl rollout status daemonset/node-agent
kubectl get all -A
```

## 110 ENTERPRISE SCHEDULING VALIDATION

- Verify distribution
- Confirm HA
- Validate utilization
- Review policies
- Confirm stability

SOLUTION EXAMPLE

```
kubectl get pods -o wide
kubectl top nodes
kubectl top pods --all-namespaces
kubectl get nodes --show-labels
kubectl get networkpolicies --all-namespaces
kubectl get poddisruptionbudgets --all-namespaces
```

## 111 ENTERPRISE NETWORKING RECOVERY

- Restore Services
- Recover Ingress
- Repair CoreDNS
- Validate Endpoints
- Confirm connectivity

SOLUTION EXAMPLE

```
kubectl get svc -A
kubectl get ingress -A
kubectl rollout restart deployment/coredns \
  -n kube-system
kubectl get endpoints -A
curl -k https://<external-ip>
```

## 112 ADVANCED NETWORK SECURITY

- Configure policies
- Restrict traffic
- Validate rules
- Secure connectivity
- Verify enforcement

SOLUTION EXAMPLE

```
kubectl apply -f networkpolicy.yaml
kubectl get networkpolicies -A
kubectl describe netpol default-deny
kubectl get pods --show-labels
kubectl exec -it pod-a -- \
  curl http://pod-b:8080
```

## 113 PERSISTENT STORAGE RECOVERY

- Recover PVs
- Recover PVCs
- Validate SCs
- Restore workloads
- Verify integrity

SOLUTION EXAMPLE

```
kubectl get pv,pvc -A
kubectl get sc
kubectl apply -f pvc.yaml
kubectl rollout restart statefulset/db
kubectl exec -it db-0 -- ls /data
kubectl exec -it db-0 -- \
  cat /data/health.txt
```

## 114 PRODUCTION BACKUP & RECOVERY

- Backup etcd
- Restore state
- Validate workloads
- Cluster consistency
- Verify success

SOLUTION EXAMPLE

```
# Backup etcd
ETCDCTL_API=3 etcdctl snapshot save /backup/
etcd-snap.db --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
# Restore etcd (if required)
etcdctl snapshot restore etcd-snap.db \
  --data-dir /var/lib/etcd-new
systemctl restart etcd kube-apiserver
kubectl get all -A
```

## 115 ENTERPRISE STORAGE VALIDATION

- Verify storage
- Dynamic provisioning
- Volume expansion
- Review health
- Ensure availability

SOLUTION EXAMPLE

```
kubectl get pv,pvc -A
kubectl get sc
kubectl get events --field-selector \
  reason=ProvisioningSucceeded
kubectl describe pvc data-pvc
kubectl get cs
```

## SOLUTION BEST PRACTICES

- Always validate before and after every change.
- Use dry-run and describe to avoid mistakes.
- Recover step-by-step and verify each layer.
- Document every action and output.
- Think like a Senior Kubernetes Administrator.

EXAM 6 KEY AREAS COVERED

- MULTI-CLUSTER & ADMIN — Contexts, Access, Health
- CONTROL PLANE RECOVERY — API Server, Scheduler, Controller, etcd
- NODE ADMINISTRATION — Drain, Upgrade, Readiness
- SCHEDULING & SCALING — Placement, Policies, HPA, Optimization
- STORAGE & BACKUP — PVs, PVCs, Backup, Recovery
- SECURITY & NETWORKING — RBAC, NetworkPolicies, Security, Recovery
- VALIDATION & PRODUCTION — Health, Readiness, Production Ready



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# MOCK EXAM 6 SENIOR ADMINISTRATOR REVIEW

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

FULL REAL-WORLD SIMULATION. ADVANCED ADMINISTRATION. HIGH-DIFFICULTY SCENARIOS.

You have completed Mock Exam 6.
This review page helps you strengthen your understanding, close gaps, and prepare for real-world production challenges.

### MULTI-CLUSTER ADMINISTRATION

- Configure contexts
- Switch clusters
- Validate connectivity
- Verify cluster health
- Confirm admin access

### CONTROL PLANE RECOVERY

- Diagnose API Server
- Recover Scheduler
- Restore Controller Manager
- Validate etcd health
- Verify cluster stability

### NODE ADMINISTRATION

- Drain worker nodes
- Upgrade kubelet
- Uncordon nodes
- Validate workloads
- Confirm node readiness

### SECURITY & CERTIFICATES

- Inspect certificates
- Renew expired certs
- Restart services
- Validate TLS
- Verify functionality

### SCHEDULING & GOVERNANCE

- Taints & tolerations
- Affinity & anti-affinity
- ResourceQuota
- LimitRange
- PriorityClass

### SCALING & RECOVERY

- HPA & Metrics
- Workload recovery
- Storage recovery
- Network recovery
- Backup & restore

## SENIOR ADMINISTRATOR MINDSET

- **Think End-to-End:** Always validate the full stack from control plane to workloads.
- **Observe Before Acting:** Collect evidence, analyze, then make precise changes.
- **Automate Repetitive Tasks:** Use scripts and IaC to ensure consistency.
- **Design for Failure:** Build systems that can recover automatically.
- **Security First:** Apply least privilege and validate continuously.
- **Document Everything:** Notes, runbooks, and postmortems build reliability.
- **Continuously Improve:** Learn from incidents and refine your systems.

## COMMAND MASTERY – SENIOR LEVEL

```
# Cluster & Context
kubectl config get-contexts
kubectl config use-context prod-cluster-1
kubectl cluster-info
kubectl get componentstatuses

# Control Plane
kubectl get pods -n kube-system
kubectl logs -n kube-system <pod>
kubectl get events -A --sort-by=.metadata.creationTimestamp

# Nodes
kubectl get nodes -o wide
kubectl drain node01 --ignore-daemonsets --delete-emptydir-data
kubectl uncordon node01
kubectl describe node node01

# Workloads
kubectl get deploy,sts,ds,pods -A
kubectl rollout status deploy/<name> -n <ns>
kubectl describe pod <pod> -n <ns>

# Networking
kubectl get svc,ing,ep -A
kubectl describe svc <svc> -n <ns>
kubectl get networkpolicies -A

# Storage
kubectl get pv,pvc,sc -A
kubectl describe pv <pv>
kubectl describe pvc <pvc> -n <ns>

# Security
kubectl get roles,rolebindings,sa -A
kubectl describe role <role> -n <ns>
kubectl auth can-i --list

# Backup & Recovery
etcdctl snapshot save backup.db
etcdctl snapshot status backup.db
```

## HIGH-DIFFICULTY AREAS REVIEW

- Multi-cluster administration
- Control plane recovery
- Certificate lifecycle management
- Advanced scheduling
- Resource governance
- Autoscaling and metrics
- Network policies and security
- Persistent storage recovery
- Backup and disaster recovery
- Cross-domain troubleshooting

## TROUBLESHOOTING APPROACH

1. **Identify the problem**
   Understand the symptoms and impact.
2. **Collect evidence**
   Logs, events, metrics, and states.
3. **Analyze root cause**
   Correlate data and validate theory.
4. **Implement fix**
   Apply the minimal effective change.
5. **Validate results**
   Confirm resolution and stability.
6. **Document**
   Record what happened and learned.

## SENIOR ADMIN BEST PRACTICES

- Use declarative configuration
- Enforce security at every layer
- Monitor continuously
- Automate backups
- Test disaster recovery regularly
- Maintain runbooks
- Perform regular audits
- Optimize resources
- Plan for scalability
- Stay updated with Kubernetes

## READINESS CHECKLIST

- ☐ I can manage multiple clusters.
- ☐ I can recover the control plane.
- ☐ I can upgrade and manage nodes.
- ☐ I understand certificates and TLS.
- ☐ I can secure and govern resources.
- ☐ I can configure advanced scheduling.
- ☐ I can scale applications automatically.
- ☐ I can troubleshoot networking issues.
- ☐ I can recover storage and workloads.
- ☐ I can backup and restore clusters.
- ☐ I can investigate and resolve incidents.
- ☐ I am ready for real-world production.

## EXAM SUCCESS FORMULA

**Knowledge** + **Practice** + **Analysis** + **Confidence**

Understand concepts deeply | Solve problems consistently | Think and troubleshoot | Stay calm and execute

**= CKA SUCCESS**

You have the skills. You have the knowledge. Execute with confidence and succeed!

EXAM 6 KEY AREAS COVERED

MULTI-CLUSTER ADMINISTRATION | CONTROL PLANE RECOVERY | NODE ADMINISTRATION | SCHEDULING & GOVERNANCE | SCALING & RECOVERY | SECURITY & CERTIFICATES | STORAGE & BACKUP

# RAPID FIRE LAB 1

## 20 TASKS. 20 MINUTES.
## SPEED. ACCURACY. CONFIDENCE.

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

HANDS-ON RAPID PRACTICE. REAL COMMANDS. REAL RESULTS. EXAM-FOCUSED. PRODUCTION-READY.

Complete all tasks as fast and accurately as possible. Use kubectl, YAML, and best practices.
Each task is independent. Move quickly and stay focused.

### 01 CREATE NAMESPACE

Create a namespace named rapid-lab1.

```
kubectl create namespace rapid-lab1
```

EXPECTED: Namespace created.

### 02 RUN NGINX POD

Run an nginx pod named web-pod in rapid-lab1.

```
kubectl run web-pod --image=nginx -n rapid-lab1
```

EXPECTED: Pod is Running.

### 03 CHECK POD STATUS

Check the status of web-pod.

```
kubectl get pod web-pod -n rapid-lab1
```

EXPECTED: Pod is Running.

### 04 CREATE DEPLOYMENT

Create a deployment named web-deploy with 3 replicas.

```
kubectl create deployment web-deploy \
--image=nginx \
--replicas=3 -n rapid-lab1
```

EXPECTED: Deployment created. 3 replicas Running.

### 05 EXPOSE SERVICE

Expose web-deploy as a NodePort service on port 80.

```
kubectl expose deployment web-deploy \
--type=NodePort --port=80 -n rapid-lab1
```

EXPECTED: Service created. NodePort assigned.

### 06 CREATE CONFIGMAP

Create a ConfigMap named app-config with key APP_MODE=production.

```
kubectl create configmap app-config \
--from-literal=APP_MODE=production -n rapid-lab1
```

ConfigMap created.

### 07 CREATE SECRET

Create a secret named app-secret with key PASSWORD=StrongPass123.

```
kubectl create secret generic app-secret \
--from-literal=PASSWORD=StrongPass123 -n rapid-lab1
```

Secret created.

### 08 CREATE PVC

Create a PersistentVolumeClaim named data-pvc with 1Gi and StorageClass gp3.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
  namespace: rapid-lab1
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
  storageClassName: gp3
```

Pods: PVC created. Bound.

### 09 DEPLOY POD WITH PVC

Create a pod using nginx image and mount data-pvc at /usr/share/nginx/html.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pvc-pod
  namespace: rapid-lab1
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
      - name: data
        mountPath: /usr/share/nginx/html
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data-pvc
```

Pod: Pod Running. PVC mounted.

### 10 SCALE DEPLOYMENT

Scale web-deploy to 5 replicas.

```
kubectl scale deployment web-deploy --replicas=5 -n rapid-lab1
```

5 replicas Running.

### 11 UPDATE IMAGE

Update web-deploy to nginx:1.25.

```
kubectl set image deployment web-deploy \
nginx=nginx:1.25 -n rapid-lab1
```

EXPECTED: Image updated.

### 12 CHECK ROLLOUT STATUS

Check rollout status of web-deploy.

```
kubectl rollout status deployment web-deploy -n rapid-lab1
```

EXPECTED: Rollout successful.

### 13 CREATE SERVICEACCOUNT

Create a service account named app-sa.

```
kubectl create serviceaccount app-sa -n rapid-lab1
```

ServiceAccount created.

### 14 BIND ROLE

Bind cluster role view to app-sa.

```
kubectl create clusterrolebinding app-sa-view \
--clusterrole=view --serviceaccount=rapid-lab1:app-sa
```

RoleBinding created.

### 15 APPLY NETWORK POLICY

Deny all ingress traffic to pods labeled app=web.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: rapid-lab1
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
  - Ingress
```

NetworkPolicy applied.

### 16 LABEL POD

Label web-pod with app=web.

```
kubectl label pod web-pod app=web -n rapid-lab1
```

Label updated.

### 17 CHECK NODES

List all nodes.

```
kubectl get nodes
```

Nodes listed.

### 18 DRAIN NODE (SIMULATION)

Cordon and drain the first worker node.

```
kubectl cordon $(kubectl get nodes \
    -o jsonpath='{.items[1].metadata.name}')
kubectl drain $(kubectl get nodes \
    -o jsonpath='{.items[1].metadata.name}') \
    --ignore-daemonsets --delete-emptydir-data
```

Node cordoned and drained.

### 19 CREATE BACKUP (ETCD SNAPSHOT SIMULATION)

Create a directory /backup and a file backup.txt.

```
mkdir -p /backup
echo "etcd-backup-$(date)" > /backup/backup.txt
```

Backup file created.

### 20 CLEANUP

Delete all resources in rapid-lab1 namespace.

```
kubectl delete namespace rapid-lab1
```

Namespace deleted.

## RAPID FIRE RULES

- Work fast but stay accurate.
- Use correct syntax.
- Verify after each task.
- Fix mistakes quickly.
- Think like a senior admin.
- Quality over speed.

## TIME PLAN (20 MINUTES)

- Tasks 01 – 05 : 0 – 5 min
- Tasks 06 – 10 : 5 – 10 min
- Tasks 11 – 15 : 10 – 15 min
- Tasks 16 – 20 : 15 – 20 min

Goal: Finish all 20 tasks.
Review results.

## COMMAND SHORTCUTS

```
kubectl get all -A
kubectl describe <resource> <name>
kubectl logs <pod> -n <ns>
kubectl exec -it <pod> -n <ns> -- /bin/bash
kubectl apply -f <file.yaml>
kubectl delete -f <file.yaml>
kubectl rollout undo deployment/<name>
kubectl top nodes
kubectl top pods -A
```

## SENIOR ADMIN TIPS

- Automate repetitive tasks.
- Use labels for organization.
- Validate configurations.
- Monitor continuously.
- Document everything.
- Secure by default.

## SELF EVALUATION

How did you perform?
Rate your performance:

★ ★ ★ ★ ★ ★

Outstanding | Good | Average | Needs Work

What can you improve?

_____
_____
_____
_____

PRACTICE DAILY. IMPROVE SPEED. BUILD CONFIDENCE. BE PRODUCTION READY.

Rapid practice builds real skill.
Real skill builds real career.

EXAM 6 KEY AREAS COVERED

- MULTI-CLUSTER ADMINISTRATION
- CONTROL PLANE RECOVERY
- NODE ADMINISTRATION
- STORAGE & BACKUP
- NETWORKING & SECURITY
- SCHEDULING & SCALING



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# RAPID FIRE LAB 2

## 30 TIMED PRACTICAL TASKS. REAL-WORLD ADMINISTRATION.

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

SPEED. ACCURACY. CONFIDENCE. HANDS-ON PRACTICE. EXAM-FOCUSED. PRODUCTION-READY.

30 timed tasks. Complete as fast and accurately as possible.
Each task is independent. Use best practices. Real commands. Real results.

### 01 CREATE NAMESPACE

Create namespace named rf2-ns1.

```
kubectl create namespace rf2-ns1
```

EXPECTED: Namespace created.

### 02 RUN POD

Run nginx pod named rf2-nginx.

```
kubectl run rf2-nginx --image=nginx
```

EXPECTED: Pod is Running.

### 03 EXPOSE POD

Expose rf2-nginx on port 8080.

```
kubectl expose pod rf2-nginx --port=8080
```

EXPECTED: Service created.

### 04 CREATE DEPLOYMENT

Create deployment rf2-web with 2 replicas.

```
kubectl create deployment rf2-web \
--image=nginx --replicas=2
```

EXPECTED: Deployment created.

### 05 SCALE DEPLOYMENT

Scale rf2-web deployment to 4 replicas.

```
kubectl scale deployment rf2-web --replicas=4
```

EXPECTED: Scaled to 4 replicas.

### 06 CREATE CONFIGMAP

Create ConfigMap rf2-config from literal.

```
kubectl create configmap rf2-config \
--from-literal=key1=value1
```

EXPECTED: ConfigMap created.

### 07 CREATE SECRET

Create secret rf2-secret with key TOKEN=abc123.

```
kubectl create secret generic rf2-secret \
--from-literal=TOKEN=abc123
```

EXPECTED: Secret created.

### 08 CREATE PVC

Create PVC rf2-pvc with 1Gi storage.

```
kubectl apply -f rf2-pvc.yaml
```

EXPECTED: PVC created.

### 09 DEPLOY WITH PVC

Deploy nginx using rf2-pvc mounted.

```
kubectl apply -f rf2-pod-pvc.yaml
```

EXPECTED: Pod Running.

### 10 CREATE SERVICE

Create ClusterIP service for rf2-web.

```
kubectl expose deployment rf2-web --port=80
```

EXPECTED: Service created.

### 11 UPDATE IMAGE

Update rf2-web image to nginx:1.25.

```
kubectl set image deployment rf2-web \
nginx=nginx:1.25
```

EXPECTED: Image updated.

### 12 CHECK ROLLOUT

Check rollout status of rf2-web.

```
kubectl rollout status deployment rf2-web
```

EXPECTED: Rollout successful.

### 13 ROLLBACK DEPLOYMENT

Rollback rf2-web to previous revision.

```
kubectl rollout undo deployment rf2-web
```

EXPECTED: Rollback done.

### 14 CREATE JOB

Create a job named rf2-job.

```
kubectl apply -f rf2-job.yaml
```

EXPECTED: Job created.

### 15 CREATE CRONJOB

Create cronjob rf2-cron (*/2 * * * *).

```
kubectl apply -f rf2-cronjob.yaml
```

EXPECTED: CronJob created.

### 16 CREATE DAEMONSET

Create DaemonSet rf2-ds on all nodes.

```
kubectl apply -f rf2-daemonset.yaml
```

EXPECTED: DaemonSet created.

### 17 TAINT NODE

Taint node with key=dedicated:NoSchedule.

```
kubectl taint nodes <node> dedicated=team:NoSchedule
```

EXPECTED: Taint applied.

### 18 TOLERATION POD

Create pod with toleration for above taint.

```
kubectl apply -f rf2-toleration-pod.yaml
```

EXPECTED: Pod scheduled.

### 19 NODE AFFINITY

Run pod with nodeAffinity to specific node.

```
kubectl apply -f rf2-affinity-pod.yaml
```

EXPECTED: Pod scheduled.

### 20 POD ANTI-AFFINITY

Run 3 pods with anti-affinity on hostname.

```
kubectl apply -f rf2-anti-affinity.yaml
```

EXPECTED: Pods spread.

### 21 CREATE NETWORK POLICY

Deny all ingress to rf2-app pods.

```
kubectl apply -f rf2-deny-ingress.yaml
```

EXPECTED: Policy applied.

### 22 ALLOW INGRESS

Allow ingress from namespace rf2-ns1.

```
kubectl apply -f rf2-allow-ingress.yaml
```

EXPECTED: Ingress allowed.

### 23 LIMIT RANGE

Create LimitRange for rf2-ns1.

```
kubectl apply -f rf2-limitrange.yaml
```

EXPECTED: LimitRange created.

### 24 RESOURCE QUOTA

Create ResourceQuota for rf2-ns1.

```
kubectl apply -f rf2-resourcequota.yaml
```

EXPECTED: Quota applied.

### 25 HORIZONTAL POD AUTOSCALER

Create HPA for rf2-web (1-5 replicas).

```
kubectl autoscale deployment rf2-web \
--cpu-percent=50 --min=1 --max=5
```

EXPECTED: HPA created.

### 26 CREATE INGRESS

Create Ingress for rf2-web.

```
kubectl apply -f rf2-ingress.yaml
```

EXPECTED: Ingress created.

### 27 EDIT CONFIGMAP

Add key2=value2 to rf2-config.

```
kubectl edit configmap rf2-config
```

EXPECTED: ConfigMap updated.

### 28 ROTATE SECRET

Update secret rf2-secret with new value.

```
kubectl create secret generic rf2-secret \
--from-literal=TOKEN=new123 --dry-run=client -o
yaml | kubectl apply -f -
```

EXPECTED: Secret updated.

### 29 CORDON & DRAIN

Cordon and drain a worker node.

```
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets
--delete-emptydir-data
```

EXPECTED: Node drained.

### 30 DELETE RESOURCES

Delete all resources in rf2-ns1.

```
kubectl delete namespace rf2-ns1
```

EXPECTED: Namespace deleted.

## RAPID FIRE RULES

- Work fast but stay accurate.
- Each task is independent.
- Use correct syntax.
- Verify after every task.
- Fix mistakes quickly.
- Think like a senior admin.
- Quality over speed.

## TIME PLAN (30 MINUTES)

- Tasks 01 - 10 : 0 - 10 min
- Tasks 11 - 20 : 10 - 20 min
- Tasks 21 - 30 : 20 - 30 min

Goal: Complete all 30 tasks.
Review results.

## COMMAND SHORTCUTS

```
kubectl get all -A
kubectl describe <resource> <name>
kubectl logs <pod> -n <ns>
kubectl exec -it <pod> -n <ns> -- /bin/bash
kubectl apply -f <file.yaml>
kubectl delete -f <file.yaml>
kubectl rollout status deployment/<name>
kubectl top nodes
kubectl top pods -A
kubectl config use-context <ctx>
```

## SENIOR ADMIN TIPS

- Automate repetitive tasks.
- Use YAML for everything.
- Validate configuration.
- Observe system behavior.
- Monitor and troubleshoot.
- Document best practices.
- Secure by default.

## SELF EVALUATION

Rate your performance

★ ★ ★ ★ ★

What went well?

What can improve?

PRACTICE FAST. PRACTICE SMART. BECOME A SENIOR KUBERNETES ADMINISTRATOR.

Real practice. Real skills. Real career.

EXAM 6 KEY AREAS COVERED

- MULTI-CLUSTER ADMINISTRATION
- CONTROL PLANE RECOVERY
- NODE ADMINISTRATION
- SECURITY & NETWORKING
- STORAGE & BACKUP
- SCHEDULING & SCALING



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# RAPID FIRE LAB 3

## 30 ADVANCED PRICTICAL TASKS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

SPEED. ACCURACY. CONFIDENCE. HANDS-ON PRACTICE. EXAM-FOCUSED. PRODUCTION-READY.

30 advanced, real-world practical tasks. Apply your skills, think fast, and demonstrate enterprise-level Kubernetes administration.

**01 Multi-Cluster Federation** — Networking
Configure and validate multi-cluster federation with secure connectivity.

**02 Advanced Network Policies** — Security
Implement complex NetworkPolicies with namespace isolation.

**03 Pod Security Standards** — Security
Enforce Pod Security Standards with restricted policies.

**04 Dynamic Admission Control** — Security
Deploy and configure custom admission controllers.

**05 Service Mesh Deployment** — Networking
Deploy Istio/Linkerd and configure service mesh for traffic management.

**06 Advanced Storage** — Storage
Configure CSI drivers and advanced storage features.

**07 Cross-Cluster Backup** — Backup
Implement cross-cluster backup and disaster recovery.

**08 Performance Optimization** — Performance
Tune scheduler, kubelet and cluster performance settings.

**09 Custom Resource Definitions** — Extensions
Create and manage CRDs with custom controllers.

**10 Advanced Logging Stack** — Monitoring
Deploy EFK/ELK stack for comprehensive log management.

**11 Monitoring & Alerting** — Monitoring
Configure Prometheus, Grafana and alerting rules.

**12 Cluster Autoscaler** — Scaling
Deploy and configure Cluster Autoscaler for node scaling.

**13 Resource Quotas** — Resource
Enforce hard resource quotas across namespaces.

**14 Priority Classes** — Scheduling
Create and apply PriorityClasses for critical workloads.

**15 Pod Disruption Budgets** — Reliability
Configure PDBs for high availability during voluntary disruptions.

**16 Node Pools** — Infrastructure
Create and manage multiple node pools with different labels.

**17 GVK-based RBAC** — Security
Configure fine-grained RBAC using specific API groups.

**18 Cluster Upgrade Strategy** — Maintenance
Perform a zero-downtime Kubernetes cluster upgrade.

**19 Traffic Management** — Networking
Configure Ingress with canary and blue/green deployment.

**20 GitOps Deployment** — Automation
Implement GitOps with Argo CD for continuous delivery.

**21 Multi-Tenant Namespace** — Multi-tenancy
Design and enforce multi-tenant namespace architecture.

**22 Advanced RBAC Policies** — Security
Create complex RBAC rules with conditional access.

**23 Federated Identity** — Security
Integrate OIDC/SAML for enterprise authentication.

**24 Secret Management** — Security
Implement external secret stores (Vault, AWS Secrets Manager).

**25 Network Encryption** — Security
Enable mTLS for service-to-service communication.

**26 Advanced Scheduling** — Scheduling
Configure taints, tolerations and node affinity rules.

**27 CI/CD Pipeline** — Automation
Integrate Kubernetes with Jenkins/GitLab for automated deployment.

**28 Disaster Recovery Automation** — Backup
Automate backup and restore using Velero with validation.

**29 Security Audit** — Security
Run security scans and compliance validation with kube-bench.

**30 Production Readiness** — Validation
Run full production readiness validation with all checks.

EXAM 6 KEY AREAS COVERED

- ADVANCED SECURITY
- HIGH AVAILABILITY & SCALING
- STORAGE & BACKUP
- NETWORKING & SERVICE MESH
- AUTOMATION & GitOps
- PRODUCTION OPERATIONS

PRACTICE. DEPLOY. VALIDATE. SUCCEED.

REAL SKILLS. REAL CONFIDENCE. REAL EXAM SUCCESS.

# HIGH-YIELD TROUBLESHOOTING SCENARIOS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

REAL PROBLEMS. FAST DIAGNOSIS. ACCURATE FIXES. PRODUCTION CONFIDENCE.

These are the most common production issues you must solve quickly in the CKA exam and in real environments. Follow the flow, run the commands, identify root cause, and fix it.

## 01 POD FAILURES

**SYMPTOMS**
- Pod not starting
- Pod in CrashLoopBackOff
- Image pull errors
- Pod stuck in Pending
- OOMKilled / Evicted

**DIAGNOSE (COMMANDS)**
```
$ kubectl get pods -A
$ kubectl describe pod <pod>
$ kubectl logs <pod>
$ kubectl logs <pod> -c <container>
$ kubectl get events --sort-by=.metadata.creationTimestamp
$ kubectl top pod <pod>
```

**ROOT CAUSES**
- Wrong image or tag
- Missing ConfigMap/Secret
- Insufficient resources
- Liveness/Readiness probe failure
- Node not available
- Pod anti-affinity rules

**FIX / RESOLUTION**
- Correct image or tag
- Create/restore ConfigMap/Secret
- Adjust resources
- Fix probes
- Pull image / Fix imagePullSecrets
- Check node health
- Relax or correct affinity rules

**PRO TIPS**
- ☑ Always check events first.
- ☑ Logs tell you the truth.
- ☑ Validate probes and resources.
- ☑ Confirm dependencies exist.
- ☑ Use kubectl describe.

## 02 SERVICE FAILURES

**SYMPTOMS**
- Service not reachable
- <none> Endpoints
- Wrong Pod receiving traffic
- External IP not assigned
- Ingress route not working

**DIAGNOSE (COMMANDS)**
```
$ kubectl get svc -A
$ kubectl describe svc <svc>
$ kubectl get endpoints <svc> -o wide
$ kubectl get pods -l <label>
$ kubectl describe ingress <ing>
$ kubectl logs -n kube-system -l k8s-app=kube-proxy
```

**ROOT CAUSES**
- No matching Pods (selector mismatch)
- Pod not Ready
- Wrong port/targetPort
- Service type mismatch
- Ingress rule incorrect
- kube-proxy or CNI issues

**FIX / RESOLUTION**
- Fix selector or labels
- Make Pods Ready
- Correct ports
- Use correct Service type
- Fix Ingress rules/host/path
- Restart kube-proxy / Fix CNI

**PRO TIPS**
- ☑ Check Endpoints.
- ☑ Verify labels match.
- ☑ Test with port-forward.
- ☑ Check DNS resolution.
- ☑ Follow traffic path.

## 03 STORAGE FAILURES

**SYMPTOMS**
- PVC stuck in Pending
- Pod fail to mount volume
- Volume not found
- Read/Write errors
- Data lost after restart

**DIAGNOSE (COMMANDS)**
```
$ kubectl get pvc,pv
$ kubectl describe pvc <pvc>
$ kubectl describe pv <pv>
$ kubectl describe pod <pod>
$ kubectl get events --sort-by=.lastTimestamp
$ df -h (inside pod)
$ mount | grep <mountPath>
```

**ROOT CAUSES**
- No matching PV
- StorageClass misconfig
- Insufficient storage
- Node disk pressure
- Permission issues
- Volume multi-attach error

**FIX / RESOLUTION**
- Create PV or fix StorageClass
- Verify access modes
- Expand storage
- Free disk space
- Fix permissions (fsGroup)
- Attach volume to single node

**PRO TIPS**
- ☑ Use dynamic provisioning.
- ☑ Check StorageClass events.
- ☑ Validate accessModes.
- ☑ Monitor node disk.
- ☑ Test write/read inside pod.

## 04 NETWORK FAILURES

**SYMPTOMS**
- Pod cannot reach another Pod
- DNS resolution fails
- Ingress unreachable
- External access fails
- NetworkPolicy blocking traffic

**DIAGNOSE (COMMANDS)**
```
$ kubectl get pods -A -o wide
$ kubectl exec -it <pod> -- nslookup <svc>
$ kubectl exec -it <pod> -- ping <ip>
$ kubectl get networkpolicy -A
$ kubectl describe node <n>
$ kubectl logs -n kube-system -l k8s-app=kube-dns
```

**ROOT CAUSES**
- Wrong DNS / CoreDNS down
- NetworkPolicy blocking
- Service/CNI misconfig
- Ingress controller down
- Node network issue
- Firewall / SG blocking

**FIX / RESOLUTION**
- Fix CoreDNS / DNS records
- Update NetworkPolicy
- Repair CNI / Service config
- Restart Ingress controller
- Fix node networking
- Open required ports

**PRO TIPS**
- ☑ Test DNS first.
- ☑ Check NetworkPolicy.
- ☑ Use exec to test connectivity.
- ☑ Review CNI and routes.
- ☑ Validate end-to-end path.

## 05 RBAC FAILURES

**SYMPTOMS**
- Forbidden error
- User cannot access resource
- ServiceAccount permission issues
- Token errors
- Role/Binding not working

**DIAGNOSE (COMMANDS)**
```
$ kubectl auth can-i <verb> <resource> -n <ns>
$ kubectl get roles,rolebindings -A
$ kubectl get clusterroles,clusterrolebindings
$ kubectl describe sa <sa> -n <ns>
$ kubectl get events -n <ns> | grep denied
$ kubectl auth whoami
```

**ROOT CAUSES**
- Missing Role/ClusterRole
- RoleBinding not applied
- Wrong namespace
- ServiceAccount missing
- Token expired/invalid

**FIX / RESOLUTION**
- Create or fix Role/ClusterRole
- Create or fix RoleBinding
- Bind in correct namespace
- Verify ServiceAccount
- Recreate token / restart pod

**PRO TIPS**
- ☑ Use kubectl auth can-i.
- ☑ Least privilege always.
- ☑ Check namespace context.
- ☑ Inspect ServiceAccount.
- ☑ Understand RBAC flow.

## TROUBLESHOOTING WORKFLOW (ALWAYS FOLLOW)

1. OBSERVE — Understand the symptom
2. GATHER INFO — Logs, events, describe, status
3. ANALYZE — Find the root cause
4. APPLY FIX — Make the correct change
5. VERIFY — Confirm issue is resolved

## SENIOR ADMIN CHECKLIST

- ☑ Validate cluster health
- ☑ Check events and logs
- ☑ Verify configurations
- ☑ Test connectivity
- ☑ Confirm resources
- ☑ Review security
- ☑ Document & learn
- ☑ Prevent recurrence

TROUBLESHOOTING IS A SKILL. PRACTICE DAILY. THINK LIKE A SENIOR ADMIN.

Real problems build real skill.
Real skill builds real career.

EXAM 6 KEY AREAS COVERED

PODS & WORKLOADS | SERVICES & NETWORKING | STORAGE & PERSISTENCE | SECURITY & RBAC | OBSERVABILITY & LOGS



veriqta veriqta veriqta veriqta veriqta

# HIGH-YIELD COMMAND CHALLENGE

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

COMPLETE. CORRECT. MASTER. SPEED MATTERS. ACCURACY WINS. EXAM READY.

This page trains your speed, accuracy, and command recall.
Solve all sections quickly and confidently. These are real exam-style challenges.

## 1 COMPLETE MISSING COMMANDS

Fill in the blanks to complete each kubectl command.

| # | TASK | COMMAND (FILL IN THE BLANKS) |
|---|---|---|
| 1 | Get all pods in default namespace | kubectl get _____ |
| 2 | Get pods in namespace dev | kubectl get pods -n _____ |
| 3 | Describe a pod named nginx | kubectl describe _____ nginx |
| 4 | Get all deployments in all namespaces | kubectl get _____ --all-namespaces |
| 5 | Delete a pod named redis | kubectl delete _____ redis |
| 6 | Apply a deployment file | kubectl apply -f _____ |
| 7 | Get services in namespace prod | kubectl get services -n _____ |
| 8 | Get nodes with wide output | kubectl get nodes -o _____ |
| 9 | View pod logs for nginx | kubectl logs _____ |
| 10 | Execute shell in a pod | kubectl exec -it _____ - /bin/bash |

| # | TASK | COMMAND (FILL IN THE BLANKS) |
|---|---|---|
| 11 | Scale deployment app to 5 replicas | kubectl scale _____ app --replicas=5 |
| 12 | Get configmap named app-config | kubectl get _____ app-config |
| 13 | Delete a service named my-svc | kubectl delete _____ my-svc |
| 14 | Get all secrets in namespace dev | kubectl get _____ -n dev |
| 15 | Edit a deployment named web | kubectl edit _____ web |
| 16 | Apply a configmap file | kubectl apply -f _____ |
| 17 | Get events sorted by time | kubectl get events --sort-by=_____ |
| 18 | Cordon a node named worker-1 | kubectl cordon _____ |
| 19 | Drain node worker-1 | kubectl drain _____ --ignore-daemonsets |
| 20 | Uncordon node worker-1 | kubectl uncordon _____ |

## 2 COMMAND CORRECTION

Each command has an error. Rewrite it correctly.

| | INCORRECT COMMAND | CORRECT COMMAND |
|---|---|---|
| 1 | kubectl get pod -n dev | |
| 2 | kubectl describe pod nginx -n | |
| 3 | kubectl delete deployments app | |
| 4 | kubectl get services -all-namespaces | |
| 5 | kubectl Logs nginx | |
| 6 | kubectl exec -it nginx /bin/bash | |
| 7 | kubectl apply -f app.yaml | |
| 8 | kubectl scale deploy app --replica 3 | |
| 9 | kubectl get nodes -o widee | |
| 10 | kubectl cordon worker-1 now | |

| | INCORRECT COMMAND | CORRECT COMMAND |
|---|---|---|
| 11 | kubectl drain worker-1 --force | |
| 12 | kubectl get secret app-secret -n devv | |
| 13 | kubectl delete svc my-svc -n prod | |
| 14 | kubectl edit deployments/web | |
| 15 | kubectl logs -f pod/nginx | |
| 16 | kubectl get pods --namespace=prod | |
| 17 | kubectl apply -f configmap.yaml | |
| 18 | kubectl top nodesee | |
| 19 | kubectl get events --sort-by time | |
| 20 | kubectl uncordon worker-1 now | |

## 3 FAST KUBECTL EXERCISES

Read the task and write the exact kubectl command.

| # | TASK | WRITE THE COMMAND |
|---|---|---|
| 1 | List all pods in namespace staging | |
| 2 | Describe the node control-plane | |
| 3 | Get all deployments in namespace prod | |
| 4 | View logs of pod api with follow | |
| 5 | Delete pod temp-pod in namespace test | |
| 6 | Create a namespace called blue | |
| 7 | Label node worker-1 with env=prod | |
| 8 | Get all persistent volumes | |
| 9 | Get pod details with YAML output | |
| 10 | Scale deployment web to 10 replicas | |

| # | TASK | WRITE THE COMMAND |
|---|---|---|
| 11 | Get configmap app-config in namespace dev | |
| 12 | List all secrets in all namespaces | |
| 13 | Exec into pod api and run ls / | |
| 14 | Get nodes not ready | |
| 15 | Drain node worker-2 ignoring daemonsets | |
| 16 | Apply the file nginx.yaml | |
| 17 | Get events from namespace kube-system | |
| 18 | Uncordon node worker-2 | |
| 19 | Delete deployment old-web | |
| 20 | Get top pods sorted by CPU usage | |

### SPEED GOALS

Section 1: Complete in 6 minutes
Section 2: Complete in 6 minutes
Section 3: Complete in 8 minutes

Total Time Goal: 20 Minutes

### CHALLENGE RULES

- [x] No looking up commands.
- [x] Focus on speed and accuracy.
- [x] Practice under exam conditions.
- [x] Review mistakes and repeat.

### SELF SCORE

Score yourself out of 60

☆ ☆ ☆ ☆ ☆

What can you improve? _____

COMMAND SPEED + ACCURACY = SENIOR ADMIN SKILLS.

Practice daily.
Be ready for the real exam.

EXAM 6 KEY AREAS COVERED

CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# SCENARIO-BASED PRODUCTION INVESTIGATION

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

REAL-WORLD SCENARIOS. STRUCTURED INVESTIGATION. ROOT CAUSE FOCUS. PRODUCTION RECOVERY.

This page teaches you how senior administrators investigate real production issues.
Follow this proven workflow every time. Do not skip any step.

## 1 DIAGNOSE

Understand the problem and scope the impact.

- Confirm the alert
- Identify affected components
- Determine user impact
- Define time of incident
- Establish severity

## 2 COLLECT EVIDENCE

Gather facts from all relevant sources.

- Logs and events
- Metrics and dashboards
- Resource status
- Configurations
- Network and security
- Time correlation

## 3 IDENTIFY ROOT CAUSE

Analyze evidence to find the real underlying cause.

- Correlate findings
- Eliminate false causes
- Identify the root cause
- Understand why it happened
- Document the cause

## 4 IMPLEMENT FIX

Apply the right fix safely and effectively.

- Choose the correct fix
- Implement the change
- Verify immediate effect
- Monitor for side effects
- Document the action

## 5 VERIFY RECOVERY

Confirm full recovery and prevent recurrence.

- Validate functionality
- Monitor system health
- Confirm user recovery
- Update runbooks
- Create follow-up tasks

## EXAMPLE SCENARIO: WEB APPLICATION IS DOWN

### 1 DIAGNOSE

Users report 503 errors. Confirm scope and impact.

```
kubectl get pods -A
kubectl get svc -A
kubectl get ingress -A
kubectl get events -A
```

Impact:
- All users
- Web application unavailable
- Started 10 minutes ago

Severity: HIGH

### 2 COLLECT EVIDENCE

Gather data from cluster and supporting systems.

```
kubectl describe pods -n web
kubectl logs -n web <pod>
kubectl describe svc web
kubectl get endpoints web
kubectl describe ingress web
kubectl get events -n web
kubectl top pods -n web
kubectl get nodes
df -h on nodes
Check monitoring dashboards
```

Evidence collected and timestamped.

### 3 IDENTIFY ROOT CAUSE

Analyze and find the real underlying cause.

Pod in CrashLoopBackOff due to missing environment variable.

Root cause: Missing env var DATABASE_URL in deployment.

Why: Recent config change removed secrets reference.

### 4 IMPLEMENT FIX

Fix the issue and restore the service.

```
kubectl set env deployment/web DATABASE_URL=... -n web

kubectl rollout restart deployment/web -n web

Or

kubectl rollout undo deployment/web -n web
```

Monitor rollout and pod status.

### 5 VERIFY RECOVERY

Verify system is healthy and users are restored.

```
kubectl get pods -n web
kubectl get svc web
kubectl get endpoints web
```

Check application health endpoint

Check monitoring dashboards

Result:
- Pods running
- Service healthy
- Users restored

Status: RESOLVED

## INVESTIGATION CHECKLIST

- [x] Confirm the alert and scope the impact
- [x] Identify all affected components
- [x] Collect logs, events, metrics, and configs
- [x] Correlate data by timeline
- [x] Identify the root cause
- [x] Understand why it happened
- [x] Implement the safest effective fix
- [x] Verify full recovery
- [x] Document everything
- [x] Create preventive action

## ESSENTIAL INVESTIGATION TOOLS

| | |
|---|---|
| KUBECTL | Primary tool for all cluster investigation. get, describe, logs, top, events |
| LOGS | Application logs, system logs, container logs. kubectl logs, logging stack |
| METRICS | CPU, Memory, Network, Disk, Requests. Prometheus, Grafana, Metrics Server |
| EVENTS | Cluster events show what happened. kubectl get events -A --sort-by=.lastTimestamp |
| NETWORK | Network policies, DNS, Endpoints, Ingress. kubectl get all -o wide, nslookup, curl |
| STORAGE | PVC, PV, StorageClass, Volume health. kubectl get pvc,pv -A, df -h, iostat |

## KEY PRINCIPLES

- Never guess. Collect evidence.
- Follow the data. Not the assumption.
- One change at a time.
- Document everything.
- Prevention is part of the fix.

INVESTIGATE LIKE A SENIOR ADMIN

## INVESTIGATION WORKFLOW SUMMARY

1 DIAGNOSE — Understand the problem
→ 2 COLLECT EVIDENCE — Gather all the facts
→ 3 IDENTIFY ROOT CAUSE — Find the real cause
→ 4 MPLEMENT FIX — Apply the right fix
→ 5 VERIFY RECOVERY — Confirm full recovery

The best administrators don't just fix problems. They prevent the next one.

# EXAM SPEED CHALLENGE

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

SOLVE UNDER STRICT TIME LIMITS. | THINK FAST. | SCORE HIGH. | EXAM READY.

This page trains your speed, precision, and command efficiency.
Complete tasks under strict time limits and improve your exam performance.

**SPEED + ACCURACY = CKA SUCCESS**

### WHY SPEED MATTERS
In the CKA exam, time is limited. The faster you solve correctly, the higher your chance of passing.

### TOTAL TIME
**60 MINUTES**
30 TASKS
2 MINUTES PER TASK

### PASSING TARGET
Accuracy: 85%+
Efficiency: 70%+
Time: Complete all tasks within 60 mins

### THE GOAL
Be fast.
Be accurate.
Be confident.
Be a CKA.

## EXAM SPEED CHALLENGE TASKS

| # | TASK | TIME LIMIT | POINTS | # | TASK | TIME LIMIT | POINTS |
|---|---|---|---|---|---|---|---|
| 1 | Create a namespace called dev | 2 min | 2 | 16 | Create a ServiceAccount builder | 2 min | 2 |
| 2 | Create a pod nginx in dev namespace | 2 min | 2 | 17 | Create a Role pod-reader | 2 min | 2 |
| 3 | Expose nginx pod using ClusterIP service | 2 min | 2 | 18 | Bind Role pod-reader to builder | 2 min | 2 |
| 4 | Create a deployment web with 3 replicas | 2 min | 2 | 19 | Create a NetworkPolicy deny-all | 2 min | 2 |
| 5 | Scale deployment web to 5 replicas | 2 min | 2 | 20 | Allow nginx pod to access web pod | 2 min | 2 |
| 6 | Update image of deployment web | 2 min | 2 | 21 | Create an Ingress for web service | 2 min | 2 |
| 7 | Rollout status of deployment web | 2 min | 2 | 22 | Add TLS secret to Ingress | 2 min | 2 |
| 8 | Rollback deployment web to previous | 2 min | 2 | 23 | Check and fix pod not starting | 2 min | 2 |
| 9 | Create a ConfigMap app-config | 2 min | 2 | 24 | Fix ImagePullBackOff error | 2 min | 2 |
| 10 | Mount ConfigMap in a pod | 2 min | 2 | 25 | Fix CrashLoopBackOff error | 2 min | 2 |
| 11 | Create a secret db-secret | 2 min | 2 | 26 | Clean up completed pods | 2 min | 2 |
| 12 | Create a pod using secret db-secret | 2 min | 2 | 27 | Drain a node worker-1 | 2 min | 2 |
| 13 | Create a PersistentVolume pv1 | 2 min | 2 | 28 | Cordon node worker-1 | 2 min | 2 |
| 14 | Create a PersistentVolumeClaim pvc1 | 2 min | 2 | 29 | Uncordon node worker-1 | 2 min | 2 |
| 15 | Mount pvc1 in a pod | 2 min | 2 | 30 | Delete all resources in namespace dev | 2 min | 2 |

**TOTAL POINTS 60**

## SCORING SYSTEM

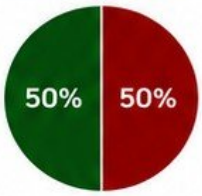
**EFFICIENCY SCORE (SPEED)**
60 POINTS
Based on time used
- 100% (60 points): Finish in 45 mins or less
- 80% (48 points): Finish in 46–55 mins
- 60% (36 points): Finish in 56–60 mins
- Below 60%: Finish over 60 mins

+

**ACCURACY SCORE (CORRECTNESS)**
60 POINTS
Based on tasks solved correctly
- 100% (60 points): 30/30 correct
- 85% (51 points): 26–29 correct
- 70% (42 points): 21–25 correct
- Below 70%: Less than 21 correct

=

## FINAL SCORE (OUT OF 120)




| GRADE | SCORE RANGE |
|---|---|
| EXCELLENT | 105 – 120 |
| GOOD | 85 – 104 |
| AVERAGE | 60 – 84 |
| NEEDS WORK | Below 60 |

## RULES

- ☑ Each task has 2 minutes.
- ☑ Do not skip tasks.
- ☑ Work in order.
- ☑ No looking up commands.
- ☑ Use kubectl describe, logs, and events when needed.
- ☑ Think, type, verify, move on.
- ☑ Speed comes with practice.

### SPEED TIPS
- Know your commands.
- Use kubectl shortcuts.
- Practice common tasks daily.
- Stay calm and focused.
- Verify quickly, move forward.

### SAMPLE TIME STRATEGY
- Tasks 1–10 : 0 – 20 minutes
- Tasks 11–20 : 20 – 40 minutes
- Tasks 21–30 : 40 – 60 minutes

Do not spend more than 2 minutes per task!

### AFTER THE CHALLENGE
- ☑ Review wrong answers.
- ☑ Understand the root cause.
- ☑ Practice again.
- ☑ Improve your weak areas.
- ☑ Repeat until you master speed + accuracy.

EXAM 6 KEY AREAS COVERED | CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS

VERIQTA
LEARN. PRACTICE. ANALYZE. SUCCEED.



# COMMON FAILURE PATTERNS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

KNOW THE MISTAKES. AVOID THE TRAPS. RECOVER FAST. PASS THE CKA.

This page highlights the most common reasons candidates fail the CKA exam, why they happen, and how to recover quickly.

**AWARENESS + PRACTICE = CKA SUCCESS**

## 1 MOST FREQUENT MISTAKES

- **Wrong Context / Namespace** — Running commands in the wrong namespace or context.
- **Incorrect Resource Type** — Using the wrong resource kind or API version.
- **YAML Syntax Errors** — Indentation, wrong fields, or invalid values in manifests.
- **Missing Labels / Selectors** — Services or Deployments not matching labels and selectors.
- **RBAC / Permission Issues** — Lack of permissions to perform actions.
- **Not Reading the Question** — Rushing and missing key requirements.
- **Time Management** — Spending too much time on one task.
- **Not Verifying the Fix** — Applying changes but not verifying properly.
- **Forgetting Clean Up** — Leaving behind resources that can affect later tasks.
- **Overcomplicating Solutions** — Using complex approaches when a simple fix works.

## 2 WHY CANDIDATES FAIL

- **Lack of Hands-On Practice** — They know theory but lack real-world kubectl experience.
- **Poor Command Recall** — Cannot remember syntax or flags under pressure.
- **Weak Troubleshooting Skills** — Do not follow a logical troubleshooting approach.
- **Ignoring Small Errors** — Small mistakes cause big failures (e.g., typos, wrong indentation).
- **Not Understanding Concepts** — Understand commands but not the underlying Kubernetes concepts.
- **Poor Time Distribution** — Spend too much time early and rush at the end.
- **Not Practicing Under Time Pressure** — Always practice slowly and fail when time is limited.
- **Not Reviewing Changes** — Make changes but do not verify results.
- **Skipping Steps** — Miss required steps in multi-part tasks.
- **Stress and Panic** — Panic leads to mistakes and bad decisions.

## 3 RECOVERY TECHNIQUES

- **Stay Calm and Focused** — Take a deep breath and focus on the task.
- **Read Carefully** — Read the question, requirements, and constraints.
- **Plan Before You Type** — Think about the steps before running commands.
- **Use Systematic Troubleshooting** — Follow a step-by-step approach: Observe → Analyze → Fix → Verify.
- **Verify Every Step** — After every change, verify the result.
- **Manage Your Time** — Allocate time per task and stick to it.
- **Use Shortcuts** — Use kubectl shortcuts you know well.
- **Check Logs and Events** — Logs and events give important clues.
- **Clean Up and Move On** — If stuck, make a safe move, clean up, and return.
- **Practice, Practice, Practice** — The best recovery is preparation and repetition.

## 4 COMMON FAILURE PATTERNS IN ACTION

| SCENARIO | WHAT HAPPENS | WHY IT HAPPENS | HOW TO RECOVER |
|---|---|---|---|
| Pod not running | Pod in CrashLoopBackOff | Wrong image, missing config, or env | Check logs → Describe pod → Fix config → Restart |
| Service not working | No endpoints | Label/selector mismatch | Check endpoints → Verify labels → Fix selector |
| Deployment not updating | Old pods still running | Wrong image tag or rollout issue | Check rollout status → Describe → Fix → Rollout restart |
| Permission denied | kubectl returns error | RBAC role/rolebinding missing | Check user permissions → Update RBAC → Retry |
| Network not reachable | Pod cannot reach service | NetworkPolicy or DNS issue | Check NetworkPolicy → Check DNS → Fix rules |
| PVC not bound | Pending PVC | StorageClass or capacity issue | Check StorageClass → Describe PVC → Fix |
| Ingress not working | 404 or timeout | Wrong rules or backend service | Check ingress → Describe → Verify service/endpoints |
| Secret not found | Pod fails to start | Secret name/namespace wrong | Check secret → Verify name/namespace → Fix |

## 5 QUICK RECOVERY CHECKLIST

- [x] Read the question carefully
- [x] Identify the problem scope
- [x] Gather information (describe, logs, events)
- [x] Analyze and identify root cause
- [x] Plan the fix before applying
- [x] Apply the correct fix
- [x] Verify the result
- [x] Check for side effects
- [x] Clean up if needed
- [x] Move to the next task

## 6 FAILURE STATS (REAL CANDIDATE TRENDS)

- 30% Time Management
- 25% Troubleshooting
- 20% Command Errors
- 15% YAML / Config Errors
- 10% Concept Gaps

## 7 REMEMBER

EVERY FAILURE IS FEEDBACK.
EVERY PRACTICE IS PROGRESS.

FAIL LESS. PRACTICE MORE. PASS CKA.

TOP PERFORMERS DON'T MAKE ZERO MISTAKES. THEY RECOVER FAST AND MOVE FORWARD.

RECOVER LIKE A SENIOR ADMIN.

EXAM 6 KEY AREAS COVERED | CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# SENIOR ADMINISTRATOR DECISION SCENARIOS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

CHOOSE THE BEST SOLUTION. | JUSTIFY DECISIONS. | EVALUATE TRADE-OFFS. | THINK LIKE A SENIOR ADMIN.

This page develops senior-level thinking for the CKA exam.
Analyze each scenario, choose the best solution, justify your decision, and evaluate trade-offs.

**SMART DECISIONS = CKA SUCCESS**

## 1 CHOOSE THE BEST SOLUTION

- Read the scenario carefully.
- Understand the environment.
- Select the most effective solution.
- Avoid Over-engineering.

## 2 JUSTIFY YOUR DECISION

- Explain why your solution is best.
- Consider reliability, security,
- performance, and simplicity.
- Use Kubernetes best practices.

## 3 EVALUATE TRADE-OFFS

- What are the benefits?
- What are the risks?
- What are the alternatives?
- What are the impacts?

## DECISION SCENARIOS

| # | SCENARIO | OPTIONS (CHOOSE ONE) | JUSTIFY YOUR DECISION | EVALUATE TRADE-OFFS |
|---|---|---|---|---|
| 1 | A stateless web application experiences high traffic spikes. How will you ensure high availability? | A. Increase replica count manually<br>B. Configure HPA with CPU metrics<br>C. Use PodDisruptionBudget only<br>D. Deploy on a single large node | | |
| 2 | A production pod keeps crashing due to OOMKilled errors. What is the best long-term solution? | A. Increase memory limit<br>B. Increase memory request<br>C. Add more replicas<br>D. Restart the pod manually | | |
| 3 | You need to expose a web application securely to the internet. What is the best approach? | A. NodePort Service<br>B. LoadBalancer Service<br>C. Ingress with TLS<br>D. ClusterIP Service | | |
| 4 | A critical database must survive node failures. What is the best deployment strategy? | A. Run as Deployment<br>B. Run as StatefulSet<br>C. Run as DaemonSet<br>D. Run as Job | | |
| 5 | You need to restrict access to namespaces for different teams. What is the best approach? | A. Use only Service Accounts<br>B. Use RBAC with Roles<br>C. Use Network Policies only<br>D. Use admission controllers only | | |
| 6 | Persistent storage is needed for multiple pods with read-write access. What is the best solution? | A. emptyDir volume<br>B. hostPath volume<br>C. ReadWriteMany PVC<br>D. ConfigMap volume | | |

## SENIOR DECISION FRAMEWORK

1. **Understand the Problem** — Analyze the requirements and constraints.
2. **Identify Options** — List possible solutions.
3. **Evaluate Each Option** — Consider pros, cons, and risks.
4. **Choose the Best Solution** — Select the option with the best overall impact.
5. **Implement and Validate** — Apply the solution and verify.
6. **Review and Improve** — Document lessons and improve for next time.

## KEY EVALUATION CRITERIA

| | |
|---|---|
| RELIABILITY | Does it improve availability and reduce risk? |
| SECURITY | Does it follow security best practices? |
| PERFORMANCE | Does it meet performance requirements? |
| SIMPLICITY | Is it simple to implement and maintain? |
| SCALABILITY | Can it scale with future growth? |

## TRADE-OFF EXAMPLES

| CHOICE | BENEFITS | TRADE-OFFS |
|---|---|---|
| Increase Replicas | High availability, better performance | Higher cost, more resources |
| Use Ingress | TLS, routing, flexibility | Additional configuration |
| Use HPA | Auto scaling, adaptability | Metrics dependency |
| Use StatefulSet | Stable identity, ordered operations | Complexity, slower scaling |
| Use Network Policies | Better security, traffic control | Can block legitimate traffic |

**SENIOR TIP:** The best administrators don't just fix problems. They make the right decisions before problems happen.

EXAM 6 KEY AREAS COVERED: CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS

# FINAL FULL-LENGTH MOCK EXAM

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

COMPLETE CKA SIMULATION. | MIXED DOMAINS. | REAL EXAM TIMING. | EXAM READY.

This is your final full-length mock exam.
Simulate the real CKA environment, time, and difficulty.

**PASS THE MOCK.**
**PASS THE CKA.**

## EXAM OVERVIEW

| TIME | TASKS | TOTAL POINTS | PASSING SCORE | ENVIRONMENT |
|---|---|---|---|---|
| 180 MINUTES (3 HOURS) | 20 TASKS APPROX. | 100 POINTS | 66%+ (66/100) | REAL EXAM SIMULATION |

## REAL EXAM CONDITIONS

- ☑ No help from others.
- ☑ No internet or external resources.
- ☑ Only kubectl, editor, and the exam environment.
- ☑ Solve problems in the cluster provided.
- ☑ Time management is critical.

## DOMAIN DISTRIBUTION

| # | DOMAIN | WEIGHT | POINTS | TOPICS COVERED |
|---|---|---|---|---|
| 1 | Cluster Architecture & Installation | 25% | 25 | Control Plane, Nodes, etcd, Networking, HA, Components |
| 2 | Workloads & Scheduling | 25% | 25 | Pods, Deployments, DaemonSets, Jobs, CronJobs, Scheduling |
| 3 | Services & Networking | 20% | 20 | Services, DNS, Ingress, NetworkPolicies, Endpoints, CoreDNS |
| 4 | Storage | 10% | 10 | Volumes, PV, PVC, StorageClasses, ConfigMaps, Secrets |
| 5 | Troubleshooting | 20% | 20 | Logs, Events, Debugging, Failure Analysis, Performance |
| | TOTAL | 100% | 100 | |

## EXAM STRATEGY

1. Read every task carefully.
2. Plan your approach before executing.
3. Solve easier tasks first.
4. Do not spend too much time on one task.
5. Verify your work before moving on.
6. Manage your time: 9 minutes per task.
7. Leave time for review at the end.

## TIME MANAGEMENT GUIDE

- Solve Tasks (150 min)
- Review (20 min)
- Buffer (10 min)

Recommended: 9 minutes per task

## TASK BREAKDOWN (EXAMPLE)

| CLUSTER ARCHITECTURE & INSTALLATION (5) | WORKLOADS & SCHEDULING (5) | SERVICES & NETWORKING (4) | STORAGE (2) | TROUBLESHOOTING (4) |
|---|---|---|---|---|
| • Cluster components | • Deployments | • ClusterIP / NodePort | • PV / PVC | • Pod failures |
| • kubeadm installation | • StatefulSets | • LoadBalancer | • StorageClasses | • Service failures |
| • Control plane setup | • DaemonSets | • Ingress resources | • Volume types | • Network issues |
| • Node joining | • Jobs & CronJobs | • NetworkPolicies | • ConfigMaps | • Storage issues |
| • Cluster upgrades | • Resource management | • DNS resolution | • Secrets | • Logs and events |

## BEFORE YOU START

- ☑ Ensure you understand the environment.
- ☑ Verify cluster access and context.
- ☑ Check available resources and versions.
- ☑ Plan your time and approach.
- ☑ Stay calm, focused, and confident.

## EXAM CHECKLIST

- ☑ Read all tasks first.
- ☑ Understand requirements.
- ☑ Plan the solution.
- ☑ Implement and test.
- ☑ Verify and validate.
- ☑ Review before submission.

## SCORING GUIDE

| SCORE RANGE | RESULT |
|---|---|
| 90 – 100 | EXCELLENT |
| 75 – 89 | GOOD |
| 66 – 74 | PASS |
| 0 – 65 | FAIL |

★ THIS IS YOUR FINAL TEST. TREAT IT LIKE THE REAL CKA EXAM.

PREPARE. EXECUTE. SUCCEED.

EXAM 6 KEY AREAS COVERED

CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta

# FINAL MOCK EXAM SOLUTIONS

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

CORRECT ANSWERS. CLEAR EXPLANATIONS. REAL CKA APPROACH. MASTER EVERY DOMAIN.

This page provides the correct solutions and explanations for the Final Full-Length Mock Exam. Review every answer, understand the reasoning, and strengthen your CKA readiness.

KNOW IT. OWN IT. PASS THE CKA.

## FINAL MOCK EXAM SOLUTIONS

| # | SCENARIO SUMMARY | CORRECT SOLUTION | EXPLANATION | KEY CONCEPT |
|---|---|---|---|---|
| 1 | Stateless web app experiencing high traffic spikes. Ensure high availability. | B. Configure HPA with CPU metrics | HPA automatically scales pods based on CPU usage, maintaining availability during spikes. | Workloads & Autoscaling |
| 2 | Production pod keeps crashing due to OOMKilled errors. | A. Increase memory limit | OOMKilled means the container exceeded its memory limit. Increase the limit to prevent termination. | Workloads & Resources |
| 3 | Expose a web application to the internet securely. | C. Ingress with TLS | Ingress with TLS provides secure external access using HTTPS with proper routing. | Services & Networking |
| 4 | Critical database must survive node failures. | B. Run as StatefulSet | StatefulSet provides stable identity, persistent storage, and ordered operations for stateful applications. | Workloads & Stateful Apps |
| 5 | Restrict access to namespaces for different teams. | B. Use RBAC with Roles | RBAC with Roles allows namespace-level access control for teams. | Security & Auth |
| 6 | Persistent storage needed for multiple pods with RWX access. | B. ReadWriteMany PVC | ReadWriteMany allows multiple pods to mount the same volume with RWX access. | Storage |
| BONUS | Cluster upgrade required with zero downtime. | Rolling upgrade with maxUnavailable=1 | Rolling upgrade updates nodes one at a time, keeping the cluster available. | Cluster Operations |

## KEY COMMAND EXAMPLES USED IN THE EXAM

**INSPECT & DEBUG**

```
kubectl get pods -A
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns>
kubectl get events -A
  --sort-by=.lastTimestamp
```

**DEPLOY & SCALE**

```
kubectl apply -f <file>.yaml
kubectl scale deployment <name>
  --replicas=5 -n <ns>
kubectl autoscale deployment <name>
  --cpu-percent=70 --min=2 --max=10
```

**NETWORKING**

```
kubectl expose deployment <name>
  --type=ClusterIP --port=80
kubectl get ingress -A
kubectl describe ingress <name>
  -n <ns>
```

**STORAGE**

```
kubectl get pvc -A
kubectl describe pv <pv-name>
kubectl patch pvc <pvc> -p
"{"spec"{"resources"{"requests"
 "{"storage":"10Gi"}}}}"
```

**SECURITY**

```
kubectl get roles -A
kubectl get rolebindings -A
kubectl auth can-i get pods
--as=system:serviceaccount:<ns>:<sa>
  -n <ns>
```

## WHAT THIS EXAM TESTS

- Real-world troubleshooting
- Command line proficiency
- Cluster administration
- Security and access control
- Networking and routing
- Storage and persistence
- Monitoring and logging
- Best practices and efficiency
- Time management under pressure
- Problem-solving and decision making

## KEY TAKEAWAYS

- Understand the problem first.
- Identify the root cause.
- Choose the simplest effective solution.
- Apply and validate using Kubernetes tools.
- Verify results and prevent recurrence.
- Document your actions.
- Practice speed with accuracy.
- Think like a senior administrator.

## SCORE GUIDE

| SCORE (%) | RANGE (POINTS) | RESULT |
|---|---|---|
| 90 – 100 | 90 – 100 | EXCELLENT |
| 75 – 89 | 75 – 89 | GOOD |
| 60 – 74 | 60 – 74 | PASS |
| 40 – 59 | 40 – 59 | NEEDS WORK |
| 0 – 39 | 0 – 39 | FAIL |

REVIEW. PRACTICE. IMPROVE. REPEAT. THAT IS HOW SENIOR ADMINISTRATORS MASTER KUBERNETES.

EXAM 6 KEY AREAS COVERED | CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS

# FINAL PERFORMANCE REPORT

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

KNOW YOUR PERFORMANCE. FIND YOUR GAPS. IMPROVE SMART. PASS THE CKA.

This is your Final Mock Exam Performance Report. Use it to understand your strengths, address weaknesses, and follow the improvement plan to pass the real CKA exam.

ANALYZE. IMPROVE. SUCCEED.

## OVERALL PERFORMANCE SUMMARY

| TOTAL SCORE | PASSING SCORE | RESULT | TIME TAKEN |
|---|---|---|---|
| 84 / 100 | 66 / 100 | PASS | 162 MIN |
| 84% | 66% | EXAM READY | (90% OF TIME) |

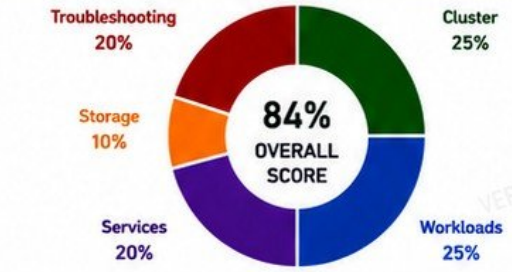
## EXAM INFORMATION

| | |
|---|---|
| Exam Type: | Full-Length Mock Exam |
| Duration: | 180 Minutes (3 Hours) |
| Tasks Attempted: | 20 / 20 |
| Completed On: | _____ |

## DOMAIN PERFORMANCE SCORES

| # | DOMAIN | WEIGHT | SCORE (%) | POINTS EARNED | PERFORMANCE |
|---|---|---|---|---|---|
| 1 | Cluster Architecture & Installation | 25% | 88% | 22 / 25 | EXCELLENT |
| 2 | Workloads & Scheduling | 25% | 80% | 20 / 25 | GOOD |
| 3 | Services & Networking | 20% | 85% | 17 / 20 | EXCELLENT |
| 4 | Storage | 10% | 70% | 7 / 10 | GOOD |
| 5 | Troubleshooting | 20% | 90% | 18 / 20 | EXCELLENT |
| | TOTAL | 100% | 84% | 84 / 100 | PASS |

## SCORE BREAKDOWN




## WEAKNESSES

- Storage topics need more practice (PV, PVC, StorageClasses).
- Improve StatefulSet and DaemonSet configuration speed.
- Workload scaling and resource management can be faster.
- Time management in multi-step tasks needs improvement.
- Double-check YAML syntax before applying manifests.

## STRENGTHS

- Strong troubleshooting skills and fast problem diagnosis.
- Excellent understanding of cluster architecture.
- Good performance in services and networking.
- Consistent command line proficiency.
- Efficient use of time and task prioritization.

## KEY TAKEAWAYS

- You understand Kubernetes core concepts well.
- You can solve complex problems under pressure.
- Focus on weak domains for maximum improvement.
- Practice more real-world troubleshooting scenarios.
- Review and repeat until concepts are second nature.

## IMPROVEMENT PLAN

| DOMAIN | CURRENT SCORE | GOAL SCORE | ACTIONS | RESOURCES | TARGET DATE |
|---|---|---|---|---|---|
| Cluster Architecture & Installation | 88% | 95%+ | Practice HA, upgrades, kubeadm, components | CKA Handbook, Labs | _____ |
| Workloads & Scheduling | 80% | 90%+ | Practice Deployments, Jobs, Controllers | CKA Handbook, Labs | _____ |
| Services & Networking | 85% | 90%+ | Practice Ingress, NetworkPolicies, DNS | CKA Handbook, Labs | _____ |
| Storage | 70% | 90%+ | Practice PV, PVC, StorageClasses, Snapshots | CKA Handbook, Labs | _____ |
| Troubleshooting | 90% | 95%+ | Practice logs, debug, root cause analysis | CKA Handbook, Labs | _____ |

## NEXT STEPS

1. Review all incorrect answers and explanations.
2. Practice weak domains daily.
3. Re-take timed mock exams.
4. Focus on speed, accuracy, and confidence.
5. Stay consistent and keep improving.

## PERFORMANCE TREND

## FINAL MESSAGE

Great work! You are on the right path.
Fix your gaps, strengthen your weak areas, and you will pass the real CKA exam.

BELIEVE. PREPARE. SUCCEED.

SUCCESS IS THE RESULT OF CONSISTENT PRACTICE AND INTELLIGENT IMPROVEMENT.

YOU ARE ALMOST THERE. FINISH STRONG!

EXAM 6 KEY AREAS COVERED

CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS

VERIQTA

VERIQTA
LEARN. PRACTICE. ANALYZE. SUCCEED.



# LAST-HOUR MOCK REVISION

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

FOCUS. RECALL. | EXECUTE FAST. | BE CONFIDENT. | PASS THE CKA.

This page is your last-hour revision guide. Review high-yield commands, common tasks, critical troubleshooting, and final reminders to pass the CKA exam.

STAY CALM. STAY FOCUSED. YOU'VE GOT THIS!

## 1 HIGH-YIELD COMMANDS (MUST KNOW)

**Get Resources**
```
kubectl get all -A
kubectl get pods -A -o wide
```

**Describe & Details**
```
kubectl describe pod <pod>
kubectl describe node <node>
```

**Logs & Debug**
```
kubectl logs <pod> -n <ns>
kubectl logs <pod> -c <container> -f
```

**Exec & Access**
```
kubectl exec -it <pod> -n <ns> -- /bin/sh
kubectl exec -it <pod> -- bash
```

**Apply & Edit**
```
kubectl apply -f <file.yaml>
kubectl edit <resource> <name>
```

**Delete & Clean**
```
kubectl delete pod <pod> -n <ns>
kubectl delete -f <file.yaml>
```

**Config & Context**
```
kubectl config get-contexts
kubectl config use-context <ctx>
```

**Namespace**
```
kubectl get ns
kubectl create ns <name>
```

**Nodes**
```
kubectl get nodes
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
```

**Top & Metrics**
```
kubectl top nodes
kubectl top pods -A
```

**Events**
```
kubectl get events -A --sort-by=.lastTimestamp
```

**Cluster Info**
```
kubectl cluster-info
kubectl get componentstatuses
```

## 2 COMMON TASKS (HIGH FREQUENCY)

- ☑ **Create a Deployment**
  ```
  kubectl create deployment <name> --image=<image>
  kubectl scale deployment <name> --replicas=<n>
  ```
- ☑ **Expose a Deployment**
  ```
  kubectl expose deployment <name>
  --type=ClusterIP|NodePort|LoadBalancer --port=80
  --target-port=80
  ```
- ☑ **Create a Namespace**
  ```
  kubectl create namespace <name>
  ```
- ☑ **Create a Pod**
  ```
  kubectl run <pod> --image=<image> --restart=Never
  ```
- ☑ **Create a ConfigMap**
  ```
  kubectl create configmap <name> --from-literal=key=value
  ```
- ☑ **Create a Secret**
  ```
  kubectl create secret generic <name> --from-literal=key=value
  ```
- ☑ **Setup Ingress**
  ```
  kubectl apply -f ingress.yaml
  ```
- ☑ **Set Resource Limits**
  ```
  resources:
    limits:
      cpu: "500m"
      memory: "256Mi"
  ```
- ☑ **Rolling Update**
  ```
  kubectl set image deployment/<name> <container>=<image>:<tag>
  kubectl rollout status deployment/<name>
  ```

## 3 CRITICAL TROUBLESHOOTING CHECKLIST

| PROBLEM | WHAT TO CHECK | COMMANDS |
|---|---|---|
| Pod Not Running | Image, Events, Logs, Resource limits, Node | kubectl get pods -A<br>kubectl describe pod <pod><br>kubectl logs <pod> |
| CrashLoopBackOff | Logs, Command, Probes, Resources | kubectl describe pod <pod><br>kubectl logs <pod> --previous |
| ImagePullBackOff | Image name, Registry, Pull secret | kubectl describe pod <pod><br>kubectl get secret |
| Service Not Working | Endpoints, Selector, Pods | kubectl get svc<br>kubectl get endpoints <svc> |
| Ingress Not Working | Rules, Backend, Service, TLS | kubectl get ingress -A<br>kubectl describe ingress <ingress> |
| No Internet / DNS | CoreDNS, NetworkPolicy, DNS | kubectl get pods -n kube-system -l k8s-app=kube-dns<br>kubectl exec -it <pod> -- nslookup kubernetes.default |
| Node Not Ready | Kubelet, Network, Disk, Resource | kubectl describe node <node><br>journalctl -u kubelet |

## 4 FINAL REMINDERS

- Read the question carefully. Understand exactly what is required.
- Plan before you execute. Think about the steps.
- Use the simplest method. Do not over-engineer solutions.
- Manage your time. Do not spend too long on one task.
- Verify your work. Validate the solution.
- Stay calm and confident. You are ready. You can pass!

## 5 LAST-MINUTE QUICK REFERENCE (MEMORIZE THESE)

**POD LIFE CYCLE**
- Pending
- Running
- Succeeded
- Failed
- Unknown

**RESTART POLICIES**
- Always
- OnFailure
- Never

**SERVICE TYPES**
- ClusterIP
- NodePort
- LoadBalancer
- ExternalName

**VOLUME TYPES**
- emptyDir
- hostPath
- ConfigMap
- Secret
- PVC

**UPDATE STRATEGIES**
- Rolling Update
- Recreate

## PORTS TO REMEMBER

| | |
|---|---|
| API Server | 6443 |
| Kubelet | 10250 |
| kube-scheduler | 10251 |
| kube-controller | 10252 |
| etcd | 2379-2380 |
| NodePort Range | 30000-32767 |

## FINAL CHECKLIST

- ☑ Understand all domains
- ☑ Practice high-yield tasks
- ☑ Review command syntax
- ☑ Solve mock scenarios
- ☑ Stay confident
- ☑ PASS THE CKA!

TRUST YOUR PREPARATION. EXECUTE WITH FOCUS. PASS WITH CONFIDENCE. | YOU ARE A FUTURE KUBERNETES ADMINISTRATOR!

EXAM 6 KEY AREAS COVERED: CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS



veriqta youtube.com/@veriqta | veriqta github.com/veriqta | veriqta linkedin.com/company/veriqta | veriqta x.com/veriqta | veriqta instagram.com/veriqta | veriqta t.me/veriqta

# CKA CONFIDENCE ASSESSMENT

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

ASSESS YOUR READINESS. KNOW YOUR LEVEL. BUILD CONFIDENCE. PASS THE CKA.

This page helps you evaluate your readiness for the CKA exam. Be honest with yourself, identify gaps, and decide if you are ready to take the exam.

SELF-AWARENESS LEADS TO SUCCESS.

## 1 SELF-EVALUATION CHECKLIST

Rate yourself honestly (1 = Weak, 5 = Strong)

| # | DOMAIN | RATING (1-5) |
|---|---|---|
| 1 | Cluster Architecture & Installation | 1 2 3 4 5 |
| 2 | Workloads & Scheduling | 1 2 3 4 5 |
| 3 | Services & Networking | 1 2 3 4 5 |
| 4 | Storage | 1 2 3 4 5 |
| 5 | Troubleshooting | 1 2 3 4 5 |

### CONFIDENCE INDICATOR

| | | |
|---|---|---|
| 4 - 5 | Strong | You understand and can perform under pressure |
| 2 - 3 | Average | You need more practice and repetition |
| 1 | Weak | You need focused study and hands-on practice |

## 2 READINESS SCORECARD

| SCORE RANGE | READINESS LEVEL | DESCRIPTION | ACTION |
|---|---|---|---|
| 90 - 100 | EXCELLENT | You are well prepared and confident. | Take the exam with confidence. |
| 75 - 89 | GOOD | You are almost ready with minor gaps. | Review weak areas and retest. |
| 60 - 74 | NEEDS IMPROVEMENT | You have noticeable gaps that need attention. | Focus on weak domains and hands-on practice. |
| 40 - 59 | NOT READY | You lack understanding in multiple areas. | Do not take the exam yet. Study and practice more. |
| 0 - 39 | NOT READY | You are not prepared for the exam. | Start from the beginning. Build strong foundation. |

YOUR TOTAL SCORE: _____ / 100

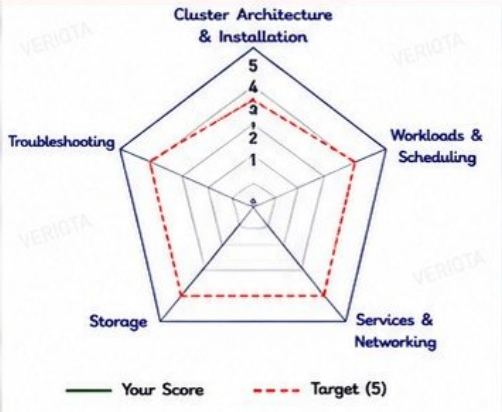
YOUR READINESS LEVEL: _____

## 3 GO / NO-GO ASSESSMENT

Answer the questions honestly (Yes / No)

| # | QUESTION | YES | NO |
|---|---|---|---|
| 1 | Can you install and configure a Kubernetes cluster from scratch? | ☐ | ☐ |
| 2 | Can you deploy, update, and roll back applications? | ☐ | ☐ |
| 3 | Can you configure Services, Ingress, and NetworkPolicies? | ☐ | ☐ |
| 4 | Can you manage persistent storage and troubleshoot storage issues? | ☐ | ☐ |
| 5 | Can you troubleshoot pod, node, network, and storage issues effectively? | ☐ | ☐ |
| 6 | Can you work efficiently under time pressure? | ☐ | ☐ |
| 7 | Have you completed at least 3 full-length mock exams? | ☐ | ☐ |
| 8 | Do you feel confident in all CKA domains? | ☐ | ☐ |

## 4 DOMAIN CONFIDENCE RADAR




## 5 STRENGTHS & WEAKNESSES

STRENGTHS (Top 2 Domains)

1. _____
2. _____

WEAKNESSES (Top 2 Domains)

1. _____
2. _____

### IMPROVEMENT FOCUS

Focus on your weak areas first. Practice until you can perform tasks without looking up answers.

## 6 FINAL DECISION

Based on your score and self-assessment

### GO

- Your score is 75 or higher
- You understand most domains
- You are confident in your skills

You are ready to take the CKA exam!

OR

### NO-GO

- Your score is below 75
- You have significant weak areas
- You are not confident yet

Do not take the exam yet. Study, practice, and retake the assessment.

CONFIDENCE COMES FROM PREPARATION. ASSESS. IMPROVE. SUCCEED.

YOU CAN PASS THE CKA!

EXAM 6 KEY AREAS COVERED

CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS

# FINAL VERIQTA EXAM SUCCESS FRAMEWORK

CKA
CERTIFIED KUBERNETES ADMINISTRATOR

PREPARE DEEPLY. EXECUTE SMARTLY. VERIFY EVERYTHING. PASS THE CKA.

This is your complete success framework. Follow it on exam day, manage your time, verify your work, and apply senior administrator principles to PASS the CKA exam.

**FOCUS. EXECUTE. VERIFY. SUCCEED.**

## 1 EXAM-DAY WORKFLOW

1. **Read Everything** — Read all instructions, tasks, and questions carefully.
2. **Plan Your Approach** — Identify tasks, estimate time, and set your strategy.
3. **Execute Systematically** — Solve tasks one by one using the simplest effective method.
4. **Verify Your Work** — Validate configuration, test, and confirm results.
5. **Review & Final Check** — Use remaining time to re-check and catch mistakes.
6. **Submit With Confidence** — Ensure all tasks are complete and submitted successfully.

## 2 TIME MANAGEMENT BLUEPRINT

- Planning 15 min (8%)
- Execution 130 min (72%)
- Verification 25 min (14%)
- Buffer 10 min (6%)

| PHASE | TIME | FOCUS | KEY ACTIONS |
|---|---|---|---|
| Planning | 15 min | Understand | Read, plan, prioritize |
| Execution | 130 min | Build & Solve | Implement solutions |
| Verification | 25 min | Validate | Test, verify, fix issues |
| Buffer | 10 min | Flex Time | Unplanned tasks / complex fixes |

### TIME EXECUTION RULES

- ☑ Do not spend too much time on one task.
- ☑ Move forward and return if needed.
- ☑ Use buffer time for unexpected issues.
- ☑ Keep at least 15 minutes for final review.

## 3 VERIFICATION CHECKLIST

- ☑ **Resources Exist** — Pods, Services, ConfigMaps, Secrets, PVCs, Deployments are created.
- ☑ **Configuration Correct** — Labels, selectors, ports, paths, policies, and references are correct.
- ☑ **Functionality Validated** — Test connectivity, endpoints, and application behavior.
- ☑ **Security Applied** — RBAC, NetworkPolicies, Secrets, and security settings verified.
- ☑ **High Availability Ready** — Replica count, probes, anti-affinity, and HA configurations checked.
- ☑ **Clean & Optimized** — No unnecessary resources, proper namespaces and hygiene.
- ☑ **All Tasks Complete** — No task left behind. All requirements fully satisfied.
- ☑ **Final Review Done** — Quick review of all solutions before submission.

## 4 SENIOR ADMINISTRATOR PRINCIPLES

- **Think Like a Cluster Owner** — Focus on reliability, security, scalability, and maintainability.
- **Choose Simplicity First** — Use the simplest solution that meets the requirement.
- **Automate When Possible** — Reduce manual steps and eliminate human error.
- **Design for Failure** — Assume failure and build for resilience and recovery.
- **Observe Before Acting** — Understand the problem fully before making changes.
- **Validate Continuously** — Test early, test often, and verify end-to-end.
- **Document Mentally** — Keep track of what you built and why.
- **Time is a Resource** — Manage time like any other critical resource.

## 5 FINAL MOTIVATION & SUCCESS ROADMAP

### SUCCESS MINDSET

- ✓ I am prepared.
- ✓ I understand the concepts.
- ✓ I know the commands.
- ✓ I can troubleshoot.
- ✓ I can solve problems.
- ✓ I stay calm and focused.
- ✓ I execute with confidence.
- ✓ I will PASS the CKA!

### 90-DAY SUCCESS ROADMAP

| Foundation (Days 1-15) | Practice (Days 16-45) | Advanced (Days 46-75) | Mastery (Days 76-90) | Exam Day VICTORY! |
|---|---|---|---|---|
| • Core concepts | • Hands-on labs | • Complex tasks | • Mock exams | • Execute |
| • Basic commands | • Scenarios | • Production patterns | • Weak areas | • Verify |
| • Cluster basics | • Troubleshooting | • Speed & accuracy | • Final polish | • PASS |

> "Preparation meets opportunity.
> "Execution creates results."
> "Verified work earns success."

### EXAM-DAY COMMANDMENT

- ★ Stay calm. You've got this.
- ★ Read everything carefully.
- ★ Plan before you start.
- ★ Use the simplest method.
- ★ Verify every solution.
- ★ Manage your time.
- ★ Do not panic.
- ★ Trust your preparation.
- ★ Finish strong.

**PASS THE CKA!**

YOU HAVE DONE THE WORK. NOW GO EARN YOUR CERTIFICATION.

BELIEVE. EXECUTE. SUCCEED. BECOME A SENIOR KUBERNETES ADMINISTRATOR!

EXAM 6 KEY AREAS COVERED | CORE CONCEPTS | TROUBLESHOOTING SKILLS | COMMAND MASTERY | SPEED & ACCURACY | REAL-WORLD SCENARIOS | EXAM READINESS



veriqta youtube.com/@veriqta
veriqta github.com/veriqta
veriqta linkedin.com/company/veriqta
veriqta x.com/veriqta
veriqta instagram.com/veriqta
veriqta t.me/veriqta